

MODULE NAME: DATABASE ADMINISTRATION

MODULE CODE: ITLAD601

COMPETENCE: Administer Oracle Database

RTQF LEVEL: 6

CREDITS: 10

SECTOR: ICT

SU-SECTOR: IT

MODULE LEADER: Ephrem Niyigaba & Gabriel Nishimwe

Purpose statement

Database is a fundamental component of every IT system, it stores data, therefore is very critical. This module describes the skills and knowledge required to administer Oracle database with data safety in mind. At the completion of this module the trainee will be able to start-up a database, secure Oracle database, manage database and document database maintenance tasks.

Table of Contents

LEARNING UNIT 1: DESCRIBE ORACLE DATABASE ARCHITECTURE.....	10
Learning Outcome 1.1: Describe oracle architecture components.....	10
1.1.1. Describing Oracle Database Definition and terminologies.....	10
1.1.1.1. Definition of Oracle Database.....	10
1.1.1.2. Database instance.....	10
1.1.1.3. Tablespace.....	11
1.1.1.4. Default tablespace.....	11
1.1.1.5. Temporary tablespace.....	12
1.1.1.6. Undo tablespace.....	12
1.1.1.7. Rollback.....	12
1.1.1.8. Redo log files.....	13
1.1.1.9. Archived redo log files.....	13
1.1.1.10. Temporary tables.....	14
1.1.1.11. Backups.....	14
1.1.2. Describing oracle database management system.....	15
1.1.2.1. Overview of Database Management Systems (DBMS).....	15
1.1.2.1.1. Key components of Oracle DBMS.....	15
1.1.2.1.2. Why does Oracle stand out?.....	15
1.1.2.2. Overview of transaction management.....	15
1.1.2.2.1. ACID properties in Oracle.....	16
1.1.2.2.2. Oracle-specific insight.....	16
1.1.2.3. Overview of data integrity.....	16
1.1.2.4. Overview of data security.....	17
1.1.3. Describing oracle database architecture.....	18
1.1.3.1. Overview of oracle grid architecture.....	18
1.1.3.1.1. Oracle clusterware.....	18
1.1.3.1.2. Oracle ASM (Automatic Storage Management).....	19
1.1.3.1.3. Oracle RAC (Real Application Clusters).....	19
1.1.3.1.4. Cluster (in Oracle context).....	20
1.1.3.2. Overview of application architecture.....	20
1.1.3.3. Overview of physical database structures.....	21
1.1.3.3.1. CDB (Container Database).....	21
1.1.3.4. Overview of logical database structures.....	22
1.1.3.5. Overview of the Oracle data dictionary.....	22
1.1.3.6. Overview of the Oracle instance.....	23
Exercises 1.1.....	24
Learning Outcome 1.2: Describe oracle memory structures and background processes.....	25

1.2.1. Describing user related processes.....	25
1.2.1.1. User Process.....	25
1.2.1.2. Server Process.....	26
1.2.1.3. Program Global Area (PGA).....	26
1.2.1.3.1. Contents of the PGA.....	27
1.2.1.3.2. Behavior in different architectures.....	27
1.2.2. Describing memory structure.....	28
1.2.2.1. System Global Area (SGA) overview.....	28
(Oracle, 2002).....	29
1.2.2.2. Required SGA areas.....	29
1.2.2.2.1. Database buffer cache.....	30
1.2.2.2.2. Redo log buffer.....	30
1.2.2.2.3. Shared pool.....	30
1.2.2.3. Optional SGA areas.....	31
1.2.2.3.1. Java pool.....	31
1.2.2.3.2. Large pool.....	31
1.2.2.4. Managing the SGA.....	31
1.2.3. Describing background processes.....	32
1.2.3.1. Database Writer Process (DBWn).....	32
1.2.3.2. Log Writer Process (LGWR).....	33
1.2.3.3. Archiver Process (ARCn).....	33
1.2.3.4. Checkpoint Process (CKPT).....	33
1.2.3.5. Process Monitor (PMON).....	34
1.2.3.6. System Monitor (SMON).....	34
1.2.3.7. Recover Process (RECO).....	34
Exercises 1.2.....	35
Learning Outcome 1.3: Describe oracle logical and physical storage structures.....	36
1.3.1. Describing logical storage of oracle database.....	36
1.3.1.1. Definition of tablespace.....	36
1.3.1.2. Types of tablespaces.....	37
1.3.1.2.1. SYSTEM tablespace.....	37
1.3.1.2.2. SYSAUX Tablespace.....	37
1.3.1.2.3. EXAMPLE Tablespace.....	37
1.3.1.2.4. TEMP Tablespace.....	38
1.3.1.2.5. UNDOTBS1 Tablespace.....	38
1.3.1.2.6. USERS Tablespace.....	39
1.3.2. Describing physical storage of oracle database.....	39
1.3.2.1. Control file.....	39

1.3.2.2. Data files.....	40
1.3.2.3. Redo log files.....	40
1.3.2.4. Parameter files.....	41
1.3.2.5. Password file.....	42
1.3.2.6. Backup files.....	42
1.3.2.7. Archived redo log files.....	43
1.3.3. Describing Client-Server model of oracle engine.....	44
1.3.3.1. Components of oracle client-server architecture.....	44
1.3.3.2. How client-server interaction works.....	44
1.3.3.3. Types of server processes.....	45
1.3.3.4. Differences in memory allocation between dedicated and shared servers.....	45
1.3.3.5. Lifecycle of a database session.....	45
1.3.3.5.1. Steps in a database session lifecycle.....	46
Exercises 1.3.....	47
LEARNING UNIT 2: MANAGE DATABASE.....	49
Learning Outcome 2.1 : Setup Oracle Database Environment.....	49
2.1.1. Installing oracle database.....	49
2.1.1.1. Installing oracle database software.....	50
Step 1: Download Required Software.....	50
Step 2: Install VirtualBox on Host Machine.....	50
Step 3: Create a New Virtual Machine.....	51
Step 4: Install Oracle Linux in VirtualBox.....	51
Step 5: Activate Network and Update OS.....	51
Step 6: Configure Shared Folder (Optional but Recommended).....	52
2.1.1.2. Planning an oracle database and oracle grid for standalone server installation..	52
2.1.1.3. Operating system groups and users required.....	53
2.1.1.4. Configuring Oracle Software owner environment (as root user).....	53
Step 1: Create Oracle groups.....	53
Step 2: Create Oracle and grid users.....	54
Step 3: Create directories.....	54
Step 4: Configure environment variables.....	54
Environment Setup for Grid (ASM).....	54
Environment Setup for Oracle (Database).....	55
Step 7: Install Oracle Database 19c.....	56
Create temporary staging directories.....	57
Copy the ZIP files to the staging directories.....	57
Extract the ZIP files.....	57

2.2.1.3 Startup OPEN.....	120
2.2.1.4 Startup FORCE.....	121
2.2.1.5 Startup RESTRICT.....	121
2.2.1.6 Starting Oracle Restart.....	121
2.2.2. Shutting Down an Oracle Database.....	121
2.2.2.1 Shutdown NORMAL.....	121
2.2.2.2 Shutdown TRANSACTIONAL.....	122
2.2.2.3 Shutdown IMMEDIATE.....	122
2.2.2.4 Shutdown ABORT.....	122
2.2.3. Shutting Down the Automatic Storage Management (ASM) Instance.....	123
2.2.4. Conceptual Flow.....	123
Exercise 2.2.....	123
Learning Outcome 2.3: Monitor database start-up and operation for irregularities.....	125
2.3.1. Configuring Automatic Restart of an Oracle Database.....	125
2.3.1.1 Configuring Oracle Restart.....	125
2.3.1.2 Starting and Stopping Components Managed by Oracle Restart.....	126
2.3.1.3 SRVCTL Command Reference for Oracle Restart.....	126
2.3.1.4 CRSCTL Command Reference.....	126
2.3.2. Managing Processes.....	127
2.3.2.1 Configuring Oracle Database for Shared Server.....	127
2.3.2.2 Configuring Database Resident Connection Pooling (DRCP).....	127
2.3 Terminating Sessions.....	127
2.3.3. Managing Memory.....	128
2.3.3.1 Using Automatic Memory Management (AMM).....	128
2.3.3.2 Configuring Memory Manually.....	128
2.3.3.3 Configuring Database Smart Flash Cache.....	128
2.3.4. Monitoring Database Operations.....	129
2.3.4.1 Monitoring Errors and Alerts.....	129
2.3.5. Conceptual Overview.....	129
Exercise 2.3.....	130
Learning Outcome 2.4: Manage Oracle storage Structure.....	131
2.4.1. Viewing Database Storage Structure Information.....	131
2.4.1.1 Viewing Control File Information.....	131
2.4.1.2 Viewing Online Redo Log File Information.....	131
2.4.1.3 Viewing Archived Redo Log File Information.....	132
2.4.1.4 Viewing Data File Information.....	132
2.4.1.5 Viewing Tablespace Information.....	132
2.4.2. Managing Tablespaces.....	132

2.4.2.1 Creating a Tablespace.....	133
2.4.2.2 Modifying a Tablespace.....	133
2.4.2.3 Dropping a Tablespace.....	133
2.4.3. Managing the Online Redo Log.....	133
2.4.3.1 Creating Online Redo Log Groups and Members.....	133
2.4.3.2 Relocating and Renaming Online Redo Log Members.....	134
2.4.3.3 Dropping Online Redo Log Groups and Members.....	134
2.4.3.4 Forcing Log Switches.....	134
2.4.3.5 Clearing an Online Redo Log File.....	134
Exercise 2.4.....	135
Learning Outcome 2.5: Manage Schema Objects.....	136
2.5.1. Introduction to Schema.....	136
2.5.1.1 Naming Schema Objects.....	136
2.5.2. Managing Tables.....	136
2.5.2.1 Viewing Tables.....	136
2.5.2.2 Viewing Table Data.....	136
2.5.2.3 Creating a Table.....	136
2.5.2.4 Modifying Table Attributes.....	137
2.5.2.5 Loading Data into a Table.....	137
2.5.2.6 Dropping a Table.....	137
2.5.3. Managing Indexes.....	137
2.5.3.1 Viewing Indexes.....	138
2.5.3.2 Creating an Index.....	138
2.5.3.3 Deleting an Index.....	138
2.5.4. Managing Views.....	138
2.5.4.1 Displaying Views.....	138
2.5.4.2 Creating a View.....	138
2.5.4.3 Deleting a View.....	138
2.5.5. Working with Other Schema Objects.....	138
2.5.5.1 Sequences.....	138
2.5.5.2 Synonyms.....	139
Exercise 2.5.....	139
LEARNING UNIT 3: SECURE ORACLE DATABASE.....	142
Learning Outcome 3.1: Administer user account and security.....	142
3.1.1. Administering Roles.....	142
3.1.1.1 Describing Roles.....	142
3.1.1.2 Creating a Role.....	142
3.1.1.3 Granting Privileges to a Role.....	143

3.1.1.4 Revoking Privileges from a Role.....	143
3.1.1.5 Modifying a Role.....	143
3.1.2. Administering User Accounts.....	143
3.1.2.1 Predefined User Accounts.....	143
3.2.2.2 User Privileges and Roles.....	143
3.2.2.3 Administrative Accounts and Privileges.....	144
3.2.2.4 Creating a User Account.....	144
3.2.2.5 Viewing User Accounts.....	144
3.2.2.6 Modifying User Account.....	144
3.2.2.7 Granting Privileges and Roles to User Accounts.....	144
3.1.3. Managing Profiles.....	145
3.1.3.1 Describing a Profile.....	145
3.1.3.2 Creating a Profile.....	145
3.1.3.3 Dropping a Profile.....	145
Exercise 3.1.....	145
Learning Outcome 3.2: Perform Database backup and recovery.....	148
3.2.1. Overview of Backup and Recovery.....	148
3.2.1.1 Introduction to Backup and Recovery.....	148
3.2.2. Starting and Configuring RMAN and Flashback Database.....	149
3.2.2.1 RMAN Architecture.....	149
3.2.2.2 Starting and Interacting with RMAN.....	149
3.2.2.3 Configuring the RMAN Environment.....	149
3.2.2.4 Using Flashback Database and Restore Points.....	150
3.2.3. Backing Up and Archiving Data.....	150
3.2.3.1 RMAN Backup Concepts.....	150
3.2.3.2 Backing Up the Database.....	150
3.2.3.3 Advanced Database Backup Features.....	150
3.2.4. Managing RMAN Backups.....	150
3.2.4.1 Reporting on RMAN Operations.....	151
3.2.4.2 Maintaining RMAN Backups and Repository Records.....	151
3.2.4.3 Managing a Recovery Catalog.....	151
3.2.5. Transferring Data with RMAN.....	151
3.2.5.1 Duplicating Databases.....	151
3.2.5.2 Creating Transportable Tablespace Sets.....	151
3.2.5.3 Transporting Data Across Platforms.....	151
3.2.6. Performing User-Managed Backup and Recovery.....	152
3.2.6.1 Making User-Managed Database Backups.....	152
3.2.6.2 Performing User-Managed Database Flashback and Recovery.....	152

Exercise 3.2.....	152
Learning Outcome 3.3: Manage Data encryption.....	155
3.3.1. Data Encryption Introduction.....	155
3.3.1.1 Preventing Database Bypass with Encryption.....	155
3.3.1.2 Oracle Advanced Security Transparent Data Encryption (TDE).....	156
3.3.1.3 Protecting Sensitive Data Using TDE Column Encryption.....	156
3.3.1.4 Protecting Entire Applications Using TDE Tablespace Encryption.....	156
3.3.1.5 Protecting the Database Using TDE Database Encryption.....	157
3.3.1.6 Performance Characteristics.....	157
3.3.1.7 Built-In Key Management.....	157
3.3.1.8 Encryption Impact for Common Operational Activities.....	157
3.3.2. Limiting Sensitive Data Exposure with Data Redaction.....	157
3.3.2.1 Oracle Advanced Security Data Redaction.....	157
3.3.2.2 Policies and Transformations.....	157
3.3.2.3 Performance Characteristics.....	158
3.3.2.4 Security Considerations.....	158
3.3.2.5 Easy to Deploy Data Redaction.....	158
3.3.2.6 Comparison to Alternative Approaches.....	158
Exercise 3.3.....	159
Learning Outcome 3.4: Manage Database auditing.....	161
3.4.1. Introduction to Auditing.....	161
3.4.1.1 Definition of Auditing.....	161
3.4.1.2 Definition of Unified Auditing.....	161
3.4.2. Configuring Audit Policies.....	162
3.4.2.1 Selecting an Auditing Type.....	162
3.4.2.2 Auditing Activities with Unified Audit Policies and the AUDIT Statement.....	163
3.4.2.3 Auditing Activities with Predefined Unified Audit Policies.....	163
3.4.2.4 Auditing Specific Activities with Fine-Grained Auditing (FGA).....	163
3.4.2.5 Audit Policy Data Dictionary Views.....	164
3.4.3. Administering the Audit Trail.....	164
3.4.3.1 Managing the Unified Audit Trail.....	164
3.4.3.2 Archiving the Audit Trail.....	164
3.4.3.3 Purging Audit Trail Records.....	165
3.4.3.4 Audit Trail Management Data Dictionary Views.....	165
Exercise 3.4.....	166
References.....	168

LEARNING UNIT 1: DESCRIBE ORACLE DATABASE ARCHITECTURE

Learning Outcome 1.1: Describe oracle architecture components

1.1.1. Describing Oracle Database Definition and terminologies

1.1.1.1. Definition of Oracle Database

An Oracle Database is a relational database management system (RDBMS) developed by Oracle Corporation. It is designed for storing, retrieving, and managing large amounts of data. Oracle Database supports SQL (Structured Query Language) for querying and manipulating data, and PL/SQL (Procedural Language for SQL) for writing stored procedures and functions.

Oracle databases are widely used in enterprise applications because they support high availability, scalability, and security features.

Example:

A banking system that handles millions of transactions daily can use an Oracle Database to manage customer records, account balances, and transaction history.

1.1.1.2. Database instance

An **Oracle Database Instance** is the set of memory structures and background processes that manage database files (Oracle, 2025).

- The **database** itself is just a set of physical files (datafiles, redo logs, control files).
- The **instance** is what makes the database usable. It loads these files into memory and manages access.

Think of it this way:

- Database = **the body** (data stored in files)
- Instance = **the brain** (processes and memory that make it function)

Example:

*When you start Oracle using STARTUP, you are actually starting the **instance**, which in turn opens the database.*

1.1.1.3. Tablespace

A **tablespace** is a logical storage unit in an Oracle database. It groups related data logically, while the actual data is stored in **datafiles** at the operating system level.

- Each tablespace consists of one or more **datafiles**.
- Tablespaces make it easier for DBAs to manage storage.

Example:

- *USERS tablespace may contain data for end-users.*
- *SYSTEM tablespace contains the data dictionary (metadata about database objects).*

1.1.1.4. Default tablespace

When a user is created in Oracle, they must have a default location to store their data. This location is called the **default tablespace**.

- If the user does not specify where to store their tables or indexes, they are placed in the **default tablespace**.
- By default, Oracle assigns the USERS tablespace for user objects.

Example:

CREATE USER john IDENTIFIED BY password

DEFAULT TABLESPACE users;

1.1.1.5. Temporary tablespace

A **temporary tablespace** is used to store intermediate results of queries and sorting operations.

- Large queries that require sorting (like ORDER BY, GROUP BY, or joins) often spill data to disk.
- To avoid filling permanent tablespaces, Oracle uses **temporary tablespaces**.

Example:

- If a query sorts millions of rows, Oracle uses TEMP tablespace to store temporary results.

 Important: A database should always have at least one temporary tablespace.

1.1.1.6. Undo tablespace

An **undo tablespace** stores undo records, which are used to:

- Roll back uncommitted transactions.
- Recover the database after failure.
- Provide **read consistency** (ensuring users see consistent data, even while other transactions are updating).

Example:

If a user runs:

UPDATE accounts SET balance = balance - 500 WHERE id = 101;

*and later decides to ROLLBACK, Oracle uses the **undo tablespace** to restore the previous values.*

1.1.1.7. Rollback

A **rollback** is the process of undoing uncommitted changes in a transaction.

- Users can issue ROLLBACK manually.
- Oracle also performs rollback automatically in case of a system crash or failed transaction.

Example:

BEGIN;

*UPDATE employees SET salary = salary * 1.1;*

ROLLBACK; -- cancels the update

Result: Salaries remain unchanged.

1.1.1.8. Redo log files

Redo log files record all changes made to the database.

- They are crucial for **data recovery**.
- Even before a change is written to the datafile, it is recorded in the **redo log buffer** and then saved into **redo log files**.

Example:

If the system crashes before a transaction is committed, Oracle uses redo logs to replay changes once the database restarts.

1.1.1.9. Archived redo log files

When redo log files are filled, Oracle can archive them before overwriting. These archived copies are called **archived redo log files**.

- They are essential for **point-in-time recovery**.
- Archiving must be enabled (ARCHIVELOG mode).

Example:

If you need to recover a database up to a specific time (say, before a wrong update was executed), archived redo logs are applied to bring the database back to that state.

1.1.1.10. Temporary tables

Temporary tables are special tables that store session-specific or transaction-specific data.

- Data in a temporary table is visible only to the session that created it.
- Oracle automatically clears data at the end of the session or transaction (depending on definition).

Example:

```
CREATE GLOBAL TEMPORARY TABLE temp_orders (  
  
    order_id NUMBER,  
  
    amount NUMBER  
  
    ) ON COMMIT DELETE ROWS;
```

Here, rows will disappear once the transaction commits.

1.1.1.11. Backups

Backups are copies of database files that can be used to restore the database in case of failure.

Types of backups in Oracle:

1. **Physical backups:** copies of datafiles, control files, and redo logs.
2. **Logical backups:** exports of database objects using tools like Data Pump (expdp/impdp).

Example:

- **RMAN (Recovery Manager)** is the most common tool for Oracle backups.

```
RMAN> BACKUP DATABASE;
```

1.1.2. Describing oracle database management system

1.1.2.1. Overview of Database Management Systems (DBMS)

An **Oracle Database** is a sophisticated form of **Relational Database Management System (RDBMS)**, designed to control the **storage, organization, and retrieval** of data while cleanly separating how data is stored from how it's accessed.

Oracle is also an **object-relational DBMS (ORDBMS)**, meaning it extends the traditional relational model with object-oriented features like user-defined types and inheritance (Oracle, 2025), (Johnson, 2019).

1.1.2.1.1. Key components of Oracle DBMS

- **Kernel code:** handles memory and disk management.
- **Data dictionary (metadata repository):** stores definitions of database structures and users.
- **Query language support:** enables SQL, PL/SQL, and Java access to manipulate data (Oracle, 2025).

1.1.2.1.2. Why does Oracle stand out?

Oracle has evolved over 40+ years into a flexible, enterprise-grade system that supports highly available, scalable, and secure data operations (Oracle, 2025), (Johnson, 2019).

Mini Example:

Imagine a hospital system: Oracle acts like a librarian who knows exactly where each record is stored (via data dictionary), how to fetch it quickly (via kernel and DBMS), and how to let doctors update it securely (via SQL/PLSQL).

1.1.2.2. Overview of transaction management

Oracle treats each sequence of SQL statements executed as a **transaction**. A single, atomic, manageable unit of work. A transaction:

- Starts with the **first SQL statement** that's not part of a transaction block.

- Ends when you issue a **COMMIT** (to save changes) or **ROLLBACK** (to undo them).

1.1.2.2.1. ACID properties in Oracle

- **Atomicity**: Either all parts of the transaction succeed or none do.
- **Consistency**: Transitions the database from one valid state to another. Oracle enforces constraints automatically.
- **Isolation**: Multiple transactions can happen without harmful interaction, thanks to Oracle's multi-version read consistency (Oracle, 2025)
- **Durability**: Once committed, changes persist even after crashes, thanks to redo logs and write-ahead mechanisms.

1.1.2.2.2. Oracle-specific insight

Oracle uses **System Change Numbers (SCNs)** to track and order transactions precisely. Critical for recovery and ensuring consistency (Oracle, 2025).

1.1.2.3. Overview of data integrity

Oracle ensures **data integrity** (accuracy, consistency) through a structured set of integrity constraints:

- **Entity integrity**: ensures every row is uniquely identified (via **PRIMARY KEY**).
- **Referential integrity**: enforces valid relationships across tables using **FOREIGN KEY** constraints.
- **Domain integrity**: validates allowed values (via **CHECK**, **NOT NULL**, **DEFAULT**).
- **User-defined integrity**: enforced through triggers or custom PL/SQL logic (Oracle, 2025).

Oracle also supports **deferrable constraints**, letting you defer checks until transaction commit—a powerful tool for complex data manipulations (Oracle, 2025).

Example:

```
CREATE TABLE employees (  
  
    emp_id NUMBER PRIMARY KEY,  
  
    dept_id NUMBER REFERENCES departments(dept_id),  
  
    salary NUMBER CHECK (salary > 0) DEFAULT 1000 NOT NULL  
  
);
```

1.1.2.4. Overview of data security

Oracle's data security framework is comprehensive:

- **Authentication:** verifies user identity (via passwords, OS/Kerberos, or SSL).
- **Authorization:** grants users precise system and object permissions (via **roles** and **privileges**).
- **Encryption:** Transparent Data Encryption (TDE) secures data at rest.
- **Auditing:** tracks user actions to support regulatory and forensic needs.
- **Network Security:** enforces secure communication channels (SSL/TLS).

Example:

```
CREATE USER analyst IDENTIFIED BY strongPass123;  
  
GRANT CREATE SESSION TO analyst;  
  
GRANT SELECT ON employees TO analyst;
```

Here, analyst can log in and query only the employees table everything else remains off-limits.

Oracle's security machinery ties back to the data dictionary to confirm who has rights to what, always updating with any structural change to keep access rules solid and enforceable Oracle Docs+1.

1.1.3. Describing oracle database architecture

1.1.3.1. Overview of oracle grid architecture

Oracle Grid Architecture embodies **grid computing**, where loosely coupled, modular hardware/software components are dynamically pooled to create highly resilient, scalable, and cost-efficient enterprise systems (Oracle, 2025) (Johnson, 2019).

In modern Oracle deployments:

- **Oracle Grid Infrastructure** combines **Oracle Clusterware** and **Oracle ASM (Automatic Storage Management)** to manage both clustered and standalone environments (Oracle, 2010)(Oracle, 2025)
- In single-instance configurations, Oracle Grid Infrastructure includes **Oracle Restart**, which continuously monitors and automatically restarts components like the database instance, listener, and ASM when failures occur (Oracle, 2010).
- In clustered environments (Oracle RAC), each node runs Grid Infrastructure to coordinate high availability and scalability, with clustered resources accessing the shared storage through ASM and Clusterware.

1.1.3.1.1. Oracle clusterware

Oracle Clusterware is the **foundation for Oracle Real Application Clusters (RAC)**.

It is cluster management software that enables multiple servers (nodes) to work together as if they were one system.

Main Functions:

- Detects and manages node (server) and process failures.
- Provides **cluster membership** (decides which servers are part of the cluster at any moment).
- Starts and stops Oracle databases, listeners, ASM instances, and other resources automatically.

- Supports high availability by failing over resources from one node to another if needed.

Analogy:

*Imagine a team of security guards patrolling a building (the cluster). Clusterware is their **walkie-talkie system and manager** that ensures all guards know who is on duty, who left, and who should take over when someone fails.*

1.1.3.1.2. Oracle ASM (Automatic Storage Management)

Oracle ASM is Oracle's built-in **volume manager and file system** specifically designed for databases.

Key Features:

- Organizes storage into **disk groups** (collections of disks managed as one).
- Provides **striping** (spreads data evenly across disks for performance).
- Provides **mirroring** (duplicates data across disks for redundancy).
- Handles database files (datafiles, control files, redo logs) automatically. DBAs don't need to manually manage OS file systems for them.

Analogy:

*Think of ASM as the **warehouse manager** that takes all your storage rooms (disks), combines them into a big logical warehouse (disk group), and automatically decides where to place items (files) for balance and safety.*

1.1.3.1.3. Oracle RAC (Real Application Clusters)

Oracle RAC allows **multiple Oracle database instances** (on different servers) to access a **single shared database** simultaneously.

Benefits:

- **High Availability (HA):** If one node fails, other nodes continue serving applications without downtime.
- **Scalability:** Add more servers (nodes) to increase processing power.
- **Load Balancing:** Distributes workload across all active instances.

Example:

In a bank running RAC, Node1 and Node2 both serve customer transactions. If Node1 crashes, Node2 continues processing without interrupting customers.

1.1.3.1.4. Cluster (in Oracle context)

In Oracle terminology, a **cluster** usually means:

- **At the system level (Clusterware/RAC):** A group of interconnected computers (nodes) that cooperate to run Oracle databases.
- **At the schema level:** An Oracle **schema cluster** is a storage method where multiple tables that share a common column are physically stored together for faster joins. (This is less common than the system-level meaning).

Mini Example:

Consider two servers forming a RAC cluster. Both run Grid Infrastructure, share data via ASM disk groups, and coordinate via Clusterware. If one server fails, the other continues servicing applications seamlessly.

1.1.3.2. Overview of application architecture

Oracle database applications are typically structured in distributed or multitier architectures:

- **Client/Server (2-tier):** A front-end client (like SQL*Plus or Forms) interacts directly with the database server. Good for small-scale, tightly coupled environments.

- **Multitier (n-tier):** Clients connect to middleware or application servers. These servers handle business logic, then interact with the Oracle database. This model enhances scalability, load balancing, performance, and security separation.
- **Oracle Net Services:** serves as the communication layer, managing protocols, sessions, listeners, and dispatching connections between clients and database instances.

Mini Example:

A web app sends requests to an Oracle-enabled application server that houses business logic. That server uses Oracle Net to connect to the database. Clients never talk directly to the DB.

1.1.3.3. Overview of physical database structures

Physical structures are OS-level artifacts that store data:

When creating a database (especially a CDB), Oracle generates essential files:

- **Data files:** store actual database content (tablespaces → tables, indexes) (Oracle, 2015).
- **Control files:** metadata about database structure (file names, locations, DBID).
- **Online redo log files:** record all changes for recovery purposes.
- Additionally, a CDB also has parameter files, network config files; PDBs share control files and redo logs with the CDB.

1.1.3.3.1. CDB (Container Database)

Starting with Oracle 12c, Oracle introduced the **Multitenant architecture**:

- **CDB (Container Database):** The root database that contains system metadata and can host multiple **PDBs (Pluggable Databases)**.
- **PDB (Pluggable Database):** A portable collection of schemas, objects, and non-system metadata. It is like a self-contained database plugged into a CDB.

Why Multitenant?

- Easier management: upgrade/patch the CDB once, all PDBs benefit.
- Consolidation: run many databases inside one CDB → saves resources.
- Flexibility: move PDBs between CDBs.

Analogy: Think of the **CDB as an apartment building** and **PDBs as individual apartments**.

- The building (CDB) has common infrastructure: elevators, security, power.
- Each apartment (PDB) is independent but benefits from shared services.

Mini Example:

In a 21c CDB with two PDBs, both PDBs write to the same redo logs and are managed by the same control file and instance.

1.1.3.4. Overview of logical database structures

Oracle's logical storage structures create an abstraction layer over physical files:

- **Data Blocks:** smallest allocation unit, akin to pages.
- **Extents:** continuous sets of data blocks allocated for a segment.
- **Segments:** collections of extents for objects like tables, indexes, or undo.
- **Tablespaces:** logical containers for segments; implemented via one or more datafiles.

Mini Example:

A user table might occupy a segment made up of three extents (each of 100 blocks) within the USERS tablespace.

1.1.3.5. Overview of the Oracle data dictionary

The **Oracle Data Dictionary** serves as the metadata hub where definitions of database structures, users, privileges, and more live and is managed internally by Oracle.

Key access views:

- **USER_**, **ALL_**, and **DBA_** prefixed views provide metadata visibility scoped by privilege, such as:
 - **DBA_OBJECTS**, **ALL_TABLES**, **USER_TAB_COLUMNS**, etc.

Mini Example: A DBA runs:

```
SELECT table_name FROM dba_tables WHERE owner = 'HR';
```

to list all tables owned by HR. Ordinary users might use user_tables instead.

1.1.3.6. Overview of the Oracle instance

An **Oracle Instance** comprises memory structures and background processes that operate on the database's physical and logical structures:

- **System Global Area (SGA):** Shared memory containing the buffer cache, shared SQL areas, optional In-Memory Column Store, etc.
- **Program Global Area (PGA):** Memory used privately by each server or background process.
- **Processes:**
 - **Background processes** (e.g., SMON, PMON, LGWR, DBWR) for housekeeping (Housekeeping tasks help you to remove old and unwanted event log data, produced by the running of EDQ jobs in a certain time interval.) and I/O.
 - **Server processes** for client interaction.
 - **Client processes** on user-end, separate from Oracle instance.

In Oracle RAC, multiple instances on different servers connect to the same database and coordinate via Grid Infrastructure.

Mini Example:

A single-instance Oracle database has one SGA, multiple PGAs (perhaps one per connection), and a set of background processes to manage recovery, logging, and clean-up.

Exercises 1.1

1. Define the following Oracle-specific terms in your own words:
 - I. Database instance
 - II. Tablespace
 - III. Undo tablespace
 - IV. Redo log files
 - V. Oracle Data Dictionary
2. Explain the difference between:
 - I. Temporary tablespace vs Undo tablespace
 - II. Redo log files vs Archived redo log files
3. In Oracle architecture, what is the role of the **System Global Area (SGA)**?
4. Which of the following best describes a **CDB (Container Database)**?
 - I. A schema that stores user tables only
 - II. A database that can hold multiple PDBs (Pluggable Databases)
 - III. A temporary storage area for redo logs
 - IV. A background process
5. The default tablespace for **system objects** in Oracle is:
 - I. USERS
 - II. SYSTEM
 - III. TEMP
 - IV. SYSAUX

6. The "c" in **Oracle 12c** stands for:
- I. Cluster
 - II. Cloud
 - III. Cache
 - IV. Core
7. Your company runs an Oracle 19c database. The database suddenly crashes.
- I. Which files will Oracle use to recover committed transactions?
 - II. Which tablespace helps in restoring uncommitted changes?
8. Imagine you are asked to consolidate 10 small applications into one Oracle database.
- I. Which feature (introduced in Oracle 12c) would you use to manage them?
 - II. Explain why this approach is better than creating 10 separate databases.

Learning Outcome 1.2: Describe oracle memory structures and background processes

1.2.1. Describing user related processes

1.2.1.1. User Process

A **User Process** represents the client-side program used to interact with the Oracle Database. This could be tools like SQL*Plus, Enterprise Manager, or any custom application.

- It's created by the operating system on the user's machine when the application starts, and it runs the code that communicates with Oracle through provided APIs (like OCI).
- It **does not directly access Oracle's memory structures** (SGA or PGA); instead, it communicates with the database through a **server process**.

Analogy:

Think of the user process as the person placing an order at a restaurant. They have the menu (application), they decide what they want, and then pass that order (SQL request) through to the waiter (server process). They don't go into the kitchen (the database) themselves.

1.2.1.2. Server Process

A **Server Process** executes on the database host and acts on behalf of the user process to interact with the Oracle instance.

- **Dedicated Server Process:** Each user connection is handled by its own dedicated server process. This process has its own memory area (PGA), processes SQL/PLSQL, and interacts with the SGA and disk storage.
- **Shared Server Process:** Multiple user processes share a pool of server processes. Requests go via a dispatcher into queues in the SGA, where any available server process picks them up. This is more efficient when many users issue intermittent requests.

Example:

In a busy turnout center, rather than having a separate chef (server process) for each customer (user process), many customers place orders, and a shared group of chefs prepares orders in rotation via a shared queue (shared server architecture).

Server Processes do:

- Parse and execute SQL statements.
- Fetch data blocks into the buffer cache (SGA).
- Return results to the user process.

1.2.1.3. Program Global Area (PGA)

The **PGA** (Program Global Area) is a **non-shared memory region** allocated to each server process (whether dedicated or background). It contains session-specific and operation-specific data.

1.2.1.3.1. Contents of the PGA

- **Private SQL area:** temporary memory for SQL execution (bind values, cursor state, runtime data)
- **Session memory:** holds variables and information for that session
- **SQL work areas:** used for memory-intensive operations like sorting, hash joins, and bitmap operations.

1.2.1.3.2. Behavior in different architectures

- **Dedicated server mode:** PGA holds private session memory and SQL areas.
- **Shared server mode:** The User Global Area (UGA) is stored in shared SGA structures; PGA then holds only process-specific stack and work areas, but not session data.

Analogy:

The PGA is like the chef's personal workstation. In dedicated mode, they have all their ingredients and tools in front of them. In shared mode, the common ingredients (UGA) are stored centrally, and they use just the tools they carry with them.

Practical example:

```
SQL>SELECT name, value/1024/1024 AS MB FROM v$pgastat;
```

```

NAME                                                                 MB
-----
aggregate PGA target parameter                                     746
aggregate PGA auto target                                       525.084961
global memory bound                                              100
total PGA inuse                                                  162.742188
total PGA allocated                                              265.714844
maximum PGA allocated                                           297.160156
total freeable PGA memory                                       71.3125
MGA allocated (under PGA)                                        0
maximum MGA allocated                                           0
process count                                                    .000052452
max processes count                                             .000072479

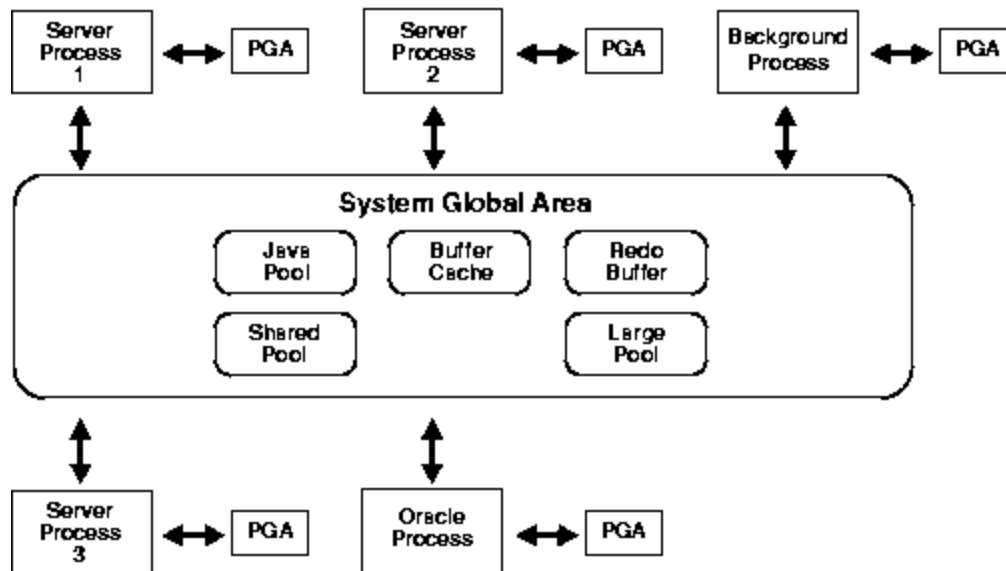
NAME                                                                 MB
-----
PGA memory freed back to OS                                     31
total PGA used for auto workareas                                0
maximum PGA used for auto workareas                             1.53320313
total PGA used for manual workareas                              0
maximum PGA used for manual workareas                           0
over allocation count                                           0
bytes processed                                                  3886.04688
extra bytes read/written                                         0
cache hit percentage                                             .000095367
recompute count (total)                                         .002508163

21 rows selected.

```

1.2.2. Describing memory structure

1.2.2.1. System Global Area (SGA) overview



(Oracle, 2002)

The **SGA** is a read/write shared memory region that all server and background processes within an Oracle instance use to access common data and control information. Each database instance has its own dedicated SGA, allocated at startup and released at shutdown.

*Think of the SGA as a **shared strategy board** in a collaborative workspace. All team members draw from it and contribute to it.*

Key core components include:

- **Database Buffer Cache**
- **Redo Log Buffer**
- **Shared Pool**

Optional components include:

- **Java Pool**
- **Large Pool**
- And others like **In-Memory Area**, **Fixed SGA**.

1.2.2.2. Required SGA areas

1.2.2.2.1. Database buffer cache

This memory area stores copies of data blocks read in from disk. It dramatically reduces disk I/O by serving frequently accessed blocks directly from memory.

- **Buffer states** include: *unused*, *clean* (unchanged), and *dirty* (modified, not yet written to disk).
- Oracle uses algorithms (like LRU-Least Recently Used) to efficiently manage which blocks remain cached.
- It supports both **current** and **consistent** read modes, the latter leveraging undo data for read consistency during concurrent transactions.

1.2.2.2.2. Redo log buffer

A **circular buffer** in memory where Oracle stores information describing changes made by transactions before writing them to redo log files.

- This ensures durability, changes are safely logged before being committed on disk.
- Managed by the background process **LGWR** (Log Writer) which writes redo entries to disk.

1.2.2.2.3. Shared pool

A vital memory area caching program data:

- **Library Cache:** Holds parsed SQL and PL/SQL code, enabling reuse and reducing parsing workload (think “soft parses” vs “hard parses”).
- **Data Dictionary Cache:** Holds metadata (information about tables, users, privileges, etc.) for quick access.
- **Server Result Cache:** Stores result sets to speed repetitive queries.
- **Reserved Pool:** A subarea set aside for large contiguous memory allocations.

*Imagine the shared pool as a **common library**, once code (SQL statements) and metadata are stored, everyone can access them without re-downloading or re-parsing.*

1.2.2.3. Optional SGA areas

1.2.2.3.1. Java pool

Dedicated memory for storing session-specific Java code and data when using Java stored procedures or Oracle JVM.

Analogy:

*A **specialized locker room** within the shared space for Java-specific objects, separate from general resources.*

1.2.2.3.2. Large pool

A memory area designed for large, specialized memory allocations including but not limited to:

- Memory for **shared server UGA** (User Global Area)
- **Parallel execution** message buffers
- **RMAN** I/O processes
- **Deferred inserts** (e.g., MEMOPTIMIZE_WRITE).

By isolating bigger allocations here, the large pool prevents memory fragmentation in the shared pool.

1.2.2.4. Managing the SGA

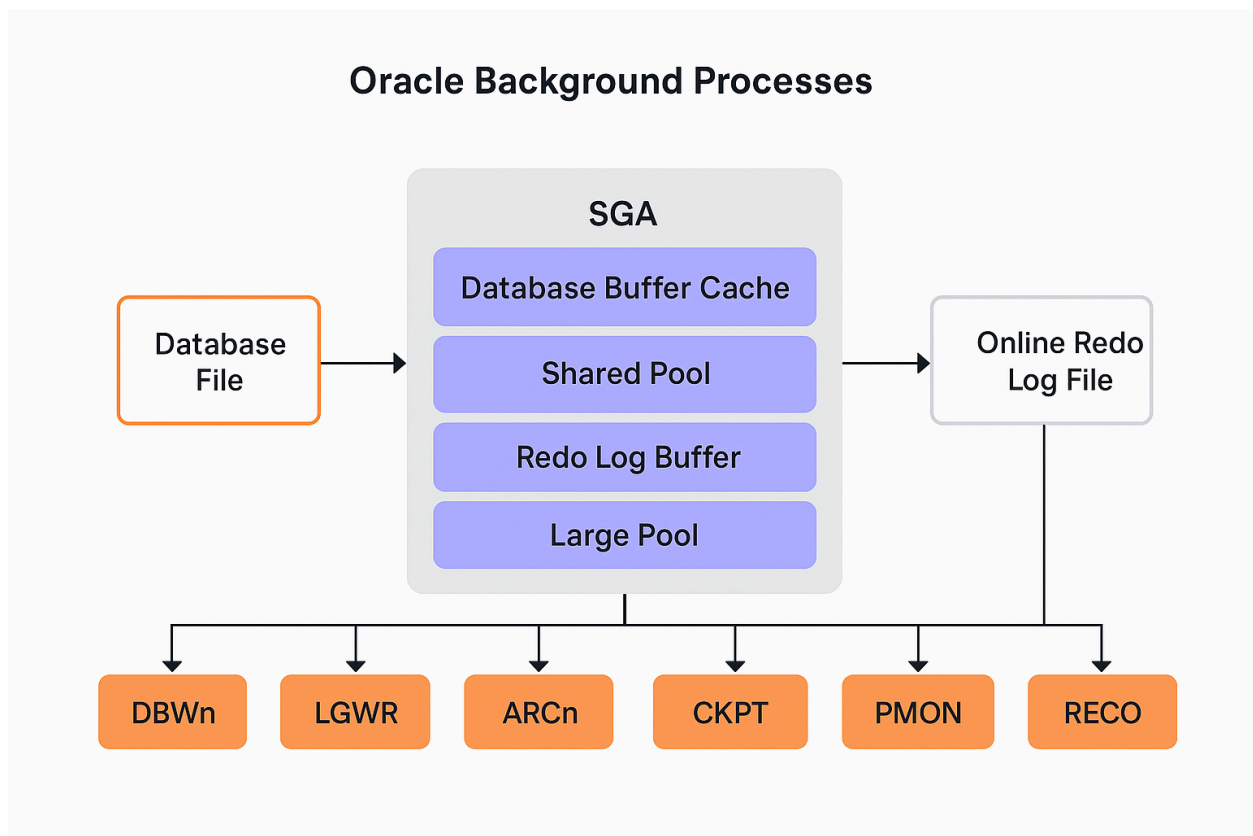
- Oracle supports **Automatic Shared Memory Management (ASMM)** via parameters like SGA_TARGET and SGA_MAX_SIZE. With SGA_TARGET, Oracle dynamically adjusts component sizes within that budget.
- When not using ASMM, DBAs can **manually size** components with parameters like DB_CACHE_SIZE, SHARED_POOL_SIZE, LARGE_POOL_SIZE, JAVA_POOL_SIZE, etc. (Oracle, 2002).

- Memory allocation is done in **granules**, which are platform-specific chunks of memory.

1.2.3. Describing background processes

When you start an Oracle database instance, Oracle automatically launches several **background processes**.

- They run in the OS and work alongside the **System Global Area (SGA)**.
- Their role is to manage memory, disk I/O, recovery, monitoring, and communication between processes.
- Without these, the database cannot function properly.



1.2.3.1. Database Writer Process (DBWn)

- Writes modified (dirty) blocks from the **Database Buffer Cache (SGA)** to the **datafiles** on disk.

- Not every change is immediately written to disk instead, DBWn writes periodically to optimize performance.
- There can be multiple DBWn processes (DBW0, DBW1, ... DBW9, etc.) for scalability.

Example:

If 100 users update rows in memory, Oracle won't write them one-by-one immediately. DBWn batches the updates and writes them efficiently to disk.

1.2.3.2. Log Writer Process (LGWR)

- Writes redo log entries from the **Redo Log Buffer (SGA)** to the **online redo log files**.
- Happens at:
 - A user commits a transaction
 - Every 3 seconds
 - When redo log buffer is one-third full
- Ensures *durability* in the ACID properties.

Example:

When you COMMIT a transaction, LGWR guarantees that redo info is on disk before Oracle confirms the commit to the user.

1.2.3.3. Archiver Process (ARCn)

- Copies **full online redo log files** to **archive log destinations** before they are reused.
- Only active if the database is in **ARCHIVELOG mode** (required for point-in-time recovery).
- Multiple ARCn processes can run in parallel.

Example:

If redo log group 1 fills up, ARCn copies it to an archive file before LGWR overwrites it.

1.2.3.4. Checkpoint Process (CKPT)

- Updates the datafile headers and control files with the latest checkpoint information.
- Signals DBWn to write dirty blocks to disk.
- Reduces crash recovery time because Oracle knows which changes are safely written.

Example:

At a checkpoint, Oracle marks datafiles as synchronized with redo logs — so recovery only needs redo after that point.

1.2.3.5. Process Monitor (PMON)

- Cleans up failed user processes.
- Releases resources (locks, memory, cursors) that were held by a terminated process.
- Restarts certain background processes if they fail.

Example:

If a user disconnects abnormally (power failure, network drop), PMON frees up their session's resources.

1.2.3.6. System Monitor (SMON)

- Performs **instance recovery** when a database starts after a crash.
- Coalesces free space in datafiles (less important in locally managed tablespaces).
- Cleans up temporary segments no longer in use.

Example:

If the system crashes after some transactions were committed but not yet written to disk, SMON applies redo logs during restart.

1.2.3.7. Recover Process (RECO)

- Used in **distributed databases** (multiple Oracle databases linked together).
- Resolves in-doubt transactions. Transactions that were left hanging due to network or system failure.

Example:

If a transaction updates data in two Oracle databases and the network breaks mid-commit, RECO will finalize or roll back the transaction once connectivity is restored.

Exercises 1.2

1. Explain the difference between physical and logical database structures in Oracle. Give one example of each.
2. What is the role of the Oracle Data Dictionary? Mention two types of information it stores.
3. In Oracle Grid Architecture, what is the purpose of Oracle Clusterware?
4. Describe the difference between an Oracle Instance and an Oracle Database.
5. Define Container Database (CDB) and Pluggable Database (PDB) using the apartment analogy.
6. Which of the following is NOT a physical structure in Oracle?
 - a) Control Files
 - b) Data Files
 - c) Tablespaces
 - d) Redo Log Files
7. The Oracle Instance is made up of:
 - a) Control files and data files
 - b) SGA and background processes
 - c) Tablespaces and extents
 - d) Users and roles
8. The Oracle Data Dictionary is stored in:
 - a) Control Files
 - b) SYSTEM Tablespace
 - c) Temporary Tablespace
 - d) Undo Tablespace
9. Use a diagram to draw and label the relationship between Oracle Instance (SGA + background processes), Oracle Database (control files, data files, redo logs), and Users/Applications.

Learning Outcome 1.3: Describe oracle logical and physical storage structures

1.3.1. Describing logical storage of oracle database

In Oracle Database, storage is organized into **logical** and **physical** structures. Understanding logical storage is crucial because it allows the database to manage data efficiently without worrying about the underlying physical files directly.

Logical storage structures include:

- **Tablespaces**
- **Segments**
- **Extents**
- **Data blocks**

We will focus on **tablespaces** in this section.

1.3.1.1. Definition of tablespace

A **tablespace** is a **logical storage unit** within an Oracle database. It groups related logical structures like tables and indexes together. Think of it as a "container" that holds database objects and organizes them for storage management.

- Each Oracle database contains **one or more tablespaces**.
- Tablespaces allow DBAs to manage storage allocation, backup, and recovery **at a logical level** without worrying about physical files.
- Every tablespace is made up of **one or more data files**, which are physical files on the disk.

Example:

Suppose you have a database for a university. You can have a tablespace for student records (STUDENTS_TBS), another for staff data (STAFF_TBS), and so on.

1.3.1.2. Types of tablespaces

Oracle provides several predefined tablespaces for different purposes. Below is a detailed description of the common tablespaces:

1.3.1.2.1. SYSTEM tablespace

- **Purpose:** Stores the core database structures, such as data dictionary and system-related tables.
- **Contains:** Metadata about database objects (tables, indexes, users, privileges).
- **Automatic Creation:** Created during database creation.
- **Important:** Never store user data here—this can affect database stability.

Example:

Data dictionary tables like USER_TABLES, DBA_USERS are stored in SYSTEM tablespace.

1.3.1.2.2. SYSAUX Tablespace

- **Purpose:** Auxiliary tablespace to reduce the load on SYSTEM tablespace.
- **Contains:** Optional but essential components like Oracle Enterprise Manager repository, Automatic Workload Repository (AWR), and other internal database components.
- **Introduced:** Oracle 10g onwards.
- **Important:** Helps keep SYSTEM tablespace small and focused.

Example:

Oracle Text indexes, Statspack/AWR repository tables are stored in SYSAUX.

1.3.1.2.3. EXAMPLE Tablespace

- **Purpose:** Used for **sample schemas** provided by Oracle for learning and testing.
- **Contains:** Sample schemas such as HR, OE, SH for training purposes.
- **Optional:** Not critical for production databases.

Example:

Tables like EMPLOYEES in the HR schema may reside in the EXAMPLE tablespace.

1.3.1.2.4. TEMP Tablespace

- **Purpose:** Stores **temporary data** generated during operations like sorting, joining, or creating indexes.
- **Temporary nature:** Data in TEMP tablespace is **not permanent**; it disappears after the session ends or operation completes.
- **Important:** Helps avoid filling up user tablespaces with temporary data.

Example:

Sorting a large ORDER table for a report may use TEMP tablespace to store intermediate results.

1.3.1.2.5. UNDOTBS1 Tablespace

- **Purpose:** Stores **undo data** which is used to:
 - Roll back transactions
 - Provide read consistency
 - Recover from errors
- **Undo tablespace** is critical for transactional integrity.

Example:

If a user updates a record but wants to rollback, the previous value is stored in UNDOTBS1.

1.3.1.2.6. USERS Tablespace

- **Purpose:** Default tablespace for **user-created objects**.

- **Contains:** Tables, indexes, and other objects created by users (except system objects).
- **Important:** You can create additional user-defined tablespaces if needed.

Example:

```
CREATE USER student_user
IDENTIFIED BY password
DEFAULT TABLESPACE USERS
TEMPORARY TABLESPACE TEMP;
```

Here, student_user will store objects in the USERS tablespace.

1.3.2. Describing physical storage of oracle database

1.3.2.1. Control file

A control file is a small binary file that keeps track of the physical structure of the database. It contains metadata about the database, such as the database name, timestamp of database creation, and the names and locations of associated data files and redo log files.

Purpose:

- Tracks the database structure.
- Records the current log sequence number.
- Helps in database recovery after a failure.

Characteristics:

- Required for database startup.
- Multiple copies are recommended for fault tolerance.

Example (OS location):

\$ORACLE_HOME/oradata/ORCL/control01.ctl

\$ORACLE_HOME/oradata/ORCL/control02.ctl

1.3.2.2. Data files

Data files store the actual user and system data for the database. Each tablespace in Oracle has at least one data file.

Purpose:

- Stores all database data: tables, indexes, system data.
- Organized into tablespaces for better management.

Characteristics:

- Managed by the database; do not edit manually.
- Can be auto-extensible.

Example:

system01.dbf → SYSTEM tablespace

users01.dbf → USERS tablespace

example01.dbf → EXAMPLE tablespace

File Name	Tablespace	Purpose
system01.dbf	SYSTEM	Stores core database info
sysaux01.dbf	SYSAUX	Auxiliary system data
users01.dbf	USERS	User-created tables

1.3.2.3. Redo log files

Redo log files record all changes made to the database, such as insertions, updates, or deletions. These are crucial for recovering committed transactions in case of failure.

Purpose:

- Supports **instance recovery** after a crash.
- Ensures **data integrity** by replaying transactions that weren't written to data files.

Characteristics:

- Always written in a circular fashion (multiplexing is recommended).
- Comprises one or more redo log groups, each having one or more members.

Example:

redo01.log, redo02.log, redo03.log

Group	Member
1	redo01.log
2	redo02.log
3	redo03.log

1.3.2.4. Parameter files

Parameter files store initialization parameters for the database instance. They are required to start an Oracle instance.

Types:

1. **PFILE (text file)** – Editable using any text editor.
2. **SPFILE (binary file)** – Editable using SQL commands; preferred in production.

Purpose:

- Configures memory, background processes, and other startup parameters.

Example:

initORCL.ora → PFILE

spfileORCL.ora → SPFILE

Key Parameters Example:

DB_NAME = ORCL

MEMORY_TARGET = 2G

CONTROL_FILES = ('/oradata/ORCL/control01.ctl','/oradata/ORCL/control02.ctl')

1.3.2.5. Password file

A password file stores database user credentials that are allowed to connect with SYSDBA or SYSOPER privileges remotely.

Purpose:

- Supports **remote administration** of the database.

Example:

orapwd file=orapwORCL password=manager entries=10

Note:

- Needed only when using **remote login** to the database.

1.3.2.6. Backup files

Backup files are copies of database files (data files, control files, redo logs) created to restore the database in case of failure.

Types:

- **Cold backup:** Database is shut down during backup.

- **Hot backup:** Database remains online and active.

Purpose:

- Ensures **disaster recovery**.
- Can be full or partial, depending on requirement.

Example:

/backup/ORCL/system01.dbf.bak

/backup/ORCL/users01.dbf.bak

1.3.2.7. Archived redo log files

Definition:

Archived redo logs are copies of filled redo log files, saved for future recovery purposes.

Purpose:

- Required for **media recovery** in case of data file loss.
- Enables **point-in-time recovery**.

Example:

/oradata/ORCL/archivelog/redo01_01.arc

/oradata/ORCL/archivelog/redo01_02.arc

Key Notes:

- Redo log files must be archived if the database is in **ARCHIVELOG mode**.
- Stored separately from active redo logs to avoid overwriting.

1.3.3. Describing Client-Server model of oracle engine

Oracle Database follows a **client-server architecture**, which separates the **user interface** (client) from the **database engine** (server). This model ensures efficiency, scalability, and secure access.

1.3.3.1. Components of oracle client-server architecture

Component	Description
Client	The application or tool that requests services from the database. Examples: SQL*Plus, Oracle SQL Developer, custom applications. It sends SQL queries or procedural calls to the server.
Server	The Oracle database engine that processes client requests. It consists of the instance (SGA + background processes) and database files .
Listener	A network service on the server that listens for incoming client connection requests and forwards them to the appropriate Oracle instance.
Network	Protocols like TCP/IP used to communicate between client and server.

1.3.3.2. How client-server interaction works

1. **Client initiates a connection** using a tool or application.
2. **Listener receives the request** and directs the connection to an Oracle instance.
3. **Server allocates a dedicated or shared server process** to handle the session.
4. **SQL statements are executed** in the server, interacting with memory (SGA) and disk (data files).
5. **Results are sent back** to the client over the network.

1.3.3.3. Types of server processes

- **Dedicated server:** Each client has a separate server process. Suitable for small-to-medium workloads.

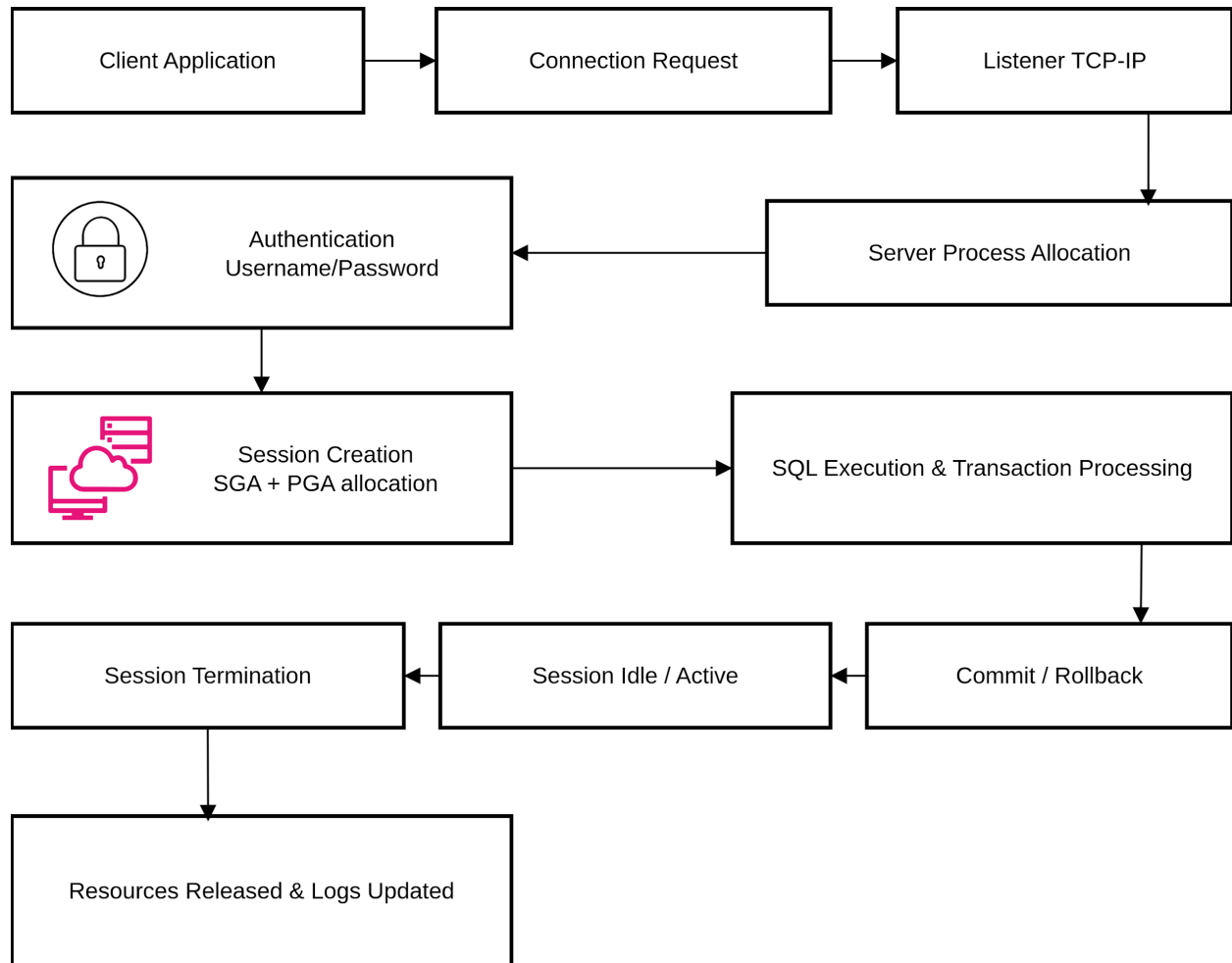
- **Shared server:** Multiple clients share a pool of server processes. Efficient for large-scale applications.

1.3.3.4. Differences in memory allocation between dedicated and shared servers

Memory Area	Dedicated Server	Shared Server
Nature of session memory	Private	Shared
Location of the persistent area	PGA	SGA
Location of part of the runtime area for <code>SELECT</code> statements	PGA	SGA
Location of the runtime area for DML/DDL statements	PGA	PGA

1.3.3.5. Lifecycle of a database session

When a client connects to an Oracle database, a **session** is created. A session is a **logical entity** representing a connected user. Understanding its lifecycle helps in **monitoring, performance tuning, and troubleshooting**.



1.3.3.5.1. Steps in a database session lifecycle

Stage	Description
1. Connection Request	Client requests a connection to the database through the listener.
2. Authentication	Oracle verifies the username and password (and optionally roles/privileges).
3. Session Creation	A session is allocated in memory (SGA), and session state is initialized.

4. SQL Execution / Transaction Processing	SQL statements are parsed, executed, and results returned. Transactions can commit or rollback.
5. Idle / Active States	Session remains idle or active depending on client activity.
6. Session Termination	Client disconnects. Oracle releases memory and resources, optionally saving session state to redo logs for recovery.

Key Points

- Sessions are tied to **server processes** (dedicated or shared).
- A session can handle multiple **transactions** during its lifetime.
- Oracle maintains session-specific data in **PGA (Program Global Area)** and SGA for shared resources

Exercises 1.3

1. Define the **Client-Server model** in Oracle.
2. What role does the **listener** play in Oracle's client-server communication?
3. Differentiate between a **dedicated server process** and a **shared server process**.
4. At what stage of the session lifecycle does **authentication** occur?
5. Which Oracle memory areas are involved when a session is created?
6. Explain how Oracle ensures security during the **authentication phase** of the client-server model.
7. A user reports that they cannot connect to the Oracle database, but the database itself is running. Which component of the client-server model would you check first, and why?
8. Suppose multiple clients are connecting to an Oracle instance, and system resources are becoming strained. Which **server process model** (dedicated vs shared) would you recommend, and why?
9. During an online transaction, a user's session suddenly terminates due to a network issue. What happens to their uncommitted transaction in Oracle? Which component of Oracle ensures data consistency?

10. Draw a combined diagram showing how **client, listener, server processes, and session lifecycle stages** interact in Oracle.
11. During monitoring, you see that several sessions are in an **idle state**. What does this mean, and how should a DBA handle excessive idle sessions?

Hands on skills exercise (if your Oracle database is configured)

You are a DBA trainee tasked with investigating an existing Oracle database to understand its logical and physical storage layout.

Tasks:

Connect to the database:

```
sqlplus / as sysdba
```

1. Explore logical storage structures:

List all tablespaces:

```
SELECT tablespace_name, status, contents FROM dba_tablespaces;
```

- List all segments (tables and indexes) in a specific tablespace:

```
SELECT segment_name, segment_type, tablespace_name, bytes/1024/1024 AS  
size_MB FROM dba_segments WHERE tablespace_name = 'USERS';
```

2. Explore physical storage structures:

List data files for each tablespace:

```
SELECT tablespace_name, file_name, bytes/1024/1024 AS size_MB  
FROM dba_data_files;
```

- List redo log files:

```
SELECT group#, member, bytes/1024/1024 AS size_MB FROM v$logfile;
```

- Check control files:

```
SHOW PARAMETER control_files;
```

LEARNING UNIT 2: MANAGE DATABASE

Learning Outcome 2.1 : Setup Oracle Database Environment

2.1.1. Installing oracle database

Installing Oracle Database is not just about running a setup wizard; it requires planning, preparing the environment, installing the software, and configuring the tools for administration.

We will use Oracle Database 19c on a VirtualBox virtual machine with Oracle Linux as the operating system.

2.1.1.1. Installing oracle database software

Step 1: Download Required Software

- **VirtualBox (7.2):** Download VirtualBox (Linux/Windows hosts).
- **Oracle Linux (OL8.9) ISO:** [Download OL8 ISO](#).
- **Oracle Database 19c** (requires free Oracle account):
 - [Linux Download](#)
 - [Windows Download](#)
- **Oracle SQL Developer:** Download from Oracle site.

Step 2: Install VirtualBox on Host Machine

Install the downloaded VirtualBox package. Example for Ubuntu:

```
sudo dpkg -i virtualbox-7.2_7.2.0-170228~Ubuntu~jammy_amd64.deb
```

- If installation fails:

```
sudo add-apt-repository universe
sudo apt install liblzfl libqt5help5 libqt5opengl5 libqt5printsupport5 libqt5xml5 libtpms0
sudo apt --fix-broken install
sudo dpkg -i virtualbox-7.2_*.deb
```

- If kernel modules fail (common with **Secure Boot**), create and enroll a **MOK** and sign VirtualBox modules:

```
openssl req -new -x509 -newkey rsa:2048 -keyout MOK.priv -outform DER -out MOK.der  
-nodes -days 36500 -subj "/CN=VirtualBox/"
```

```
sudo mokutil --import MOK.der
```

```
# reboot → enroll key
```

```
sudo /usr/src/linux-headers-$(uname -r)/scripts/sign-file sha256 ~/MOK.priv ~/MOK.der  
$(modinfo -n vboxdrv)
```

```
sudo /usr/src/linux-headers-$(uname -r)/scripts/sign-file sha256 ~/MOK.priv ~/MOK.der  
$(modinfo -n vboxnetflt)
```

```
sudo /usr/src/linux-headers-$(uname -r)/scripts/sign-file sha256 ~/MOK.priv ~/MOK.der  
$(modinfo -n vboxnetadp)
```

```
sudo modprobe vboxdrv vboxnetflt vboxnetadp
```

```
lsmod | grep vbox
```

Step 3: Create a New Virtual Machine

- **Name:** OracleDB19c (as an example)
- **Type:** Linux
- **Version:** Oracle Linux (64-bit)
- **Memory:** 8 GB (minimum 2 GB if limited)
- **Virtual Hard Disk:** 50 GB (dynamically allocated)
- Mount the OL8 ISO in **Storage** → **Optical Drive**.

Step 4: Install Oracle Linux in VirtualBox

- Boot the VM from the ISO and start installation.
- Select **Server with GUI** (recommended).
- Set **root password** and create a user.

- Complete installation and reboot.

Step 5: Activate Network and Update OS

Inside the VM, configure DNS to persist (as root user):

```
nmcli connection show
sudo nmcli connection modify "enp0s3" ipv4.dns "8.8.8.8 8.8.4.4"
sudo nmcli connection modify "enp0s3" ipv4.ignore-auto-dns yes
sudo nmcli connection up "enp0s3"
```

Update all packages:

```
sudo yum update -y
```

Step 6: Configure Shared Folder (Optional but Recommended)

On host:

```
mkdir ~/vboxshare
```

- In VirtualBox → VM Settings → Shared Folders → Add New:
 - **Folder Path:** /home/<your-user>/vboxshare
 - **Folder Name:** share
 - Enable **Auto-mount** and **Make Permanent**.

In VM:

```
sudo mkdir -p /mnt/share
sudo mount -t vboxsf share /mnt/share
```

Now you can copy files in/out using /mnt/share.

2.1.1.2. Planning an oracle database and oracle grid for standalone server installation

Before starting installation, you must plan for:

Resource	Minimum Requirement	Recommended
RAM	2 GB	8 GB
Disk Space	20 GB	50 GB+
CPU Cores	1	2–4 cores
OS	Linux/Windows	Oracle Linux 7/8/9 (64-bit)
Oracle Software	Oracle Database 19c or up to 21c	Latest release

2.1.1.3. Operating system groups and users required

Group	Purpose
oinstall	Owns Oracle software inventory
dba	Database administrators group
oper	For database operators (limited privileges)

2.1.1.4. Configuring Oracle Software owner environment (as root user)

Before installing Oracle software, you must create the necessary **user accounts and groups**. (For Oracle database 21c, it is not necessary to do Step 1 up to 3 because its pre-install does that. It means you have to install oracle database 21c pre-install first).

Step 1: Create Oracle groups

```
sudo groupadd oinstall
```

```
sudo groupadd dba
```

```
sudo groupadd oper
```

```
# ASM/Grid Infrastructure groups
sudo groupadd asmadmin # full control of ASM
sudo groupadd asmdba # DB users who can access ASM
sudo groupadd asmoper # limited ASM operations
```

Step 2: Create Oracle and grid users

```
sudo useradd -g oinstall -G dba,oper oracle
sudo passwd oracle

sudo useradd -g oinstall -G asmadmin,asmdba,asmoper,dba grid
sudo passwd grid
```

Step 3: Create directories

```
# Create directories
sudo mkdir -p /u01/app/oracle/product/19.0.0/dbhome_1
sudo mkdir -p /u01/app/19.0.0/grid
sudo mkdir -p /u01/oraInventory

# Set ownership
sudo chown -R grid:oinstall /u01/app/19.0.0/grid /u01/app/oraInventory
sudo chown -R oracle:oinstall /u01/app/oracle

# Set permissions
sudo chmod -R 775 /u01/app/oracle
sudo chmod -R 775 /u01/app/19.0.0/grid
Sudo chmod -R 775 /u01/app/oraInventory
sudo chmod -R 755 /u01/app
```

Step 4: Configure environment variables

Environment Setup for Grid (ASM)

Login as grid user:

```
su - grid
```

```
nano ~/.bash_profile
```

Add:

```
# Grid Infrastructure environment
export ORACLE_BASE=/u01/app/grid
export ORACLE_HOME=/u01/app/19.0.0/grid
export ORACLE_SID=+ASM
export PATH=$ORACLE_HOME/bin:$PATH
export LD_LIBRARY_PATH=$ORACLE_HOME/lib:$LD_LIBRARY_PATH
```

Save and reload:

```
source ~/.bash_profile
```

Environment Setup for Oracle (Database)

Login as oracle user:

```
su - oracle
```

```
nano ~/.bash_profile
```

Add:

```
# Database environment
export ORACLE_BASE=/u01/app/oracle
export ORACLE_HOME=$ORACLE_BASE/product/19.0.0/dbhome_1
export ORACLE_SID=ORCLCDB
export PATH=$ORACLE_HOME/bin:$PATH
```

```
export LD_LIBRARY_PATH=$ORACLE_HOME/lib:$LD_LIBRARY_PATH
```

Save and reload:

```
source ~/.bash_profile
```

Step 7: Install Oracle Database 19c

Install prerequisites (as root user):

```
sudo yum install oracle-database-preinstall-19c.x86_64 -y
```

```
Sudo dnf config-manager --enable ol8_addons
```

```
Yum repolist
```

```
Sudo yum install oracleasm-support
```

```
Sudo systemctl -p
```

Setup the loop device

```
Sudo losetup /dev/loop0 /tmp/asm_disk1.img
```

```
Losetup -a
```

```
Sudo nano /etc/udev/rule.d/99-oracle-asm.rules
```

Add the following rule:

```
KERNEL="loop0",      OWNER="grid",      GROUP="asmadmin",      MODE="0660",  
SYMLINK+="asm-disk1"
```

Save

`Sudo udevadm control --reload-rules`

`Sudo udevadm trigger`

`Ls -l /dev/asm*`

`Sudo oracleasm createdisk ASM_DISK1 /dev/loop0`

Create temporary staging directories

`sudo mkdir -p /tmp/oracle_db`

`sudo mkdir -p /tmp/oracle_grid`

- oracle_db → for Database installation files
- oracle_grid → for Grid/ASM installation files

Copy the ZIP files to the staging directories

`sudo cp /mnt/share/LINUX-X64_193000_db_home.zip /tmp/oracle_db/`

`sudo cp /mnt/share/LINUX.X64_193000_grid_home.zip /tmp/oracle_grid/`

Extract the ZIP files

Database home:

`cd /tmp/oracle_db`

`sudo unzip LINUX-X64_193000_db_home.zip -d /u01/app/oracle/product/19.0.0/dbhome_1`

Grid home:

`cd /tmp/oracle_grid`

`unzip LINUX.X64_193000_grid_home.zip -d /u01/app/19.0.0/grid (as grid user)`

N.B: If disk is full you can increase it but shutting down the VM, navigate to the directory where the .vdi is stored, open in terminal and run:

VBoxManage modifyhd OracleGrid.vdi --resize 204800

Switch VM ON and:

- Lsblk
- Df -h
- Fdisk /dev/sda
- P
- D
- N
- N
- W
- Partprobe /dev/sda
- Pvresize /dev/sda2
- Vgdisplay
- Lvdisplay
- Lvextend -l +100%FREE /dev/mapper/ol-root
- Resize2fs /dev/mapper/ol-root
- Lsblk -f /dev/mapper/ol-root
- Xfs_growfs /
- Df -h /

Set ownership (important!)

Grid home → grid user

sudo chown -R grid:oinstall /u01/app/19.0.0/grid

Database home → oracle user

sudo chown -R oracle:oinstall /u01/app/oracle/product/19.0.0/dbhome_1

Configure .Xauthority

- In default (sudoer) user, create the **.Xauthority**: touch ~/.Xauthority
- Generate a new x11 authorization cookie:

```
echo $DISPLAY
```

```
xauth generate :0 . trusted
```

- Check if the .Xauthority file is created: ls -la ~/.Xauthority
- Allow **oracle** user to access the X server: xhost +SI:localuser:oracle
- Allow **grid** user to access the X server: xhost +SI:localuser:grid
- Switch to **oracle** user and set the display: export DISPLAY=:0
- Switch to **grid** user and set the display: export DISPLAY=:0 in bash_profile

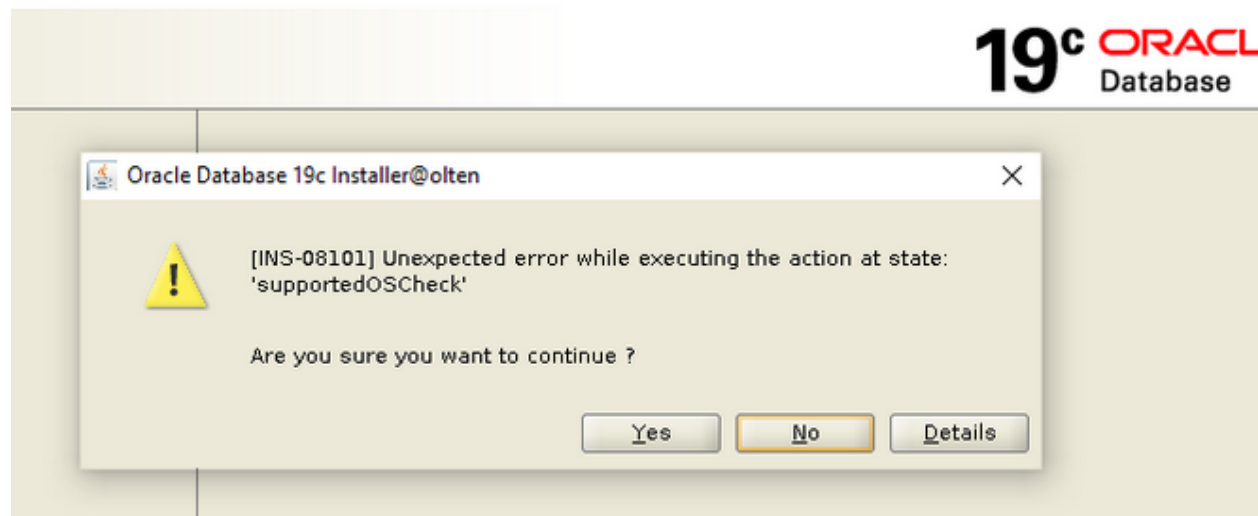
Grid/ASM installation → login as grid user:

```
su - grid
```

```
cd /u01/app/19.0.0/grid
```

```
./gridSetup.sh
```

If you face this error:



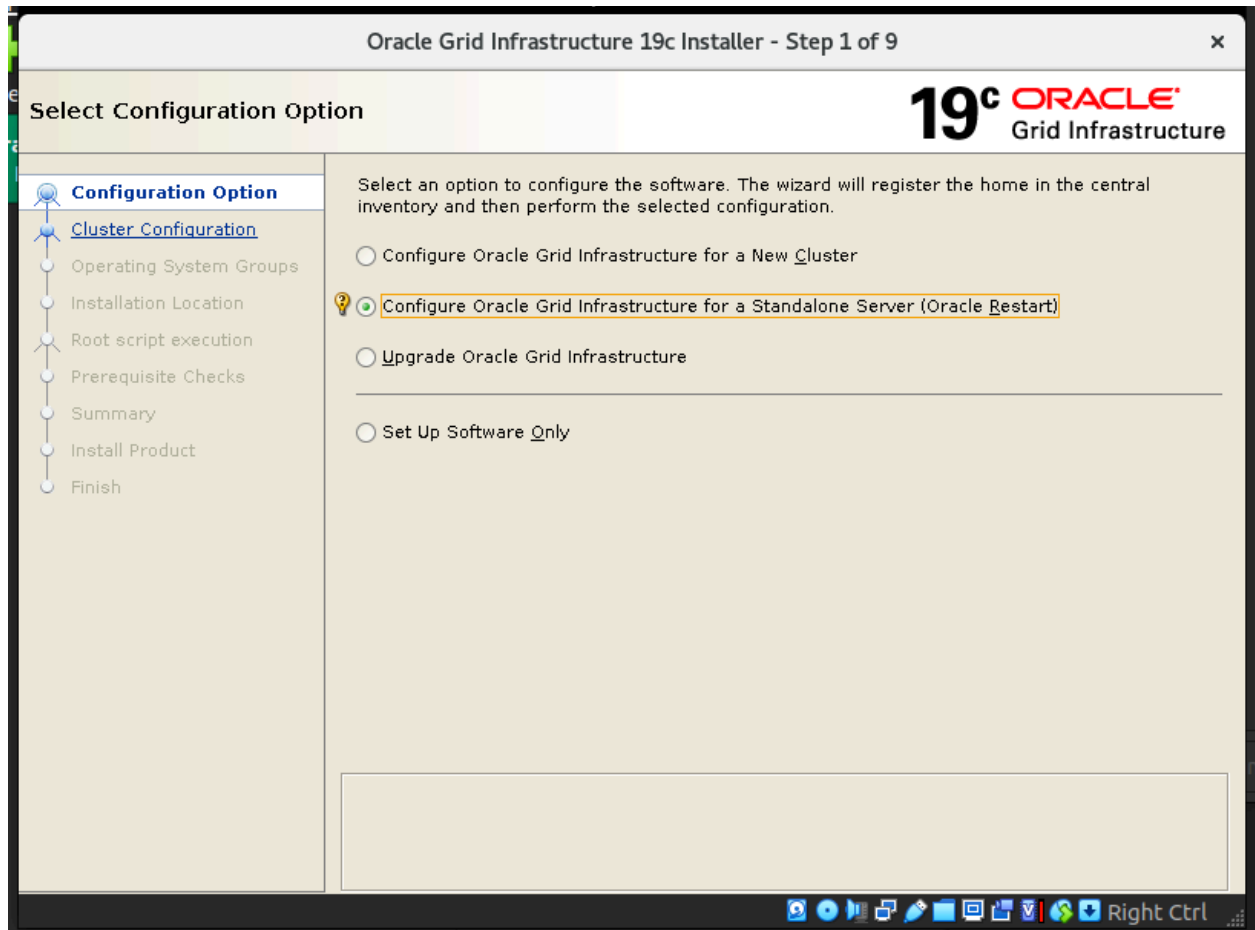
Set the OS parameter in your terminal to a previous version and re-start the Oracle Universal Installer.

export CV_ASSUME_DISTID=OEL7.8

Or Edit the file cvu_config which is located in \$ORACLE_HOME/cv/admin, change the line from

Fallback to this distribution id

CV_ASSUME_DISTID=OEL7.8



Oracle Grid Infrastructure 19c Installer - Step 2 of 10

19c

ORACLE

Grid Infrastructure

Create ASM Disk Group

Configuration Option

Create ASM Disk Group

ASM Password

Operating System Groups

Installation Location

Root script execution

Prerequisite Checks

Summary

Install Product

Finish

Select Disk Group characteristics and select disks

Disk group name

DATA

Redundancy

☐ Flex
☐ High
☐ Normal
☒ External

Allocation Unit Size

4

MB

Select Disks

Show Candidate/Provisioned Disks

	Disk Path	Size (in MB)	Status
☒	/dev/loop0	10240	Candidate

Disk Discovery Path: /dev/loop*

Change Discovery Path...

☐ Configure Oracle ASM Filter Driver

Select this option to configure ASM Filter Driver(AFD) to simplify configuration and management of disk devices by Oracle ASM.

Help

< Back

Next >

Install

Cancel

Oracle Grid Infrastructure 19c Installer - Step 3 of 10

Specify ASM Password

19c ORACLE
Grid Infrastructure

Configuration Option

Create ASM Disk Group

ASM Password

Operating System Groups

Installation Location

Root script execution

Prerequisite Checks

Summary

Install Product

Finish

The new Oracle Automatic Storage Management (Oracle ASM) instance requires its own SYS user with SYSASM privileges for administration. Oracle recommends that you create a less privileged ASMSNMP user with SYSDBA privileges to monitor the ASM instance.

Specify the password for these user accounts.

☐ Use different passwords for these accounts

	Password	Confirm Password
SYS		
ASMSNMP		

☒ Use same passwords for these accounts

Specify Password:

Confirm Password:

Help

< Back

Next >

Install

Cancel



KARONGI COLLEGE

RWANDA POLYTECHNIC

62

Oracle Grid Infrastructure 19c Installer - Step 4 of 11

Specify Management Options

19c ORACLE
Grid Infrastructure

Configuration Option

Create ASM Disk Group

ASM Password

Management Options

Operating System Groups

Installation Location

Root script execution

Prerequisite Checks

Summary

Install Product

Finish

You can configure to have this instance of Oracle Grid Infrastructure and Oracle Automatic Storage Management to be managed by Enterprise Manager Cloud Control. Specify the details of the Cloud Control configuration to perform the registration.

☐ Register with Enterprise Manager (EM) Cloud Control

OMS host:

OMS port:

EM Admin User Name:

EM Admin Password:

Help

< Back

Next >

Install

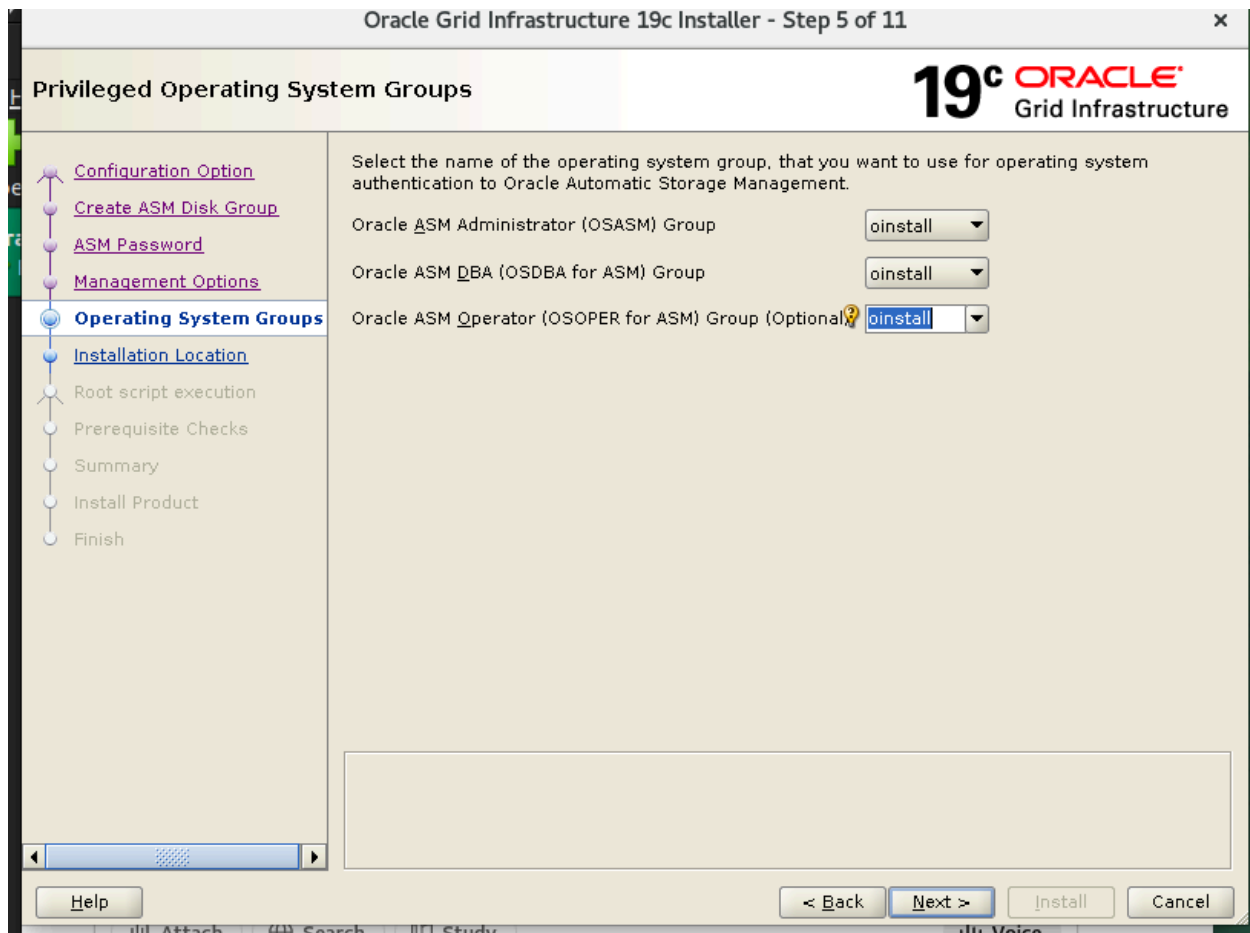
Cancel

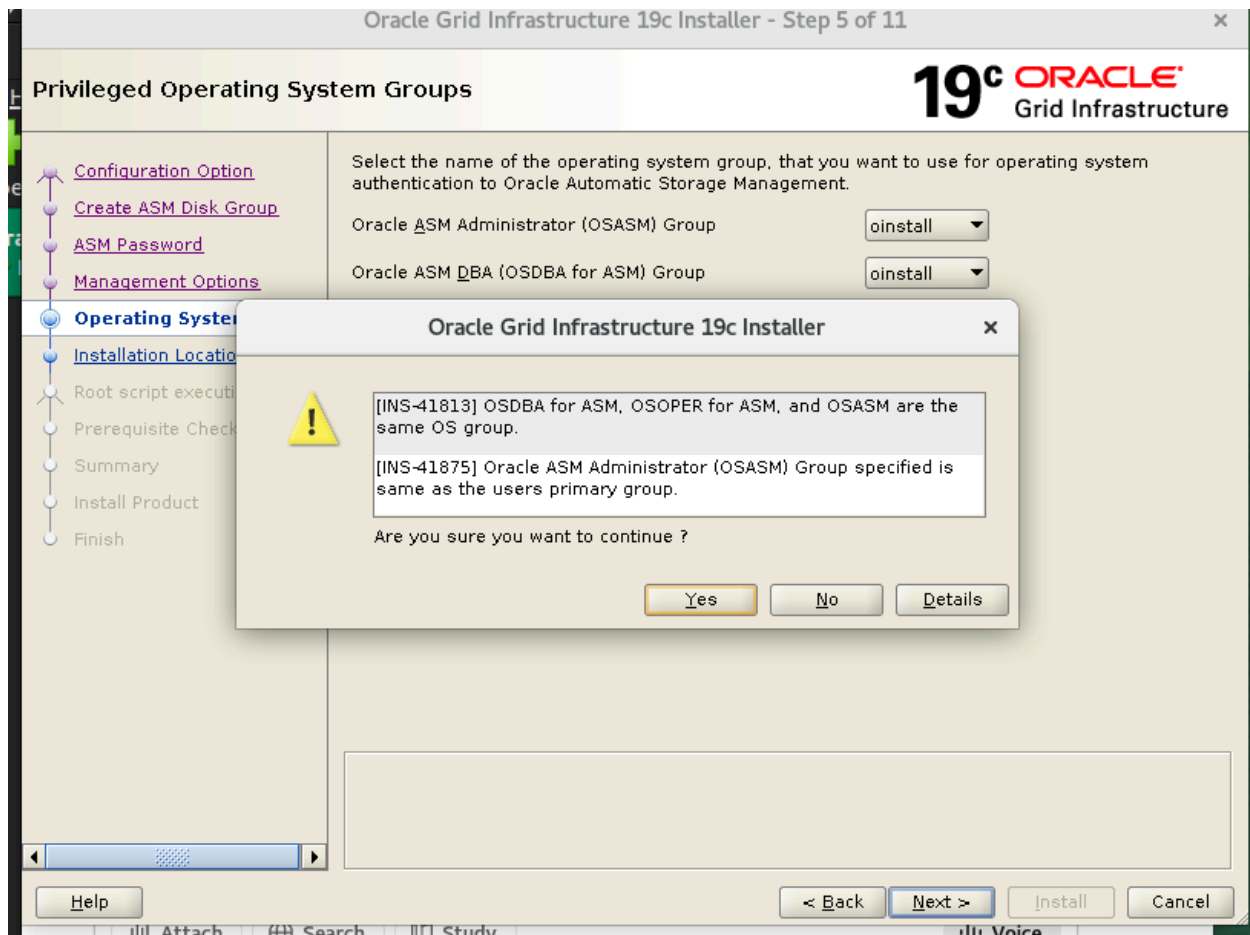


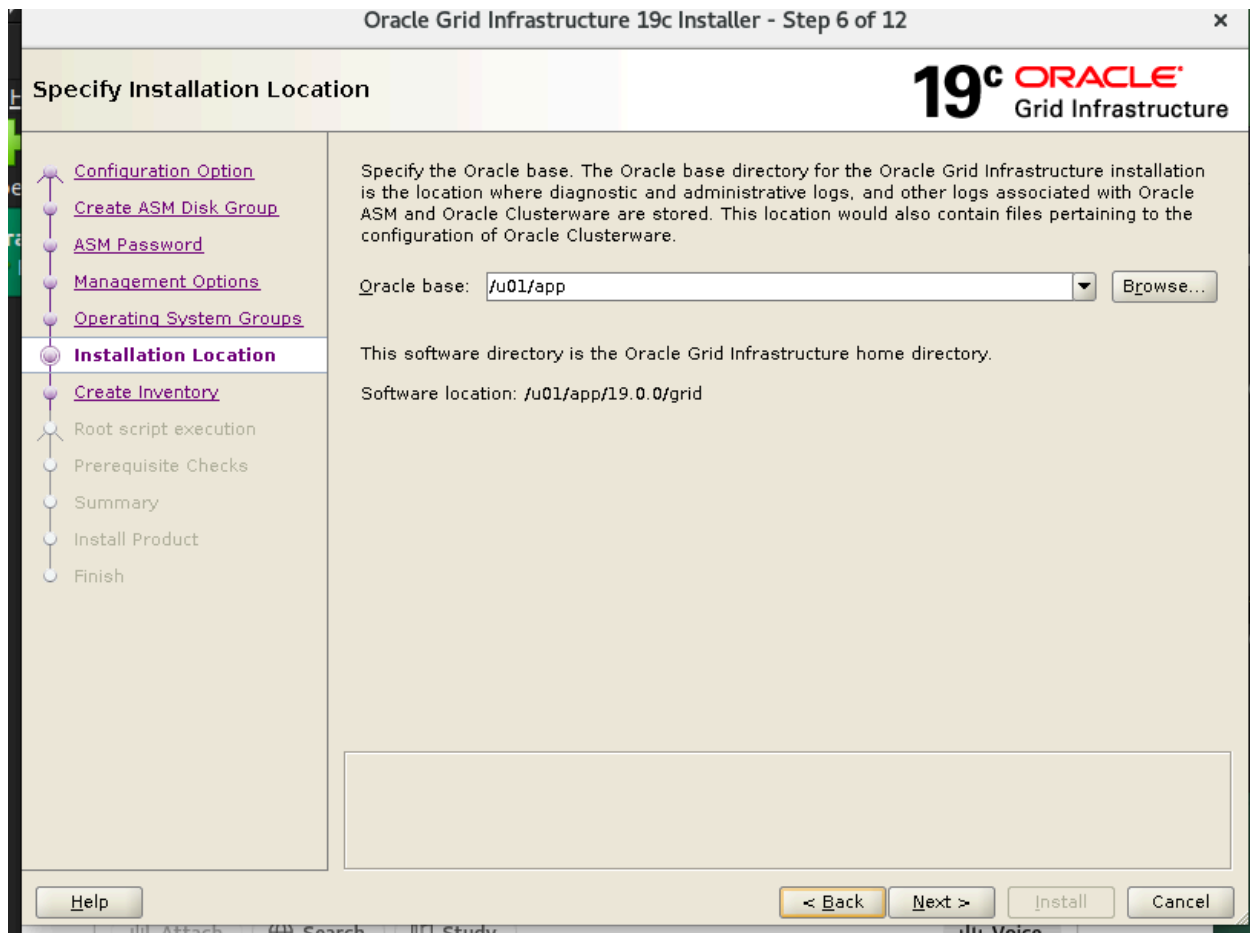
RWANDA POLYTECHNIC

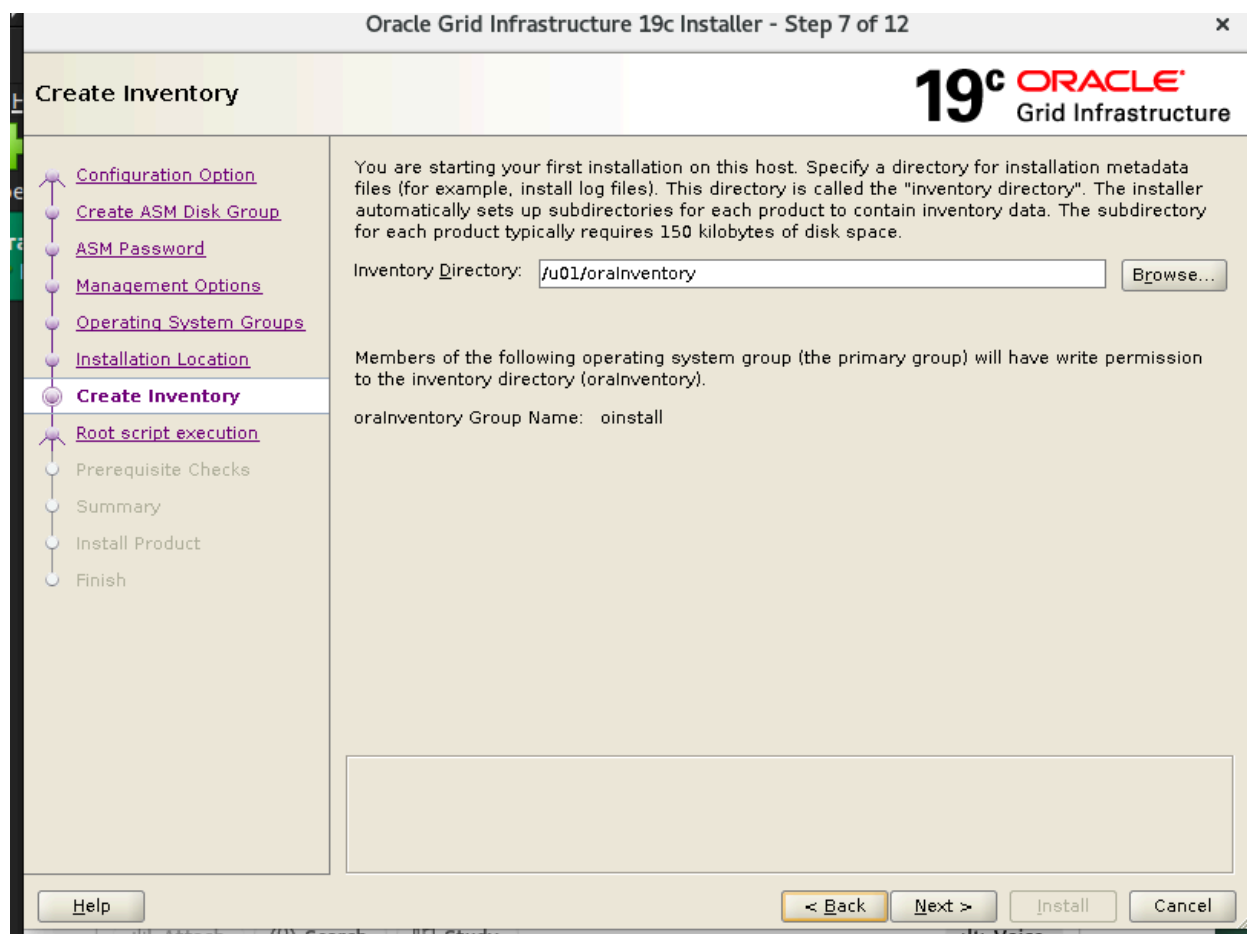
KARONGI COLLEGE

63









Oracle Grid Infrastructure 19c Installer - Step 8 of 12

Root script execution configuration

19c ORACLE
Grid Infrastructure

Configuration Option

Create ASM Disk Group

ASM Password

Management Options

Operating System Groups

Installation Location

Create Inventory

Root script execution

Prerequisite Checks

Summary

Install Product

Finish

During the software configuration, certain operations have to be performed as "root" user. You can choose to have the installer perform these operations automatically by specifying inputs for one of the options below. The input specified will also be used by the installer to perform additional prerequisite checks.

☐ Automatically run configuration scripts

☒ Use "root" user credential

Password :

☐ Use sudo

Program path :

User name :

Password :

Help

< Back

Next >

Install

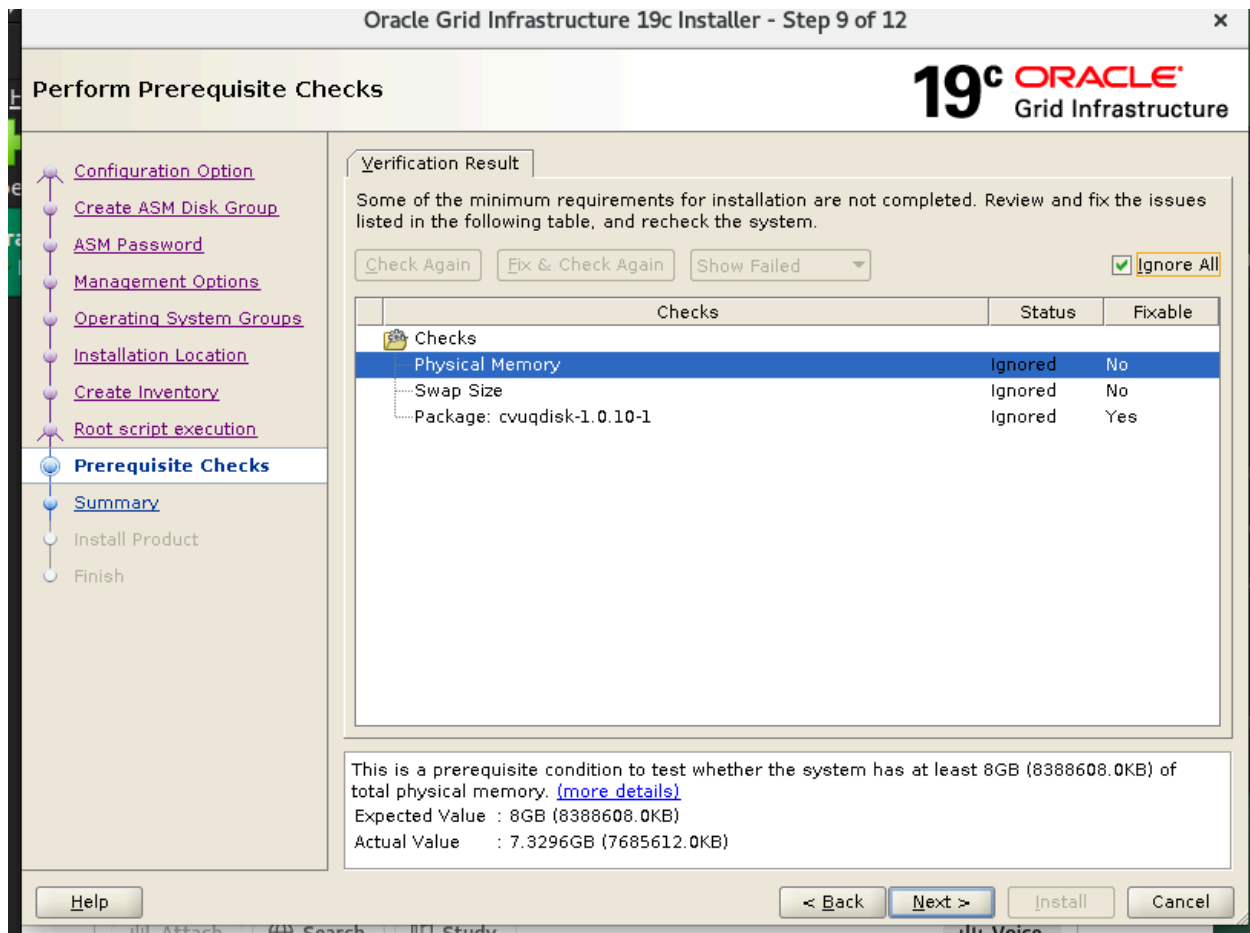
Cancel

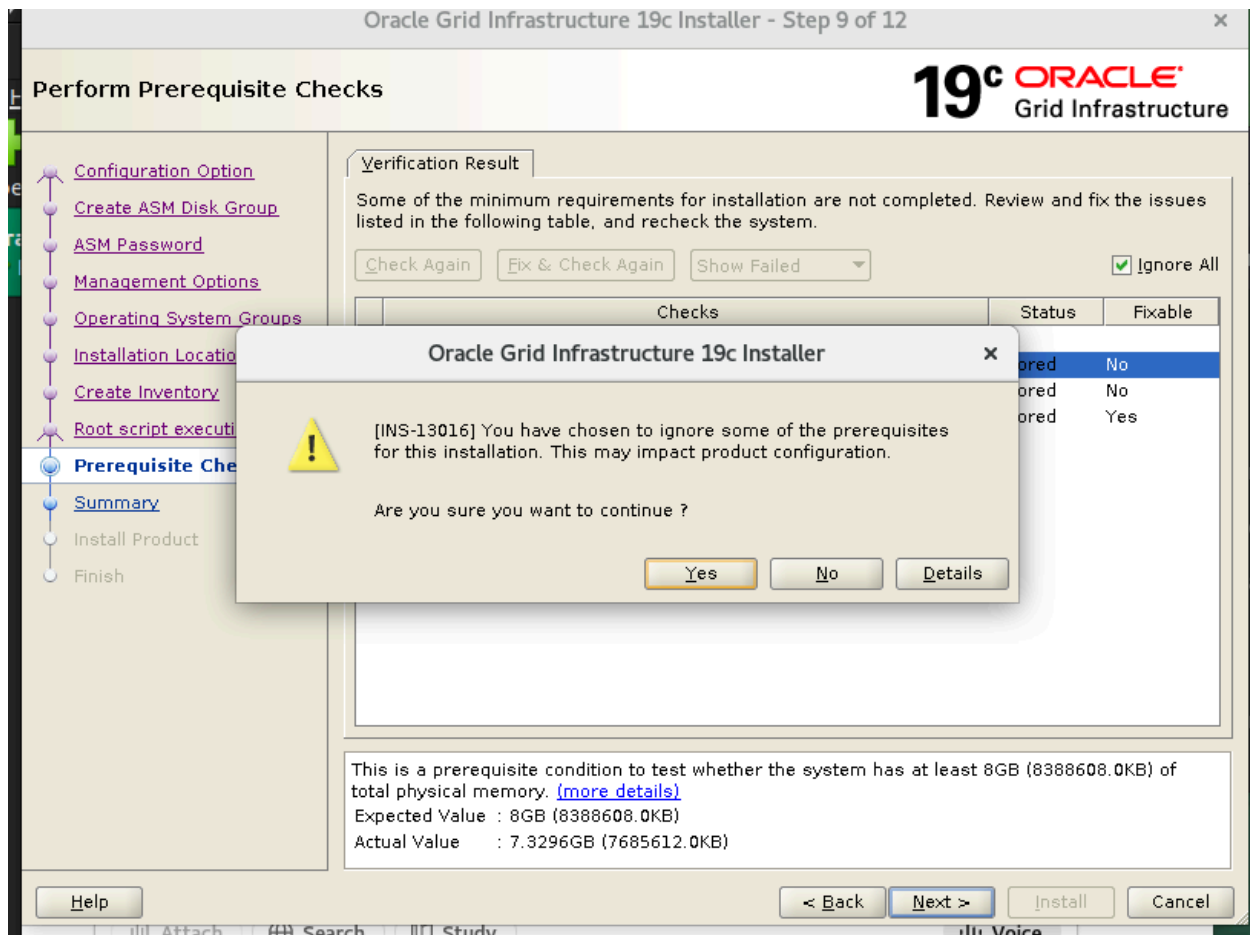


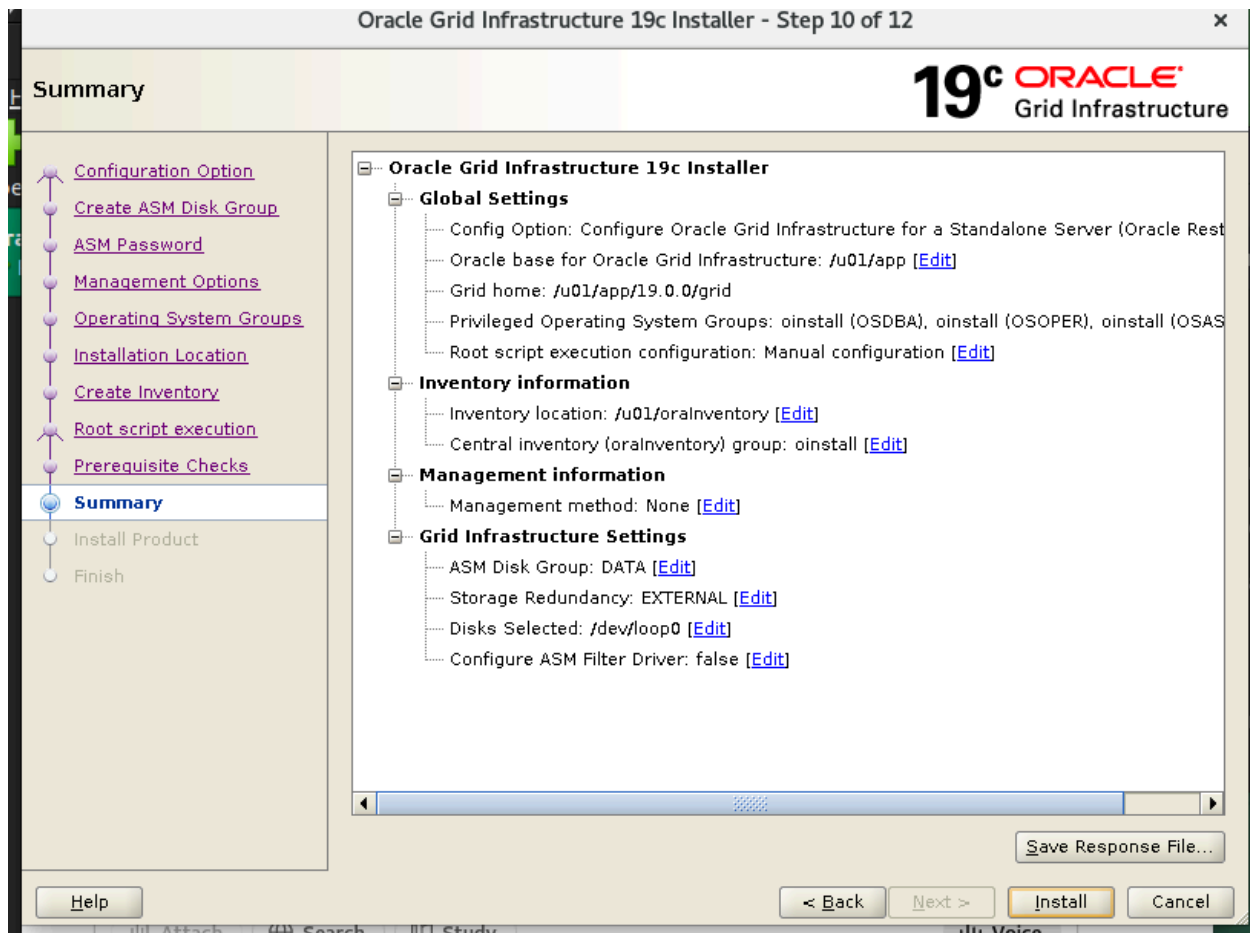
KARONGI COLLEGE

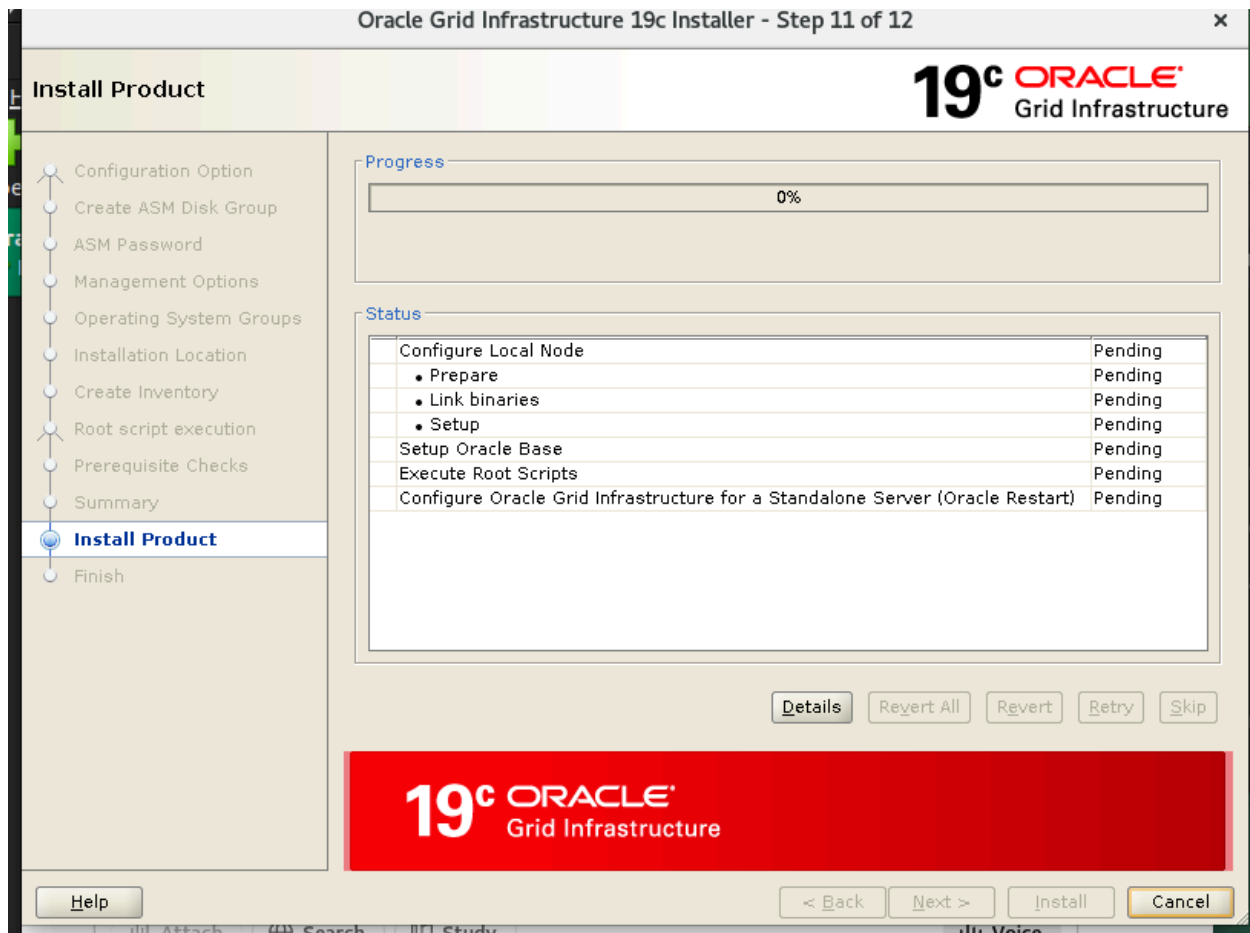
RWANDA POLYTECHNIC

68









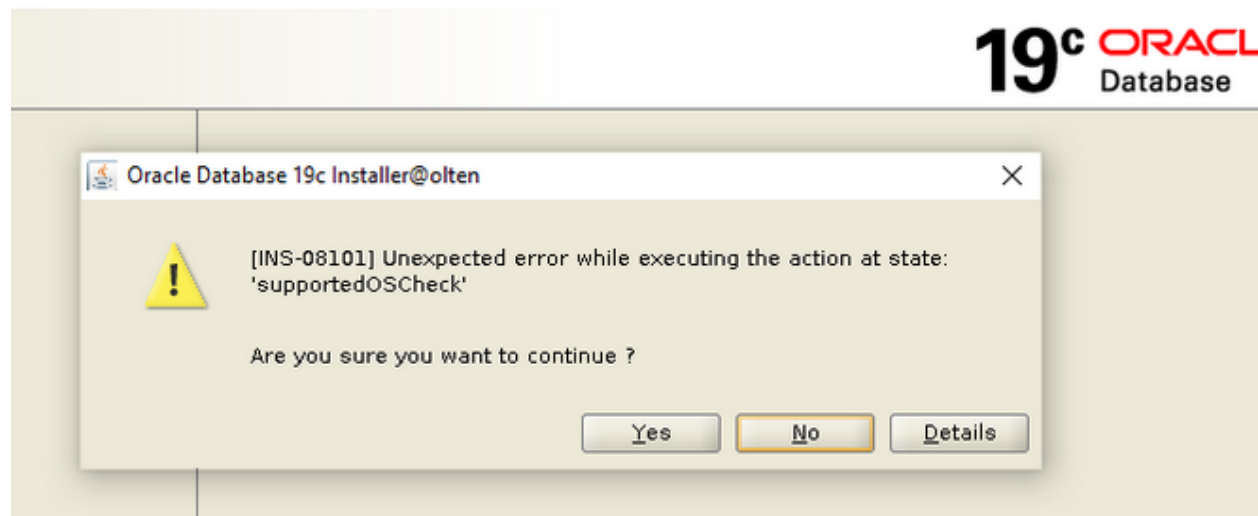
Database installation → login as oracle user:

su - oracle

cd /u01/app/oracle/product/19.0.0/dbhome_1

./runInstaller

If you face this error:



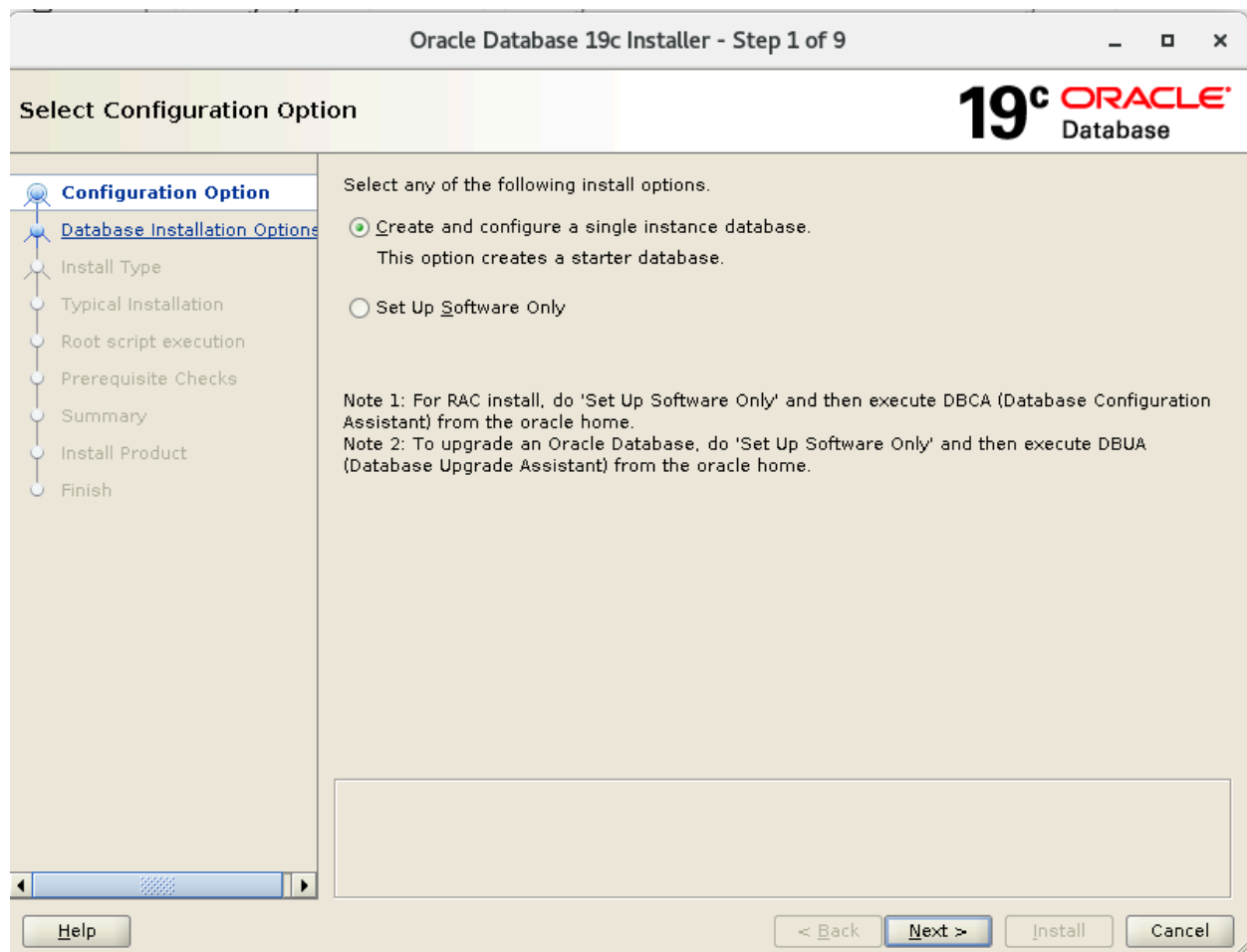
Set the OS parameter in your terminal to a previous version and re-start the Oracle Universal Installer.

export CV_ASSUME_DISTID=OEL7.8

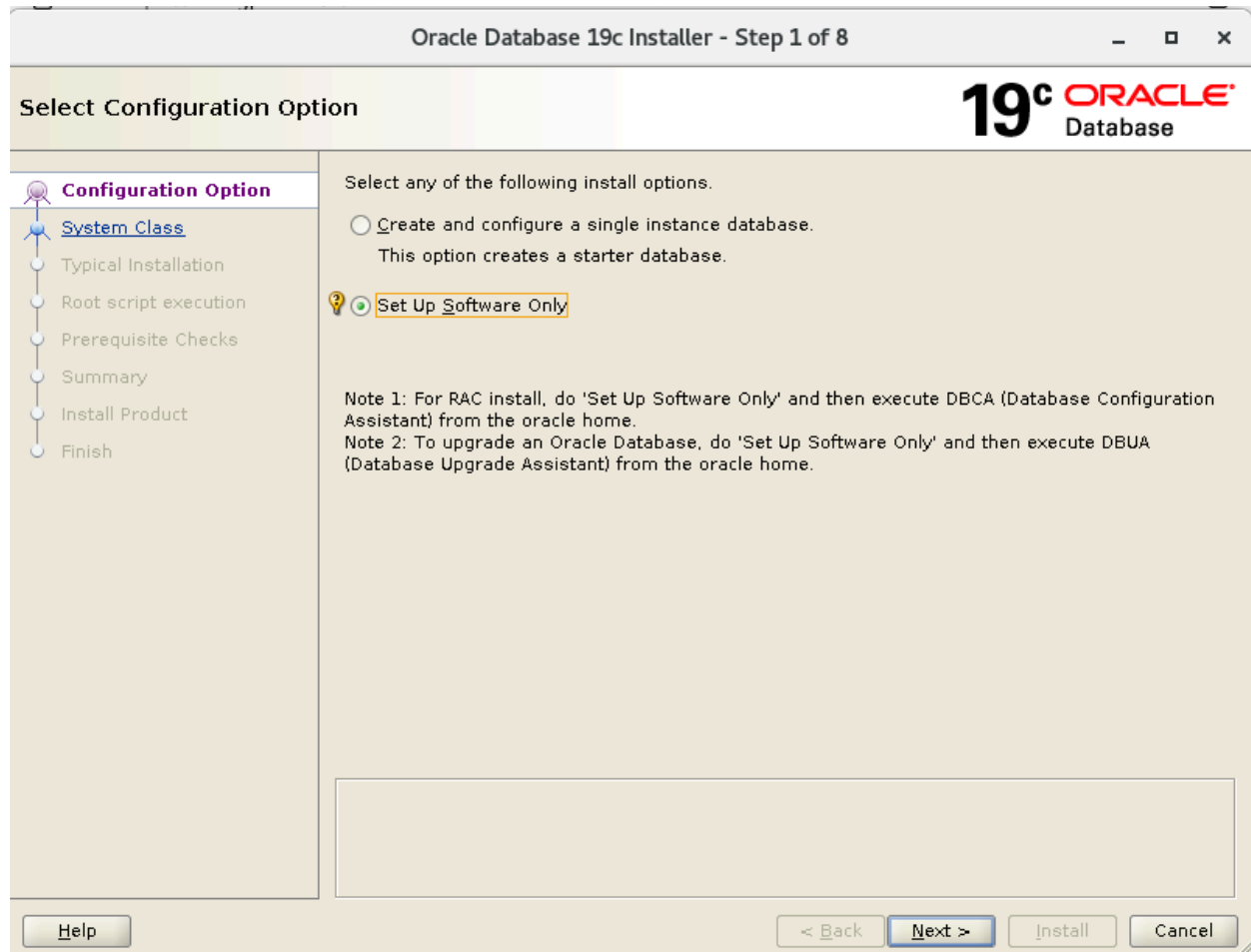
Or Edit the file cvu_config which is located in \$ORACLE_HOME/cv/admin, change the line from

Fallback to this distribution id

CV_ASSUME_DISTID=OEL7.8



If is for grid infrastructure set it like as follow:



Step 8: Install Visual Studio Code

```
sudo rpm --import https://packages.microsoft.com/keys/microsoft.asc
```

```
sudo dnf config-manager --add-repo https://packages.microsoft.com/yumrepos/vscode
```

```
sudo dnf install code -y
```

Step 9: Install Oracle SQL Developer Extension in VS Code

- Open VS Code.
- Go to Extensions (Ctrl+Shift+X).
- Search for **Oracle Developer Tools for VS Code** (by Oracle Corporation).
- Install it, then configure connection to your Oracle DB

2.1.1.5. Describing tools for administering an oracle database

a) SQL*Plus

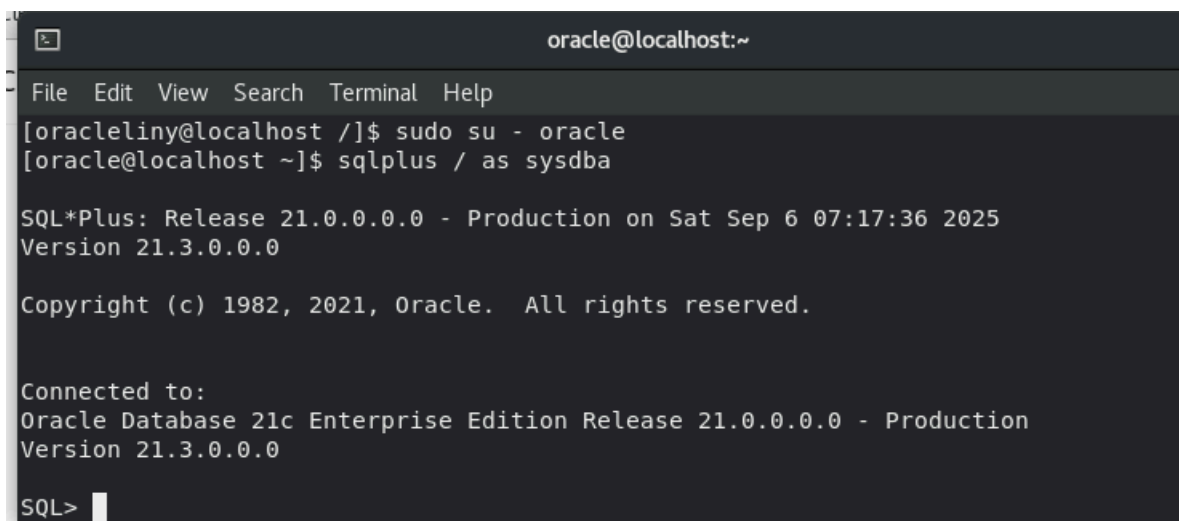
- Installed automatically with Oracle Database.

Test:

Sudo su - oracle

sqlplus / as sysdba

SQL>



```
oracle@localhost:~  
File Edit View Search Terminal Help  
[oracle@localhost ~]$ sudo su - oracle  
[oracle@localhost ~]$ sqlplus / as sysdba  
  
SQL*Plus: Release 21.0.0.0.0 - Production on Sat Sep 6 07:17:36 2025  
Version 21.3.0.0.0  
  
Copyright (c) 1982, 2021, Oracle. All rights reserved.  
  
Connected to:  
Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Production  
Version 21.3.0.0.0  
  
SQL>
```

SQL> SELECT name FROM v\$database;

SQL> SELECT instance_name, status FROM v\$instance;

b) DBCA (Database Configuration Assistant)

- Used for creating/configuring databases.

Configure .Xauthority

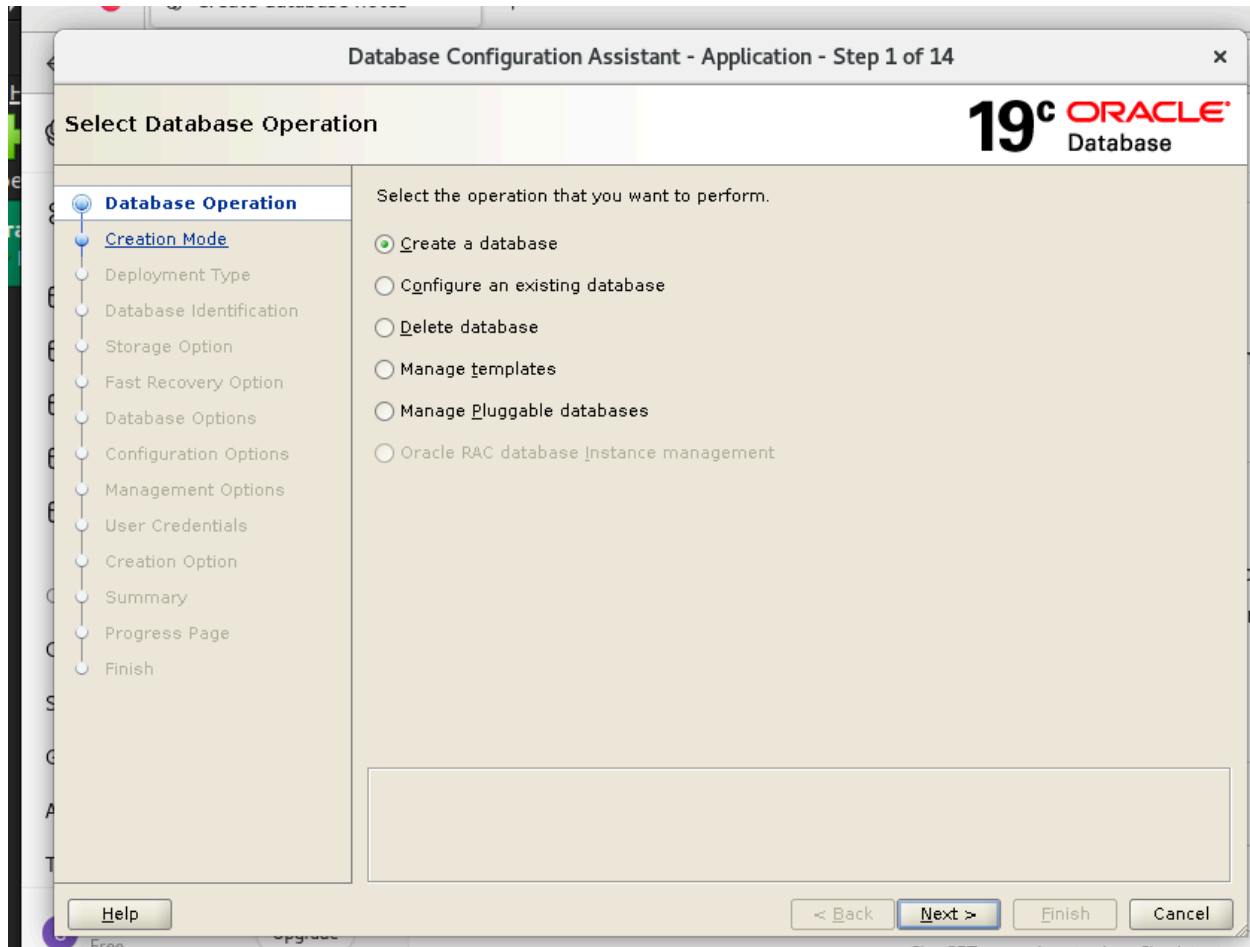
- In default (sudoer) user, create the **.Xauthority**: touch ~/.Xauthority
- Generate a new x11 authorization cookie:

echo \$DISPLAY

xauth generate :0 . trusted

- Check if the .Xauthority file is created: `ls -la ~/.Xauthority`
- Allow **oracle** user to access the X server: `xhost +SI:localuser:oracle`
- Switch to **oracle** user and set the display: `export DISPLAY=:0`

Start dbca:



Database Configuration Assistant - Create 'racdb1' database - Step 4 of 15

Specify Database Identification Details

19^c ORACLE[®] Database

Database Operation

Creation Mode

Deployment Type

Database Identification

Storage Option

Fast Recovery Option

Network Configuration

Data Vault Option

Configuration Options

Management Options

User Credentials

Creation Option

Summary

Progress Page

Finish

Provide a unique database identifier information. An Oracle database is uniquely identified by a Global database name, typically of the form "name.domain".

Global database name:

SID:

Service name:

☐ Create as Container database

A Container database can be used for consolidating multiple databases into a single database, and it enables database virtualization. A Container database (CDB) can have zero or more pluggable databases (PDB).

☒ Use Local Undo tablespace for PDBs

☐ Create an empty Container database

☒ Create a Container database with one or more PDBs

Number of PDBs:

PDB name:

Help

< Back

Next >

Finish

Cancel

Database Configuration Assistant - Create 'racdb1' database - Step 5 of 15

Select Database Storage Option

19c ORACLE Database

Database Operation

Creation Mode

Deployment Type

Database Identification

Storage Option

Fast Recovery Option

Network Configuration

Data Vault Option

Configuration Options

Management Options

User Credentials

Creation Option

Summary

Progress Page

Finish

☐

Use template file for database storage attributes

Storage type and location for database files will be picked up from the specified template (General Purpose or Transaction Processing).

☒

Use following for the database storage attributes

All the database files will be put at the specified location below. You can customize the name and location of each datafile in the subsequent screen.

Database files storage type:

Automatic Storage Management (ASM)

Database files location:

+DATA/{DB_UNIQUE_NAME}

Browse...

Oracle Managed files option will enable Oracle to automatically generate the names of the datafiles for simplified database management.

☒ Use Oracle-Managed Files (OMF)

Multiplex redo logs and control files...

File location variables...

Help

< Back

Next >

Finish

Cancel

Database Configuration Assistant - Create 'racdb1' database - Step 6 of 15

Select Fast Recovery Option

19c ORACLE Database

Choose the recovery options for the database.

☐ Specify Fast Recovery Area

Recovery files storage type: Automatic Storage Management (ASM)

Fast Recovery Area: +DATA

Fast Recovery Area size: 8256 MB

☐ Enable archiving

Database Operation
Creation Mode
Deployment Type
Database Identification
Storage Option
Fast Recovery Option
Network Configuration
Data Vault Option
Configuration Options
Management Options
User Credentials
Creation Option
Summary
Progress Page
Finish

Database Configuration Assistant - Create 'racdb1' database - Step 7 of 15

Specify Network Configuration Details

19c ORACLE Database

Database Operation

Creation Mode

Deployment Type

Database Identification

Storage Option

Fast Recovery Option

Network Configuration

Data Vault Option

Configuration Options

Management Options

User Credentials

Creation Option

Summary

Progress Page

Finish

Listener selection

Listeners from Grid Infrastructure home and Database Oracle home are listed below. Specify the listener name and port to create a new listener in database Oracle home.

	Name	Port	Oracle home	Status
☒	LISTENER	1521	/u01/app/19.0.0/grid	Up

☐ Create a new listener

Listener name:

Listener port:

Oracle home: /u01/app/oracle/product/19.0.0/dbhome_1

Help

< Back

Next >

Finish

Cancel

Database Configuration Assistant - Create 'racdb1' database - Step 8 of 15

19^c ORACLE[®]
Database

Select Oracle Data Vault Config Option

- Database Operation
- Creation Mode
- Deployment Type
- Database Identification
- Storage Option
- Fast Recovery Option
- Network Configuration
- Data Vault Option**
- Configuration Options
- Management Options
- User Credentials
- Creation Option
- Summary
- Progress Page
- Finish

☐ **Configure Oracle Database Vault**

Database Vault owner:

Password: Confirm password:

☐ **Create a separate account manager**

Account manager:

Password: Confirm password:

☐ **Configure Oracle Label Security**

☐ Configure Oracle Label Security with OJD

Database Configuration Assistant - Create 'racdb1' database - Step 9 of 15

Specify Configuration Options

19c ORACLE Database

- Database Operation
- Creation Mode
- Deployment Type
- Database Identification
- Storage Option
- Fast Recovery Option
- Network Configuration
- Data Vault Option
- Configuration Options**
- Management Options
- User Credentials
- Creation Option
- Summary
- Progress Page
- Finish

Memory | Sizing | Character sets | Connection mode | Sample schemas

☒ Use Automatic Shared Memory Management

SGA size: 2244 MB 390 2992 7480

PGA Size: 748 MB

☐ Use Manual Shared Memory Management

Shared pool size: 0 MB

Buffer cache size: 0 MB

Java pool size: 0 MB

Large pool size: 0 MB

PGA size: 0 MB

Total memory for database 0 MB

☐ Use Automatic Memory Management

Memory target: 2992 MB 390 2992 7480 40%

Help < Back Next > Finish Cancel

Database Configuration Assistant - Create 'racdb1' database - Step 10 of 15

Specify Management Options

19c ORACLE Database

Database Operation

Creation Mode

Deployment Type

Database Identification

Storage Option

Fast Recovery Option

Network Configuration

Data Vault Option

Configuration Options

Management Options

User Credentials

Creation Option

Summary

Progress Page

Finish

Specify the management options for the database.

☐ Configure Enterprise Manager (EM) database express

EM database express port: 5500

☐ Register with Enterprise Manager (EM) cloud control

O_MS host:

O_MS port:

EM admin username:

EM admin password:

ASMSNMP user password:

Help

< Back

Next >

Finish

Cancel

Database Configuration Assistant - Create 'racdb1' database - Step 11 of 15

Specify Database User Credentials

19c ORACLE Database

Database Operation

Creation Mode

Deployment Type

Database Identification

Storage Option

Fast Recovery Option

Network Configuration

Data Vault Option

Configuration Options

Management Options

User Credentials

Creation Option

Summary

Progress Page

Finish

You must specify passwords for the following user accounts in the new database for security reasons.

☐ Use different administrative passwords

	Password	Confirm password
<u>S</u> YS		
<u>S</u> YSTEM		

☒ Use the same administrative password for all accounts

Password: Confirm password:

Help

< Back

Next >

Finish

Cancel

Database Configuration Assistant - Create 'racdb1' database - Step 12 of 15

19c ORACLE Database

Select Database Creation Option

- Database Operation
- Creation Mode
- Deployment Type
- Database Identification
- Storage Option
- Fast Recovery Option
- Network Configuration
- Data Vault Option
- Configuration Options
- Management Options
- User Credentials**
- Creation Option**
- Summary
- Progress Page
- Finish

Select the database creation options.

☒ **Create database**

Specify the SQL scripts you want to run after the database is created. The scripts are run in the order listed below.

Post DB creation scripts: [Browse...](#)

☐ **Save as a database template**

Template name:

Template location: [Browse...](#)

Description:

☐ **Generate database creation scripts**

Destination directory: [Browse...](#)

Following advanced configuration options can be used to configure initialization parameters and customize database storage locations.

[All Initialization Parameters...](#) [Customize Storage Locations...](#)

[Help](#) [< Back](#) [Next >](#) [Finish](#) [Cancel](#)

c) NetCA (Network Configuration Assistant)

- Configures listener.

Run:

netca

- Steps:
 - Add Listener → Port 1521
 - Start automatically

Check listener:

lsnrctl status

d) Oracle Enterprise Manager (OEM Database Express)

Oracle EM Express is installed automatically with Oracle Database 19c. It provides a lightweight browser-based management console which runs inside the **Oracle XML DB (XDB)** component.

How to Access

1. Make sure the **listener** and **database** are started:

In oracle user run this to start the listener: **lsnrctl start**

In oracle user run this to access the database server: **sqlplus / as sysdba**

In server (SQL>), start the oracle database instance: **startup;**

2. Check EM Express port (default is 5500):

```
SELECT dbms_xdb_config.getHttpsPort() FROM dual;
```

3. If it's 0, start it:

```
SQL> EXEC DBMS_XDB_CONFIG.SETHTTPSPORT(5500);
```

```
SQL> ALTER SYSTEM SET dispatchers='(PROTOCOL=TCP)(SERVICE=XDB)' SCOPE=BOTH;  
System altered.
```

3. Open a browser and navigate to:

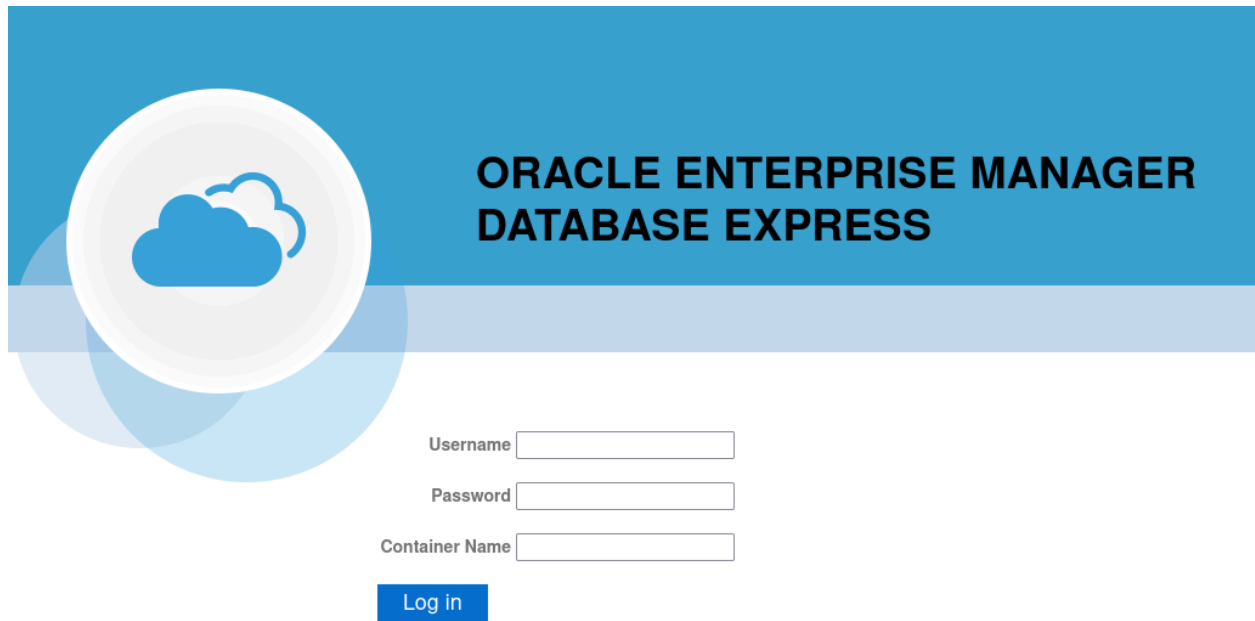
<https://localhost:5500/em> or replace localhost with your IP address (check it with: ip a)

4. Login Screen

Username: SYS or SYSTEM

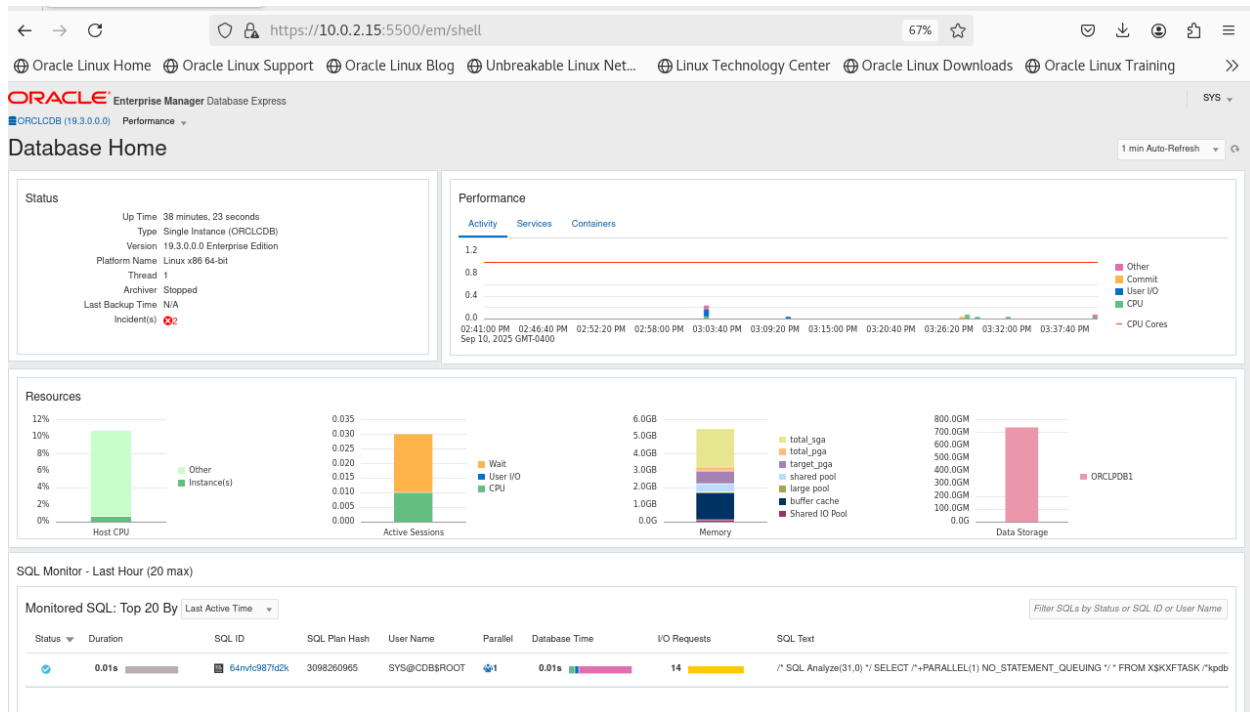
Password: database password (set during DBCA)

Container Name: Leave blank



5. After Login

- You'll see the EM Express **home dashboard**.
- From here you can:
 - View performance charts (CPU, sessions, SQL response time)
 - Manage users and security
 - Monitor storage and tablespaces
 - View PDBs and container details



Troubleshooting

- If browser asks for extra username/password before Oracle login:
 - Use SYS, SYSTEM, or unlock and use XDB user.
- If the page does not load:

Make sure firewall allows port 5500:

```
sudo firewall-cmd --permanent --add-port=5500/tcp
sudo firewall-cmd --reload
```

To switch to HTTP (no SSL popups):

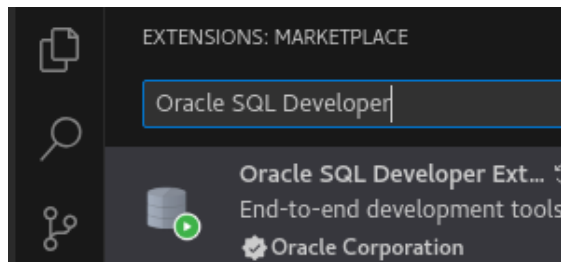
```
EXEC DBMS_XDB_CONFIG.SETHTTPSPORT(0);
EXEC DBMS_XDB_CONFIG.SETHTTPSPORT(5501);
```

e) Oracle SQL Developer

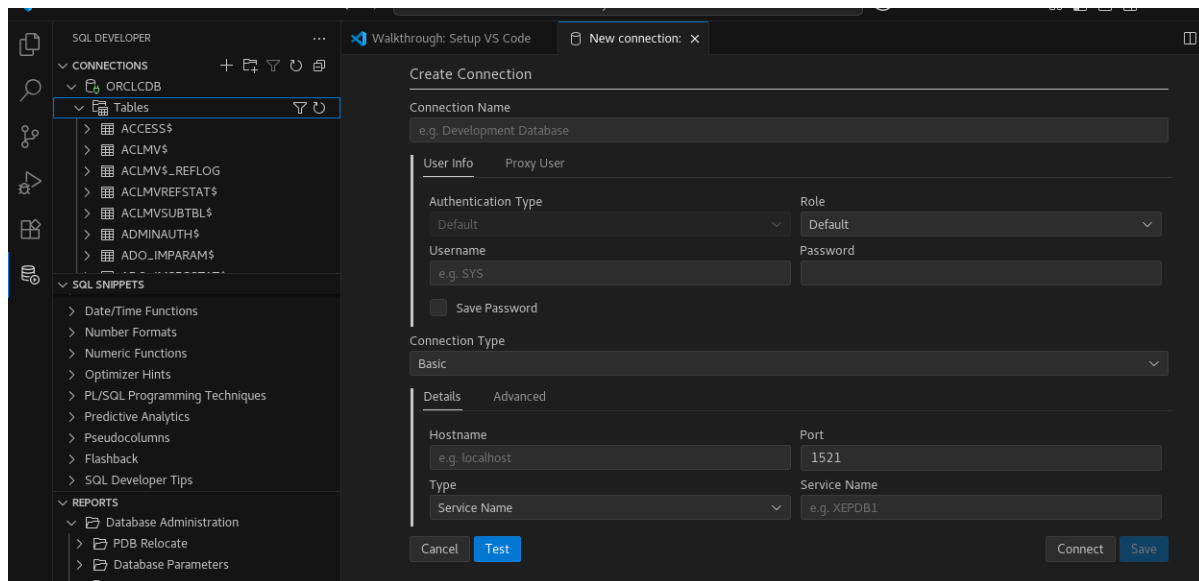
Oracle SQL Developer may be installed as an extension in vs code. First of all we need vs code

- `sudo rpm --import https://packages.microsoft.com/keys/microsoft.asc`
- `sudo dnf config-manager --add-repo https://packages.microsoft.com/yumrepos/vscode --releasever=9code`
- `sudo dnf install code -y`

After installing vs code, search Oracle SQL Developer extension and install it:



After successfully installed, you may create a connection of the mounted database.



To check the mounted database, you can use:

```
SQL> SELECT name FROM v$database;
```

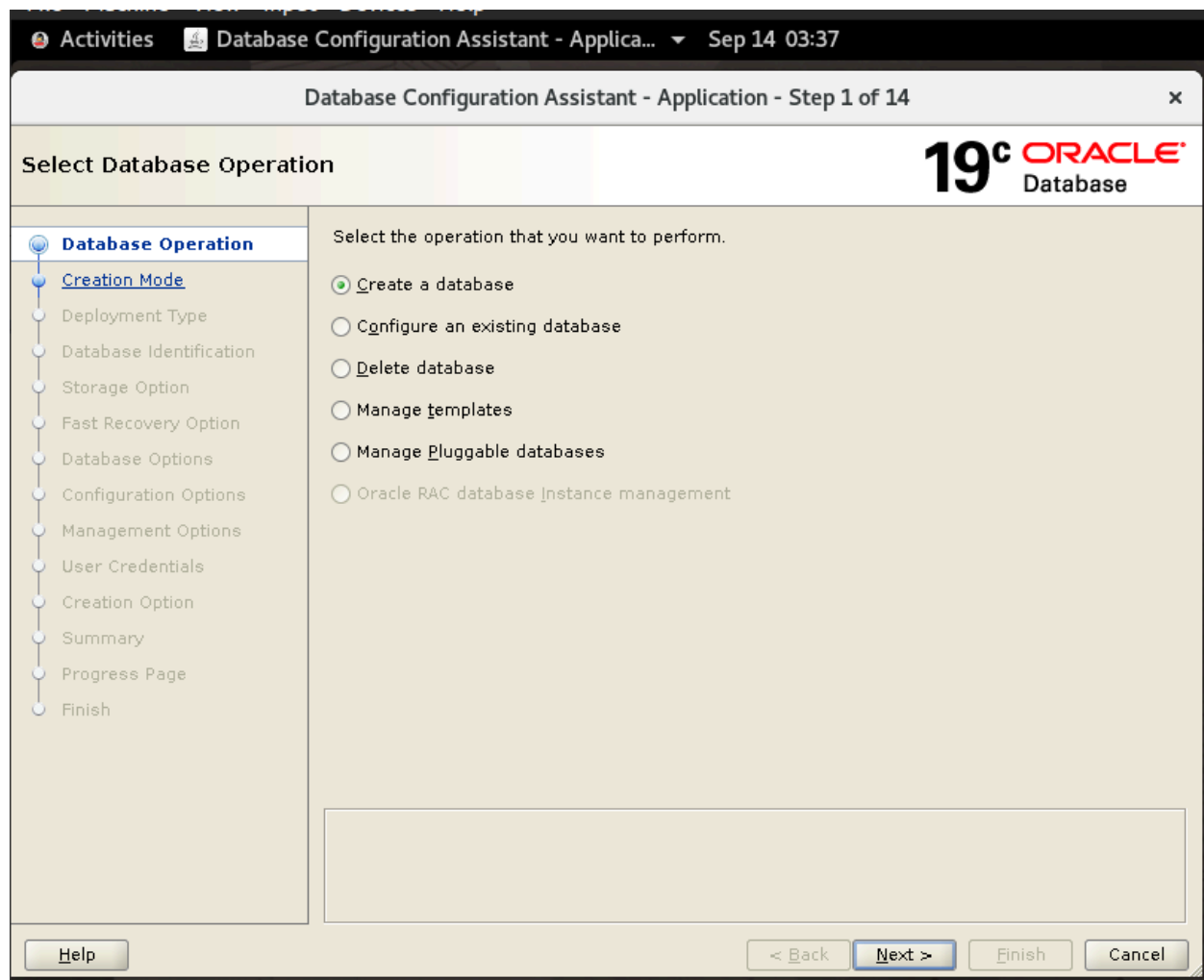
You can check also instance:

```
SQL> SELECT instance_name, status FROM v$instance;
```

2.1.2. Creating oracle database

2.1.2.1. Creating database with DBCA

```
oracle@localhost:~  
File Edit View Search Terminal Help  
[oraclelinux@localhost ~]$ touch ~/.Xauthority  
[oraclelinux@localhost ~]$ xauth generate :0 . trusted  
[oraclelinux@localhost ~]$ ls la ~/.Xauthority  
ls: cannot access 'la': No such file or directory  
/home/oraclelinux/.Xauthority  
[oraclelinux@localhost ~]$ sudo ls la ~/.Xauthority  
[sudo] password for oraclelinux:  
ls: cannot access 'la': No such file or directory  
/home/oraclelinux/.Xauthority  
[oraclelinux@localhost ~]$ ls -la ~/.Xauthority  
-rw-----. 1 oraclelinux oraclelinux 66 Sep 14 03:23 /home/oraclelinux/.Xauthor  
ity  
[oraclelinux@localhost ~]$ xhost +SI:localuser:oracle  
localuser:oracle being added to access control list  
[oraclelinux@localhost ~]$ sudo su - oracle  
[oracle@localhost ~]$ export DISPLAY=:0  
[oracle@localhost ~]$ dbca
```



Activities Database Configuration Assistant - Applica... Sep 14 03:38

Database Configuration Assistant - Create a database - Step 2 of 14

Select Database Creation Mode

19c ORACLE Database

- Database Operation
- Creation Mode**
- Deployment Type
- Database Identification
- Storage Option
- Fast Recovery Option
- Database Options
- Configuration Options
- Management Options
- User Credentials
- Creation Option
- Summary
- Progress Page
- Finish

☐ Typical configuration

Global database name:

Storage type:

Database files location:

Fast Recovery Area (FRA):

Database character set:

Administrative password:

Confirm password:

☒ Create as Container database

Pluggable database name:

☒ **Advanced configuration**

Activities
Database Configuration Assistant - Applica...
Sep 14 03:39

Database Configuration Assistant - Create a database - Step 3 of 14

19^c ORACLE[®] Database

Select Database Deployment Type

- Database Operation
- Creation Mode
- Deployment Type**
- Database Identification
- Storage Option
- Fast Recovery Option
- Database Options
- Configuration Options
- Management Options
- User Credentials
- Creation Option
- Summary
- Progress Page
- Finish

Select the type of database you want to create.

Database type: Oracle Real Application Cluster (RAC) datab...

Configuration type: Admin Managed

Select a template for your database.

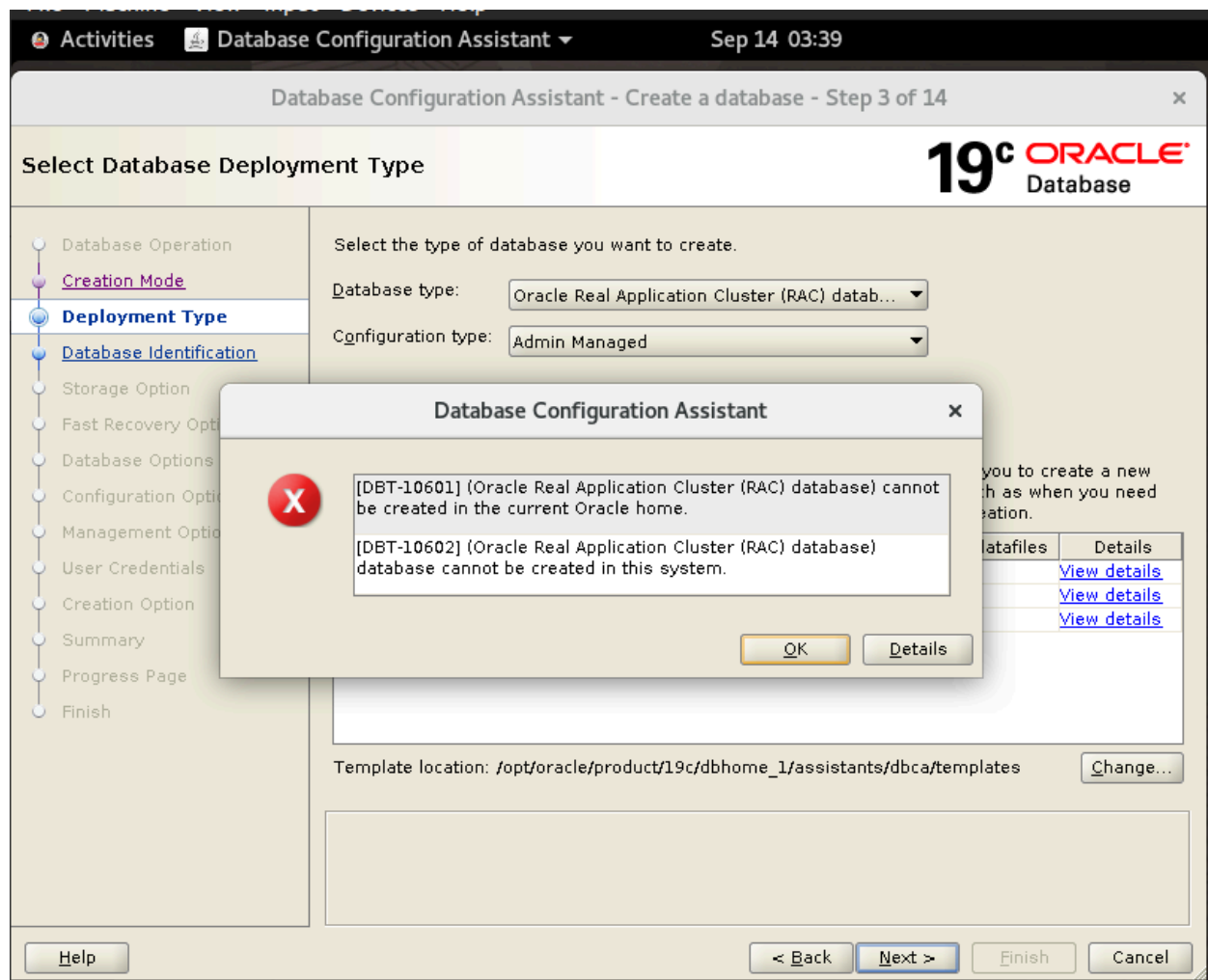
Templates that include datafiles contain pre-created databases. They allow you to create a new database quickly. Use templates without datafiles only when necessary, such as when you need to change attributes like block size that cannot be altered after database creation.

	Template name	Include datafiles	Details
☐	Data Warehouse	Yes	View details
☒	General Purpose or Transaction Processing	Yes	View details
☐	Custom Database	No	View details

Template location: /opt/oracle/product/19c/dbhome_1/assistants/dbca/templates Change...

Help

< Back
Next >
Finish
Cancel



Oracle RAC can't be used in the .rpm version of oracle. It may require to install grid infrastructure oracle which supports ASM and RAC.

Activities
Database Configuration Assistant - Create ...
Sep 14 03:41

Database Configuration Assistant - Create a database - Step 3 of 14

19^c ORACLE[®] Database

Select Database Deployment Type

- Database Operation
- Creation Mode
- Deployment Type**
- Database Identification
- Storage Option
- Fast Recovery Option
- Database Options
- Configuration Options
- Management Options
- User Credentials
- Creation Option
- Summary
- Progress Page
- Finish

Select the type of database you want to create.

Database type: Oracle Single Instance database

Configuration type: Admin Managed

Select a template for your database.

Templates that include datafiles contain pre-created databases. They allow you to create a new database quickly. Use templates without datafiles only when necessary, such as when you need to change attributes like block size that cannot be altered after database creation.

	Template name	Include datafiles	Details
☐	Data Warehouse	Yes	View details
☒	General Purpose or Transaction Processing	Yes	View details
☐	Custom Database	No	View details

Template location: /opt/oracle/product/19c/dbhome_1/assistants/dbca/templates Change...

Help

< Back
Next >
Finish
Cancel

Activities Database Configuration Assistant - Create ... Sep 14 03:42

Database Configuration Assistant - Create a database - Step 4 of 14

Specify Database Identification Details

19c ORACLE Database

- Database Operation
- Creation Mode
- Deployment Type
- Database Identification**
- Storage Option
- Fast Recovery Option
- Database Options
- Configuration Options
- Management Options
- User Credentials
- Creation Option
- Summary
- Progress Page
- Finish

Provide a unique database identifier information. An Oracle database is uniquely identified by a Global database name, typically of the form "name.domain".

Global database name:

SID:

Service name:

☒ Create as Container database

A Container database can be used for consolidating multiple databases into a single database, and it enables database virtualization. A Container database (CDB) can have zero or more pluggable databases (PDB).

☒ Use Local Undo tablespace for PDBs

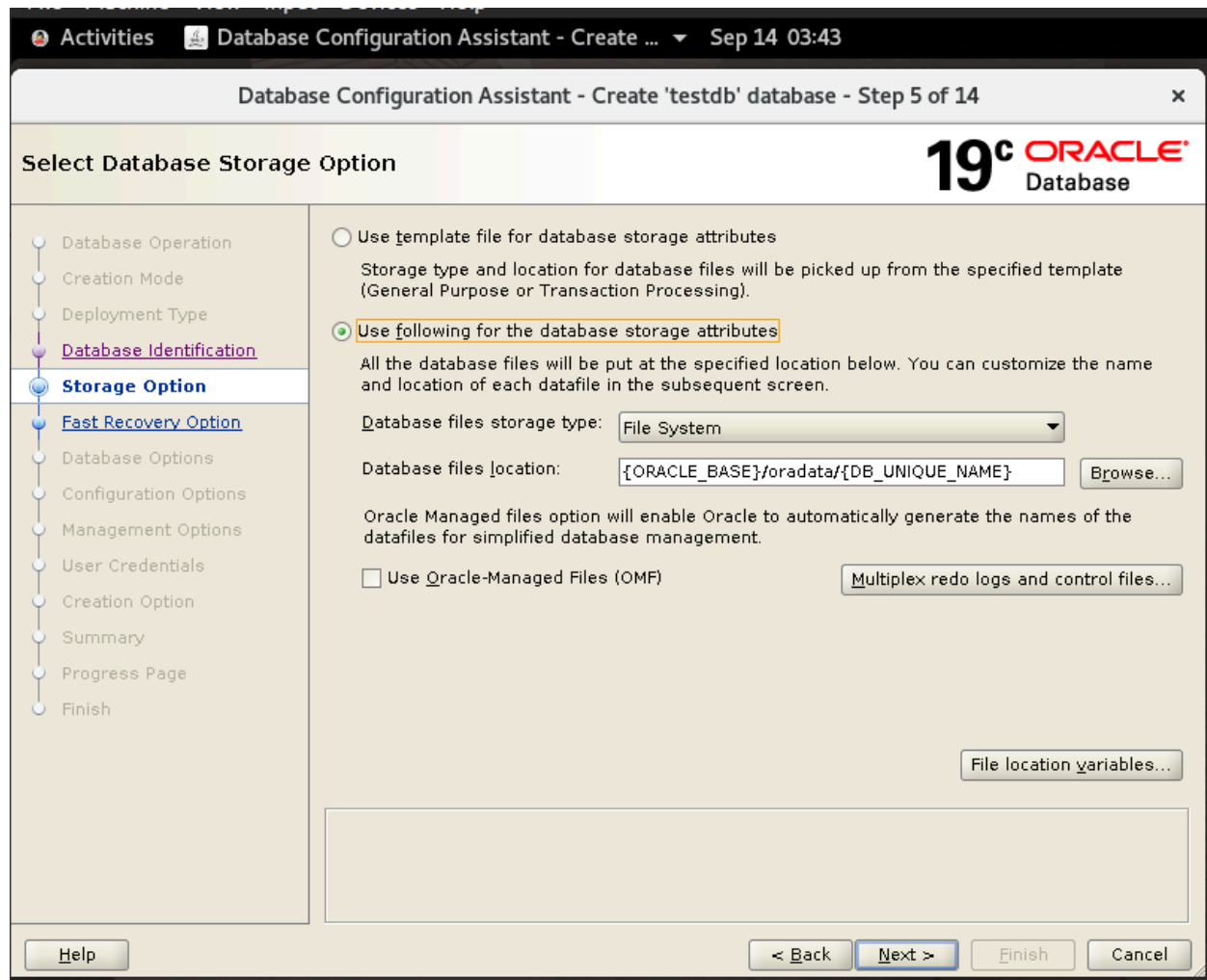
☐ Create an empty Container database

☒ Create a Container database with one or more PDBs

Number of PDBs:

PDB name prefix:

Help < Back Next > Finish Cancel



Activities Database Configuration Assistant - Create ... Sep 14 03:43

Database Configuration Assistant - Create 'testdb' database - Step 6 of 14

Select Fast Recovery Option

19c ORACLE Database

- Database Operation
- Creation Mode
- Deployment Type
- Database Identification
- Storage Option
- Fast Recovery Option**
- Database Options
- Configuration Options
- Management Options
- User Credentials
- Creation Option
- Summary
- Progress Page
- Finish

Choose the recovery options for the database.

☒ Specify Fast Recovery Area

Recovery files storage type: File System

Fast Recovery Area: {ORACLE_BASE}/fast_recovery_area/{DB_UNIQUE_} [Browse...](#)

Fast Recovery Area size: 17208 MB

☒ Enable archiving [Edit archive mode parameters...](#)

[Help](#) [< Back](#) [Next >](#) [Finish](#) [Cancel](#)

Activities Database Configuration Assistant - Create ... Sep 14 03:44

Database Configuration Assistant - Create 'testdb' database - Step 7 of 14

Specify Network Configuration Details

- Database Operation
- Creation Mode
- Deployment Type
- Database Identification
- Storage Option
- Fast Recovery Option
- Network Configuration**
- Configuration Options
- Management Options
- User Credentials
- Creation Option
- Summary
- Progress Page
- Finish

Listener selection

Listeners from current Oracle home are listed below. Specify the listener name and port to create a new listener in current Oracle home.

	Name	Port	Oracle home	Status
✓	LISTENER	1521	/opt/oracle/product/19c/dbhome_1	Down

☐ Create a new listener

Listener name:

Listener port:

Oracle home: /opt/oracle/product/19c/dbhome_1

Help

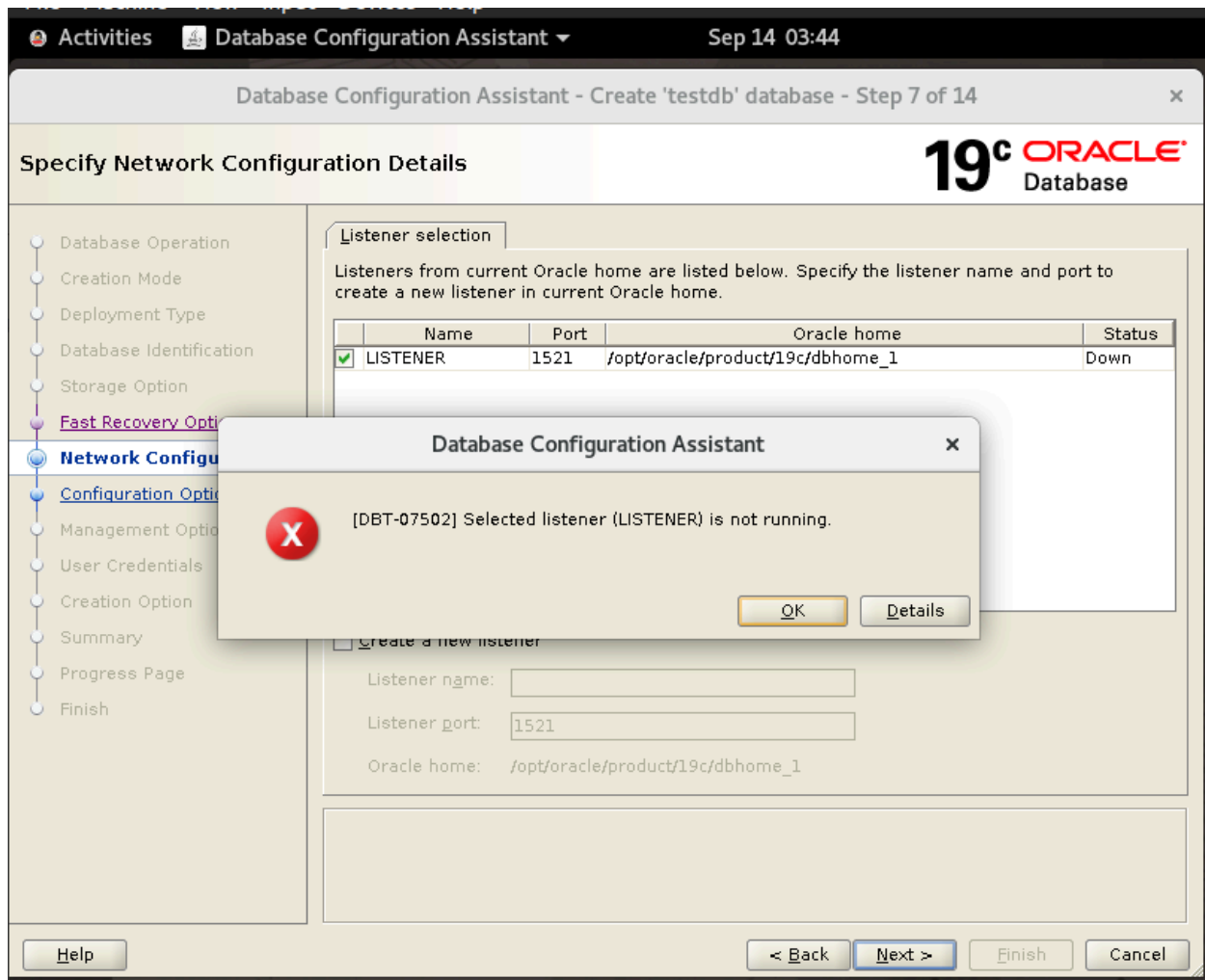
< Back

Next >

Finish

Cancel

19^c ORACLE[®]
Database



Activities Terminal Sep 14 03:45

Database Configuration Assistant - Create 'testdb' database - Step 7 of 14

oracle@localhost:~

```
File Edit View Search Terminal Tabs Help

oracle@localhost:~
[oraclelinux@localhost ~]$ sudo su - oracle
[sudo] password for oraclelinux:
[oracle@localhost ~]$ lsnrctl status

LSNRCTL for Linux: Version 19.0.0.0.0 - Production on 14-SEP-2025 03:44:58

Copyright (c) 1991, 2019, Oracle. All rights reserved.

Connecting to (DESCRIPTION=(ADDRESS=(PROTOCOL=TCP)(HOST=localhost)(PORT=1521)))
TNS-12541: TNS:no listener
TNS-12560: TNS:protocol adapter error
TNS-00511: No listener
Linux Error: 111: Connection refused
Connecting to (DESCRIPTION=(ADDRESS=(PROTOCOL=IPC)(KEY=EXTPROC1521)))
TNS-12541: TNS:no listener
TNS-12560: TNS:protocol adapter error
TNS-00511: No listener
Linux Error: 111: Connection refused
```



```

Linux Error: 111: Connection refused
[oracle@localhost ~]$ lsnrctl start

LSNRCTL for Linux: Version 19.0.0.0.0 - Production on 14-SEP-2025 03:45:06

Copyright (c) 1991, 2019, Oracle. All rights reserved.

Starting /opt/oracle/product/19c/dbhome_1/bin/tnslsnr: please wait...

TNSLSNR for Linux: Version 19.0.0.0.0 - Production
System parameter file is /opt/oracle/product/19c/dbhome_1/network/admin/listener.ora
Log messages written to /opt/oracle/diag/tnslsnr/localhost/listener/alert/log.xml
Listening on: (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=localhost)(PORT=1521)))
Listening on: (DESCRIPTION=(ADDRESS=(PROTOCOL=ipc)(KEY=EXTPROC1521)))

Connecting to (DESCRIPTION=(ADDRESS=(PROTOCOL=TCP)(HOST=localhost)(PORT=1521)))
STATUS of the LISTENER
-----
Alias                     LISTENER
Version                   TNSLSNR for Linux: Version 19.0.0.0.0 - Production
Start Date                14-SEP-2025 03:45:08
Uptime                    0 days 0 hr. 0 min. 0 sec
Trace Level               off
Security                  ON: Local OS Authentication
SNMP                      OFF
Listener Parameter File   /opt/oracle/product/19c/dbhome_1/network/admin/listener.ora
Listener Log File         /opt/oracle/diag/tnslsnr/localhost/listener/alert/log.xml
Listening Endpoints Summary...
  (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=localhost)(PORT=1521)))
  (DESCRIPTION=(ADDRESS=(PROTOCOL=ipc)(KEY=EXTPROC1521)))
The listener supports no services
The command completed successfully

```

Database Configuration Assistant - Create 'testdb' database - Step 9 of 15

Specify Configuration Options

19^c ORACLE[®] Database

- Database Operation
- Creation Mode
- Deployment Type
- Database Identification
- Storage Option
- Fast Recovery Option
- Network Configuration
- Data Vault Option
- Configuration Options**
- Management Options
- User Credentials
- Creation Option
- Summary
- Progress Page
- Finish

Memory
Sizing
Character sets
Connection mode

☒ Use Automatic Shared Memory Management

SGA size:

2240

MB

390

2987

7468

PGA Size:

747

MB

☐ Use Manual Shared Memory Management

Shared pool size:

0

MB

Buffer cache size:

0

MB

Java pool size:

0

MB

Large pool size:

0

MB

PGA size:

0

MB

Total memory for database 0 MB

☐ Use Automatic Memory Management

Memory target:

2987

MB

390

2987

7468

39%

Help

< Back
Next >
Finish
Cancel

Database Configuration Assistant - Create 'testdb' database - Step 10 of 15

Specify Management Options

19c ORACLE Database

Database Operation

Creation Mode

Deployment Type

Database Identification

Storage Option

Fast Recovery Option

Network Configuration

Data Vault Option

Configuration Options

Management Options

User Credentials

Creation Option

Summary

Progress Page

Finish

Specify the management options for the database.

☒ Configure Enterprise Manager (EM) database express

EM database express port: 5500

☐ Configure EM database express port as global port

☐ Register with Enterprise Manager (EM) cloud control

OMS host:

OMS port:

EM admin username:

EM admin password:

Help

< Back

Next >

Finish

Cancel



RWANDA POLYTECHNIC

KARONGI COLLEGE

106

Database Configuration Assistant - Create 'testdb' database - Step 11 of 15

Specify Database User Credentials
19c ORACLE Database

- Database Operation
- Creation Mode
- Deployment Type
- Database Identification
- Storage Option
- Fast Recovery Option
- Network Configuration
- Data Vault Option
- Configuration Options
- Management Options
- User Credentials**
- Creation Option
- Summary
- Progress Page
- Finish

You must specify passwords for the following user accounts in the new database for security reasons.

☒ Use different administrative passwords

	Password	Confirm password
SYS		
SYSTEM		
PDBADMIN		

☐ Use the same administrative password for all accounts

Password: Confirm password:

Help

< Back
Next >
Finish
Cancel

Database Configuration Assistant - Create 'testdb' database - Step 12 of 15

Select Database Creation Option

19c ORACLE
Database

Database Operation

Creation Mode

Deployment Type

Database Identification

Storage Option

Fast Recovery Option

Network Configuration

Data Vault Option

Configuration Options

Management Options

User Credentials

Creation Option

Summary

Progress Page

Finish

Select the database creation options.

Create database

Specify the SQL scripts you want to run after the database is created. The scripts are run in the order listed below.
Post DB creation scripts:

Browse...

Save as a database template

Template name:

dbca_template_2025-09-14_03-27-0

Template location:

/opt/oracle/product/19c/dbhome_1/assistants/dbca/template

Browse...

Description:

Generate database creation scripts

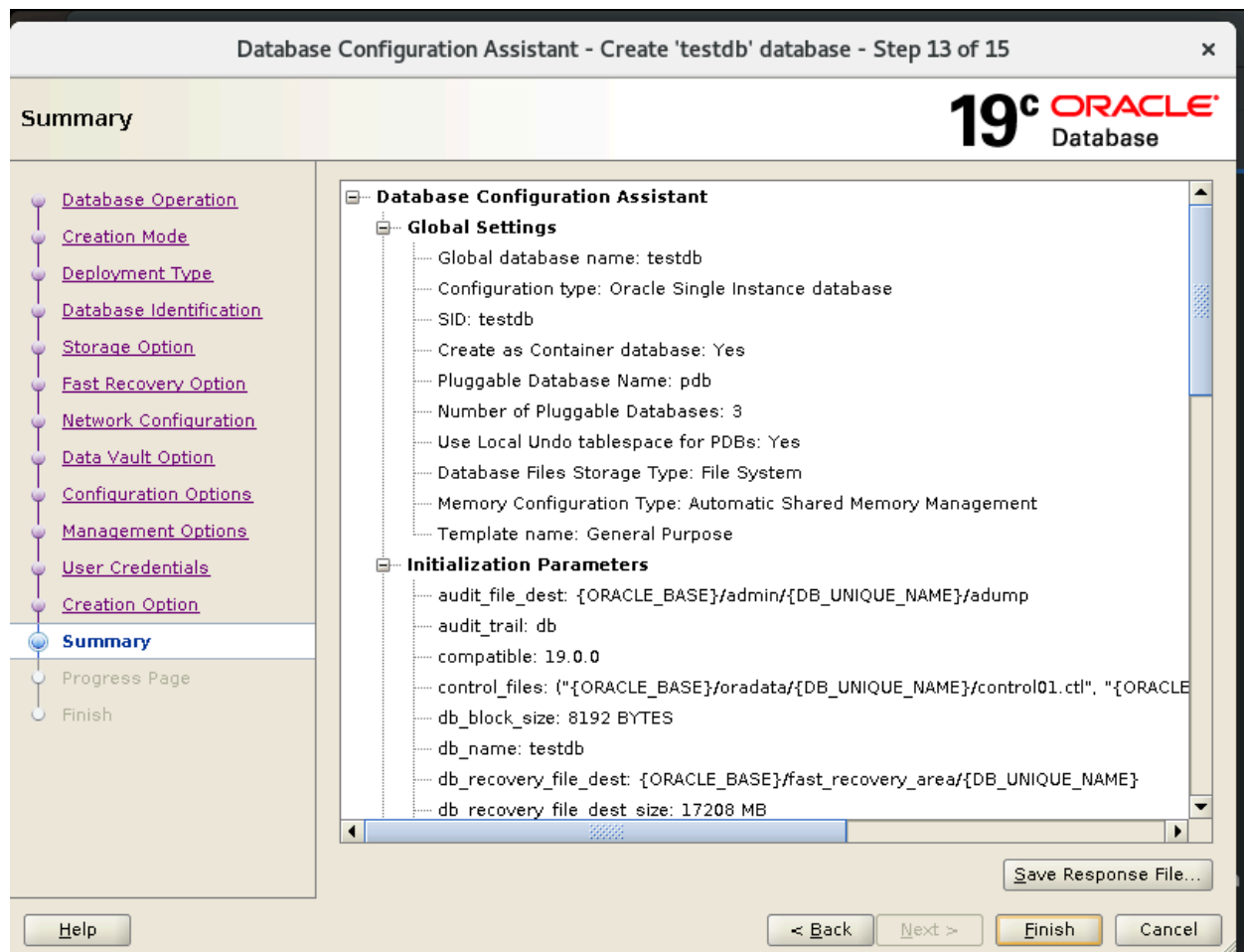
Destination directory:

{ORACLE_BASE}/admin/{DB_UNIQUE_NAME}/scripts

Browse...

Following advanced configuration options can be used to configure initialization parameters and customize database storage locations.

All Initialization Parameters...Customize Storage Locations...



2.1.2.2. Creating database with CREATE DATABASE statement

In oracle user, type: sqlplus / as sysdba

```

Version 19.3.0.0.0

Copyright (c) 1982, 2019, Oracle. All rights reserved.

Connected to:
Oracle Database 19c Enterprise Edition Release 19.0.0.0.0 - Production
Version 19.3.0.0.0

SQL> SELECT status FROM v$instance;

STATUS
-----
OPEN

SQL> SHUTDOWN IMMEDIATE;
Database closed.
Database dismounted.
ORACLE instance shut down.
SQL> STARTUP NOMOUNT;
ORACLE instance started.

Total System Global Area 2348809512 bytes
Fixed Size 9137448 bytes
Variable Size 520093696 bytes
Database Buffers 1811939328 bytes
Redo Buffers 7639040 bytes
SQL> SELECT status FROM v$instance;

STATUS
-----
STARTED

SQL>

```

```

[oracle@localhost ~]$ export ORACLE_SID=MYDB
[oracle@localhost ~]$ mkdir -p /opt/oracle/dbs
[oracle@localhost ~]$ vi /opt/oracle/dbs/initMYDB.ora

```

In initMYDB.ora, paste the following:

```

db_name='MYDB'
control_files='/opt/oracle/oradata/MYDB/control01.ctl','/opt/oracle/oradata/MYDB/control02.ctl'
memory_target=1G
processes=150
audit_file_dest='/opt/oracle/admin/MYDB/adump'
db_block_size=8192
undo_management=auto
undo_tablespace=undotbs1
diagnostic_dest='/opt/oracle'

```

```

[oracle@localhost ~]$ mkdir -p /opt/oracle/admin/MYDB/adump
[oracle@localhost ~]$ sqlplus / as sysdba

SQL*Plus: Release 19.0.0.0.0 - Production on Sun Sep 14 04:22:15 2025
Version 19.3.0.0.0

Copyright (c) 1982, 2019, Oracle. All rights reserved.

Connected to an idle instance.

SQL> STARTUP NOMOUNT PFILE='/opt/oracle/dbs/initMYDB.ora';
ORACLE instance started.

Total System Global Area 1073737800 bytes
Fixed Size                  8904776 bytes
Variable Size               616562688 bytes
Database Buffers            444596224 bytes
Redo Buffers                 3674112 bytes
SQL> █

```

```

version 19.3.0.0.0
[oracle@localhost ~]$ mkdir -p /opt/oracle/oradata/MYDB
[oracle@localhost ~]$ █

```

In sqlplus, create database now:

```

CREATE DATABASE MYDB
  USER SYS IDENTIFIED BY "SYS_12345"
  USER SYSTEM IDENTIFIED BY "SYSTEM_12345"
  LOGFILE GROUP 1 ('/opt/oracle/oradata/MYDB/redo01.log') SIZE 100M,
           GROUP 2 ('/opt/oracle/oradata/MYDB/redo02.log') SIZE 100M
  MAXLOGFILES 5
  MAXLOGMEMBERS 5
  MAXDATAFILES 100
  CHARACTER SET AL32UTF8
  NATIONAL CHARACTER SET AL16UTF16
  EXTENT MANAGEMENT LOCAL
  DATAFILE '/opt/oracle/oradata/MYDB/system01.dbf' SIZE 700M REUSE AUTOEXTEND ON NEXT 10M MAXSIZE
UNLIMITED
  SYSAUX DATAFILE '/opt/oracle/oradata/MYDB/sysaux01.dbf' SIZE 600M REUSE AUTOEXTEND ON NEXT 10M
MAXSIZE UNLIMITED
  DEFAULT TABLESPACE users
  DATAFILE '/opt/oracle/oradata/MYDB/users01.dbf' SIZE 500M REUSE AUTOEXTEND ON NEXT 10M
MAXSIZE UNLIMITED
  DEFAULT TEMPORARY TABLESPACE temp
  TEMPFILE '/opt/oracle/oradata/MYDB/temp01.dbf' SIZE 200M REUSE AUTOEXTEND ON NEXT 10M
MAXSIZE UNLIMITED
  UNDO TABLESPACE undotbs1
  DATAFILE '/opt/oracle/oradata/MYDB/undotbs01.dbf' SIZE 200M REUSE AUTOEXTEND ON NEXT 10M
MAXSIZE UNLIMITED;
|

```


After successfully installed, don't forget these:

1. Run post database creation scripts:

```
@?/rdbms/admin/catalog.sql
```

```
@?/rdbms/admin/catproc.sql
```

2. Connect as SYSTEM and run:

```
CONNECT system/<temporary_password_if_any>
```

```
@?/sqlplus/admin/pupbld.sql
```

3. Create SPFILE (if desired):

```
CREATE PFILE='/u01/app/oracle/product/19.0.0/dbhome_1/dbs/initmyDB.ora'  
FROM SPFILE;
```

Listing all databases created so far

In oracle user type: ls /u01/app/oracle/oradata/

2.1.2.3. Database design templates

A **database design template** in DBCA is a pre-defined or custom XML-based file that stores database creation options.

Templates simplify database creation by saving common configuration choices.

2.1.2.3.1. Types of templates in DBCA

DBCA offers two main template types:

1. **Seed templates**

- Come pre-shipped with Oracle (e.g., General Purpose, Data Warehouse).
- Contain both structure + pre-created seed data.

- Faster to create a new database.

2. Non-Seed Templates

- Contain only **structural metadata** (parameters, options, tablespaces).
- No pre-created datafiles or seed database.
- Database creation takes longer (DBCA must create fresh datafiles).

2.1.2.3.2. Operations you can do with templates

Using DBCA, you can:

- **Use a template** to quickly create a database.
- **Create a custom template** from an existing database.
 - You can capture structure only (metadata) or structure + datafiles.
- **Manage templates:**
 - Delete templates you no longer need.
 - Export/import templates for reuse on other systems.

2.1.2.3.3. Benefits of using templates

- **Standardization:** ensures all databases are created with consistent settings.
- **Time-saving:** avoids repeatedly entering configuration choices.
- **Portability:** templates can be reused across environments (dev, test, prod).
- **Flexibility:** can include only metadata (parameters, options) or both metadata + prebuilt datafiles.

2.1.2.3.4. Managing templates in DBCA (Steps)

a) Creating a database from a template

1. Launch DBCA:
dbca
2. Choose **Create Database**.

3. Select a template (e.g., *General Purpose*, *Data Warehouse*, or custom).
4. Configure options (name, storage, character set, memory).
5. DBCA builds the new database using the chosen template.

b) Creating a Custom Template

1. Launch DBCA.
2. Choose **Manage Templates**.
3. Select **Create a database template**.
4. Choose:
 - *From an existing database (structure only)*, or
 - *From an existing database (structure + datafiles)*.
5. Save the template with a name (e.g., MyCompany_Prod_Template).

c) Using Command-Line mode

DBCA also supports silent/command-line mode for template management, e.g.:

```
dbca -silent -createTemplateFromDB -sourceDB ORCLCDB -templateName MyTemplate  
-maintainFileLocations true
```

2.1.2.4. Configuring an oracle database

2.1.2.4.1. Configuring the Listener

- **Purpose:** The listener enables client applications to connect to the Oracle database over the network.
- **Key Configuration File:** listener.ora (located in \$ORACLE_HOME/network/admin).

Steps to Configure a Listener:

Open listener.ora and define:

```
LISTENER =  
(DESCRIPTION_LIST =  
  (DESCRIPTION =  
    (ADDRESS = (PROTOCOL = TCP)(HOST = your_hostname)(PORT = 1521))  
  )  
)
```

- Start the listener:

lsnrctl start

- Check listener status:

lsnrctl status

- **Optional Commands:**

- Stop: lsnrctl stop
- Reload configuration: lsnrctl reload

2.1.2.4.2. Specifying initialization parameters

- **Initialization parameters** control database behavior, memory, processes, and file locations.
- **Types of Initialization Files:**
 - **PFILE (Parameter File):** text-based (initDBNAME.ora).
 - **SPFILE (Server Parameter File):** binary, can be modified dynamically.
- **Common Parameters:**
 - DB_NAME – database name
 - CONTROL_FILES – control file locations
 - DB_BLOCK_SIZE – database block size
 - SGA_TARGET / PGA_AGGREGATE_TARGET – memory allocation

- UNDO_TABLESPACE – default undo tablespace

2.1.2.4.3. Managing initialization parameters using a server parameter file (SPFILE)

- **SPFILE Advantages:** Can be modified while the database is running; persistent across startups.

Creating SPFILE from PFILE:

```
CREATE SPFILE FROM
PFILE='/u01/app/oracle/product/19.0.0/dbhome_1/dbs/initDBNAME.ora';
```

Modifying Parameters:

Dynamically:

```
ALTER SYSTEM SET parameter_name=value SCOPE=SPFILE;
```

- (SCOPE=SPFILE or SCOPE=BOTH for immediate effect)

Viewing Parameters:

```
SHOW PARAMETER parameter_name;
```

2.1.2.4.4. Cloning a Database with CloneDB

- **Purpose:** Create a new database as a copy of an existing one, for testing, development, or backup purposes.
- **Steps:**
 - Ensure source database is consistent (shutdown or use RMAN hot backup).
 - Use **CloneDB** feature in DBCA (GUI) or command-line:

```
dbca -silent -cloneDatabase -gdbname NEWDB -sid NEWDB \
```

```
-sourceDB ORIGDB -responseFile NO_VALUE
```

- Specify storage locations, initialization parameters, and listener configuration for the clone.

- **Notes:** Clone can include:
 - Full copy of datafiles
 - Only structural metadata

2.1.2.4.5. Dropping a Database

- **Purpose:** Remove a database and all its associated files (datafiles, control files, redo logs).
- **Steps:**

Connect as SYSDBA:

```
sqlplus / as sysdba
```

1. Shutdown the database:

```
SHUTDOWN IMMEDIATE;
```

2. Start instance in **NOMOUNT** mode (optional for DROP DATABASE):

```
STARTUP NOMOUNT;
```

3. Drop the database:

```
DROP DATABASE;
```

Exercise 2.1

You are a new DBA tasked with setting up a fresh Oracle database environment on a server so that developers can start using it for their applications.

Tasks:

1. Prepare the system environment:
 - Ensure the server meets Oracle requirements (disk space, memory, OS version).
 - Create required OS users and groups for Oracle installation (oracle, dba).

Set environment variables:

```
export ORACLE_HOME=/u01/app/oracle/product/19.0.0/dbhome_1
```

```
export ORACLE_SID=orcl
export PATH=$ORACLE_HOME/bin:$PATH
```

2. Install Oracle software:

- Run the Oracle installer (GUI or silent mode).
- Choose “Create and configure a database” option (or install software only).
- Configure default options (memory allocation, character set, listener).

3. Create a new database (if not done during installation):

Use DBCA (Database Configuration Assistant) or SQL*Plus:

```
dbca
```

- Provide database name, SID, and storage locations for data files.

4. Verify installation and environment setup:

Check the Oracle version:

```
sqlplus / as sysdba
SELECT * FROM v$version;
```

Verify listener is running:

```
lsnrctl status
```

5. Configure basic connectivity:

- Create a TNS entry in tnsnames.ora.

Test connection from a client machine using:

```
sqlplus user/password@DB_ALIAS
```

Deliverables / Observations:

- Screenshot or log of Oracle installation completion.
- Screenshot of v\$version output.
- Verification that listener is running (lsnrctl status).

- Short note on environment variables and their purpose (ORACLE_HOME, ORACLE_SID, PATH).

Learning Outcome 2.2: Manage Oracle instance

2.2.1. Starting Up an Oracle Database

When an Oracle database is started, it passes through three possible phases: **NOMOUNT**, **MOUNT**, and **OPEN**. The DBA can stop at any stage depending on the required operation.

2.2.1.1. Startup NOMOUNT

- At this stage, the Oracle **instance** is created.
- Memory structures (SGA) and background processes are initialized.
- Control files and datafiles are not yet accessed.
- Typical uses: creating a new database, or recreating control files.

Example:

```
STARTUP NOMOUNT;
```

2.2.1.2 Startup MOUNT

- The instance is started and control files are read.
- The database is mounted but not opened, meaning datafiles and redo logs remain closed.
- Useful for performing operations such as database renaming, enabling archive log mode, or certain recovery activities.

Example:

```
STARTUP MOUNT;
```

2.2.1.3 Startup OPEN

- The database is fully opened after the instance is mounted.
- Control files, datafiles, and redo log files are all accessed.
- At this point, users may connect and perform normal operations.

Example:

STARTUP;

2.2.1.4 Startup FORCE

- Used when the database instance is unresponsive or cannot be started normally.
- It first aborts the currently running instance, and then restarts it.

Example:

STARTUP FORCE;

2.2.1.5 Startup RESTRICT

- Opens the database in restricted mode.
- Only users with the RESTRICTED SESSION privilege may connect.
- This mode is suitable for administrative tasks such as upgrades or structural maintenance.

Example:

STARTUP RESTRICT;

2.2.1.6 Starting Oracle Restart

- Oracle Restart is part of Oracle Grid Infrastructure, ensuring automatic restart of database components (database, ASM instance, listener) after a system reboot or failure.
- It is managed with the srvctl utility.

Example:

srvctl start database -d NEW2DB

srvctl start asm

srvctl start listener

2.2.2. Shutting Down an Oracle Database

The Oracle database can be shut down in several modes. The choice determines how user sessions and transactions are handled.

2.2.2.1 Shutdown NORMAL

- Waits until all connected users disconnect voluntarily.
- Does not allow new connections.
- Cleanest shutdown but may take a long time.

Example:

SHUTDOWN;

2.2.2.2 Shutdown TRANSACTIONAL

- Waits for active transactions to complete.
- After ongoing transactions are finished, all sessions are disconnected and the database is shut down.
- Suitable when transactional consistency is required before stopping.

Example:

SHUTDOWN TRANSACTIONAL;

2.2.2.3 Shutdown IMMEDIATE

- Rolls back active transactions immediately.
- Disconnects all sessions and shuts down the database without waiting.
- Most commonly used shutdown method in administration.

Example:

SHUTDOWN IMMEDIATE;

2.2.2.4 Shutdown ABORT

- Instantly terminates the instance without rolling back transactions.
- Requires instance recovery on the next startup.
- Should only be used in emergencies when the database is not responding.

Example:

```
SHUTDOWN ABORT;
```

2.2.3. Shutting Down the Automatic Storage Management (ASM) Instance

- The ASM instance manages Oracle database storage structures such as disk groups.
- It can be stopped either from within SQL*Plus or via Oracle Restart utilities.

Example (SQL*Plus as SYSASM):

```
sqlplus / as sysasm  
SHUTDOWN IMMEDIATE;
```

Example (Oracle Restart):

```
srvctl stop asm
```

2.2.4. Conceptual Flow

- **Startup sequence:**
NOMOUNT → MOUNT → OPEN
- **Shutdown options:**
NORMAL → TRANSACTIONAL → IMMEDIATE → ABORT

The choice of mode in both startup and shutdown depends on the administrative task and the operational requirements of the system.

Exercise 2.2

You are a database administrator (DBA) for a company. The production Oracle database needs routine maintenance, and you have been asked to start the instance, check its status, perform basic monitoring, and then safely shut it down.

Tasks:

1. Connect as SYSDBA:
 - Open a terminal (Linux) or SQL*Plus (Windows).

Connect to the database using SYSDBA privileges:

```
sqlplus / as sysdba
```

2. Check the instance status:

Query the instance status and version:

```
SELECT instance_name, status, version FROM v$instance;
```

- Explain the meaning of the status (e.g., STARTED, MOUNTED, OPEN).
3. Start the Oracle instance (if not running):

Start the instance in stages:

```
STARTUP NOMOUNT;  
STARTUP MOUNT;  
STARTUP;
```

- Observe what each stage does.

4. Monitor instance parameters and memory:

View basic memory structures:

```
SHOW PARAMETER sga;  
SHOW PARAMETER pga;
```

- Check active sessions:

```
SELECT username, status FROM v$session;
```

5. Perform instance shutdown:

Practice different shutdown methods:

```
SHUTDOWN NORMAL;  
SHUTDOWN IMMEDIATE;  
SHUTDOWN ABORT;
```

- Observe differences in behavior.

6. Optional challenge:

Restart the instance and ensure automatic startup at system boot (if using Oracle on Linux):

```
srvctl add instance -d <DB_NAME> -i <INSTANCE_NAME> -n <NODE_NAME>
```

Deliverables / Observations:

- A screenshot of v\$instance after startup.
- A short explanation of the memory structures (SGA, PGA).
- Notes on which shutdown method is safest for production.

Learning Outcome 2.3: Monitor database start-up and operation for irregularities

2.3.1. Configuring Automatic Restart of an Oracle Database

Automatic restart ensures that critical database components (database instances, listeners, ASM, etc.) are restarted automatically after a failure or a system reboot. This is achieved through **Oracle Restart**, part of Oracle Grid Infrastructure for a standalone server.

2.3.1.1 Configuring Oracle Restart

- Oracle Restart monitors and automatically restarts components such as:

- Oracle Database instances
 - Oracle ASM instances
 - Oracle Listeners
- Configuration is done using the srvctl utility.

Example:

```
srvctl add database -d NEWDB -o /u01/app/oracle/product/19.0.0/dbhome_1  
srvctl add listener -l LISTENER -p 1521
```

2.3.1.2 Starting and Stopping Components Managed by Oracle Restart

- **Start database:**

```
srvctl start database -d NEWDB
```

- **Stop database:**

```
srvctl stop database -d NEWDB
```

- **Start listener:**

```
srvctl start listener -l LISTENER
```

- **Stop listener:**

```
srvctl stop listener -l LISTENER
```

2.3.1.3 SRVCTL Command Reference for Oracle Restart

- **srvctl add database:** Registers a database with Oracle Restart.
- **srvctl start database / srvctl stop database:** Starts or stops the database.
- **srvctl config database:** Displays database configuration details.
- **srvctl status database:** Shows the current state of the database.
- **srvctl remove database:** Removes the database from Oracle Restart.

2.3.1.4 CRSCTL Command Reference

- CRSCTL manages Oracle Clusterware and high availability services. Even on standalone systems, it is used to control Grid Infrastructure services.
- Common commands:
 - **crsctl check has:** Checks if Oracle High Availability Services are running.
 - **crsctl start has:** Starts Oracle High Availability Services.
 - **crsctl stop has:** Stops Oracle High Availability Services.

2.3.2. Managing Processes

2.3.2.1 Configuring Oracle Database for Shared Server

- The **Shared Server architecture** reduces memory use by allowing multiple user sessions to share server processes.
- Suitable for systems with many concurrent users.

Initialization parameter example:

```
ALTER SYSTEM SET SHARED_SERVERS = 5;  
ALTER SYSTEM SET DISPATCHERS = '(PROTOCOL=TCP)(DISPATCHERS=3)';
```

2.3.2.2 Configuring Database Resident Connection Pooling (DRCP)

- DRCP provides a pool of database server processes shared across multiple client applications.

- Useful in environments with many middle-tier connections (e.g., web apps).

Example:

```
EXEC DBMS_CONNECTION_POOL.START_POOL;
```

2.3 Terminating Sessions

- Database administrators may need to terminate sessions that are idle, blocking, or misbehaving.

Example:

```
ALTER SYSTEM KILL SESSION 'sid,serial#';
```

- To find session IDs:

```
SELECT sid, serial#, username, status FROM v$session;
```

2.3.3. Managing Memory

2.3.3.1 Using Automatic Memory Management (AMM)

- AMM dynamically manages the SGA and PGA based on workload.
- Controlled by the MEMORY_TARGET parameter.

Example:

```
ALTER SYSTEM SET MEMORY_TARGET = 2G SCOPE=SPFILE;  
ALTER SYSTEM SET MEMORY_MAX_TARGET = 3G SCOPE=SPFILE;
```


2.3.3.2 Configuring Memory Manually

- If AMM is not used, DBAs can configure memory manually by setting individual SGA and PGA parameters.

Examples:

```
ALTER SYSTEM SET SGA_TARGET = 1G SCOPE=SPFILE;  
ALTER SYSTEM SET PGA_AGGREGATE_TARGET = 500M SCOPE=SPFILE;
```

2.3.3.3 Configuring Database Smart Flash Cache

- Extends the database buffer cache by using flash storage.
- Especially useful on systems with SSDs.

Example:

```
ALTER SYSTEM SET DB_FLASH_CACHE_FILE = '/u02/flash/flashcache01'  
SCOPE=SPFILE;  
ALTER SYSTEM SET DB_FLASH_CACHE_SIZE = 10G SCOPE=SPFILE;
```

2.3.4. Monitoring Database Operations

2.3.4.1 Monitoring Errors and Alerts

- Oracle records errors and alerts in the **Automatic Diagnostic Repository (ADR)**.
- Tools:

Data Dictionary Views:

```
SELECT * FROM v$alert_log;  
SELECT * FROM dba_outstanding_alerts;
```

- **ADR Command Interpreter (ADRCI):**

Adrci

> show alert -tail -f

- Alerts include critical issues like ORA errors, space warnings, or instance problems.

2.3.5. Conceptual Overview

- **Oracle Restart:** Ensures high availability of instances, listeners, and ASM.
- **Process Management:** Enables efficient resource use (Shared Server, DRCP) and allows session control.
- **Memory Management:** Automatic or manual tuning ensures optimal performance.
- **Monitoring:** Continuous observation of errors and alerts is essential for proactive administration.

Exercise 2.3

You are the database administrator of **NEW2DB**. The database is hosted on Oracle Linux. Carry out the following tasks to monitor startup and detect irregularities:

A. Startup Modes

Start the database in **NOMOUNT** mode and query the alert log to check for errors.

STARTUP NOMOUNT;

1. What background processes do you see running at this stage?

Mount the database and list the control files in use.

STARTUP MOUNT;
SELECT name FROM v\$controlfile;

2. Open the database fully and confirm the status.

```
STARTUP OPEN;
```

```
SELECT status FROM v$instance;
```

B. Monitoring Errors and Alerts

Use the following query to check for recent errors in the alert log:

```
SELECT message_text  
FROM v$diag_alert_ext  
WHERE originating_timestamp > SYSDATE - 1;
```

1. Display active alert notifications in the database:

```
SELECT * FROM dba_outstanding_alerts;
```

2. Generate a simulated alert by forcing a log switch:

```
ALTER SYSTEM SWITCH LOGFILE;
```

3. Review the alert log to confirm the entry.

C. Detecting Irregularities in Operation

Query instance uptime:

```
SELECT startup_time FROM v$instance;
```

1. Check sessions that are waiting due to resource issues:

```
SELECT sid, event, wait_class FROM v$session_wait WHERE wait_class <> 'Idle';
```

2. View invalid database objects that may indicate irregularities:

```
SELECT owner, object_name, object_type FROM dba_objects  
WHERE status='INVALID';
```

D. Reflection Questions

1. What differences did you observe between **NOMOUNT**, **MOUNT**, and **OPEN** stages?
2. Why is checking the **alert log** important after every startup?

3. How can monitoring **invalid objects** and **session waits** help identify operational irregularities?

Learning Outcome 2.4: Manage Oracle storage Structure

2.4.1. Viewing Database Storage Structure Information

The physical storage structure of an Oracle database consists of **control files, online redo logs, archived redo logs, data files, and tablespaces**. Oracle provides data dictionary views to query their metadata.

2.4.1.1 Viewing Control File Information

- Control files maintain critical information about database structure.
- Query:

```
SELECT name FROM v$controlfile;
```

2.4.1.2 Viewing Online Redo Log File Information

- Online redo logs record all changes made to the database.
- Query:

```
SELECT group#, member, status FROM v$logfile;  
SELECT group#, bytes/1024/1024 AS size_MB, archived, status FROM v$log;
```

2.4.1.3 Viewing Archived Redo Log File Information

- Archived redo logs are historical copies of redo logs, used in recovery.
- Query:

```
SELECT name, sequence#, applied FROM v$archived_log;
```

2.4.1.4 Viewing Data File Information

- Data files store actual user and system data.
- Query:

```
SELECT file_id, file_name, tablespace_name, bytes/1024/1024 AS size_MB  
FROM dba_data_files;
```

2.4.1.5 Viewing Tablespace Information

- Tablespaces are logical storage units grouping data files.
- Query:

```
SELECT tablespace_name, file_name, bytes/1024/1024 AS size_MB  
FROM dba_data_files  
ORDER BY tablespace_name;
```

2.4.2. Managing Tablespaces

Tablespaces provide a logical layer between physical data files and database objects.

2.4.2.1 Creating a Tablespace

```
CREATE TABLESPACE sales  
DATAFILE '/u01/app/oracle/oradata/NEW2DB/sales01.dbf' SIZE 100M AUTOEXTEND ON;
```

2.4.2.2 Modifying a Tablespace

- Add data file:

```
ALTER DATABASE ADD DATAFILE '/u01/app/oracle/oradata/NEW2DB/sales02.dbf' SIZE  
50M;
```

- **Resize a data file:**

```
ALTER DATABASE DATAFILE '/u01/app/oracle/oradata/NEW2DB/sales01.dbf' RESIZE  
200M;
```

2.4.2.3 Dropping a Tablespace

Remove a tablespace and its contents:

```
DROP TABLESPACE sales INCLUDING CONTENTS AND DATAFILES;
```

2.4.3. Managing the Online Redo Log

Redo logs are essential for recovery. Administrators can manage their structure for performance and fault tolerance.

2.4.3.1 Creating Online Redo Log Groups and Members

- **Add a new redo log group:**

```
ALTER DATABASE ADD LOGFILE GROUP 4  
('/u01/app/oracle/oradata/NEW2DB/redo04a.log',  
'/u01/app/oracle/oradata/NEW2DB/redo04b.log') SIZE 100M;
```

- **Add a new member to an existing group:**

```
ALTER DATABASE ADD LOGFILE MEMBER  
'/u01/app/oracle/oradata/NEW2DB/redo02b.log' TO GROUP 2;
```

2.4.3.2 Relocating and Renaming Online Redo Log Members

- **Take group offline, move file at OS level, then update database:**

```
ALTER DATABASE RENAME FILE
```

```
'/u01/app/oracle/oradata/NEW2DB/redo02a.log'  
TO '/u02/app/oracle/oradata/NEW2DB/redo02a.log';
```

2.4.3.3 Dropping Online Redo Log Groups and Members

- **Drop a member:**

```
ALTER DATABASE DROP LOGFILE MEMBER  
'/u01/app/oracle/oradata/NEW2DB/redo02b.log';
```

- **Drop a group:**

```
ALTER DATABASE DROP LOGFILE GROUP 4;
```

2.4.3.4 Forcing Log Switches

- Log switches move writing from one redo log group to the next.

```
ALTER SYSTEM SWITCH LOGFILE;
```

2.4.3.5 Clearing an Online Redo Log File

- If a redo log file is corrupted and not needed for recovery, it can be cleared.

```
ALTER DATABASE CLEAR LOGFILE GROUP 3;
```

Exercise 2.4

You are the DBA of the database **NEW2DB**. Perform the following tasks to practice managing storage structures:

A. Viewing Storage

1. Display all control files.

2. List redo log groups and members.
3. Show all datafiles and their tablespaces.
4. Check if the database is running in ARCHIVELOG or NOARCHIVELOG mode.

B. Managing Tablespaces

1. Create a new tablespace students_ts with an initial datafile size of 50 MB.
2. Add another datafile of 30 MB to the same tablespace.
3. Drop the students_ts tablespace with all its contents.

C. Managing Online Redo Logs

1. Add a new redo log group with two members.
2. Force a log switch to archive the current log.
3. Clear a redo log group if it is marked as INVALID or CORRUPT.

D. Managing Archived Redo Logs

1. Enable ARCHIVELOG mode if not enabled.
2. Archive the current redo log manually.
3. List all archived redo logs generated so far.

E. Reflection

1. Why is it important to multiplex **control files** and **redo log files**?
2. What is the impact of running in **NOARCHIVELOG** mode for a production database?
3. When dropping a tablespace, why is it recommended to include the INCLUDING CONTENTS AND DATAFILES clause?

Learning Outcome 2.5: Manage Schema Objects

2.5.1. Introduction to Schema

A **schema** is a logical collection of database objects owned by a single user.

Schema objects include tables, indexes, views, sequences, synonyms, procedures, and other structures.

2.5.1.1 Naming Schema Objects

- Each schema object must have a unique name within the schema.
- Names must begin with a letter, can include letters, numbers, and underscores, and are case-insensitive (unless quoted).

Fully qualified naming uses the format:

schema_name.object_name

Example:

```
SELECT * FROM hr.employees;
```

2.5.2. Managing Tables

Tables are the fundamental schema objects for storing data in rows and columns.

2.5.2.1 Viewing Tables

```
SELECT table_name FROM user_tables;
```

2.5.2.2 Viewing Table Data

```
SELECT * FROM employees;
```

2.5.2.3 Creating a Table

```
CREATE TABLE employees (  
    emp_id    NUMBER PRIMARY KEY,  
    first_name VARCHAR2(50),  
    last_name  VARCHAR2(50),  
    hire_date  DATE,  
    salary     NUMBER(10,2)  
);
```

2.5.2.4 Modifying Table Attributes

- **Add a column:**

```
ALTER TABLE employees ADD (department_id NUMBER);
```

- **Modify a column:**

```
ALTER TABLE employees MODIFY (salary NUMBER(12,2));
```

- **Drop a column:**

```
ALTER TABLE employees DROP COLUMN department_id;
```

2.5.2.5 Loading Data into a Table

- **Insert single row:**

```
INSERT INTO employees VALUES (101, 'John', 'Doe', SYSDATE, 5000);
```

- **Insert multiple rows using INSERT ALL:**

```
INSERT ALL  
  INTO employees VALUES (102, 'Jane', 'Smith', SYSDATE, 6000)  
  INTO employees VALUES (103, 'Mike', 'Brown', SYSDATE, 7000)  
SELECT * FROM dual;
```

2.5.2.6 Dropping a Table

```
DROP TABLE employees PURGE;
```

2.5.3. Managing Indexes

Indexes improve the performance of queries by allowing faster data retrieval.

2.5.3.1 Viewing Indexes

```
SELECT index_name, table_name FROM user_indexes;
```

2.5.3.2 Creating an Index

```
CREATE INDEX emp_lastname_idx ON employees(last_name);
```

2.5.3.3 Deleting an Index

```
DROP INDEX emp_lastname_idx;
```

2.5.4. Managing Views

Views are logical representations of queries, providing an abstraction layer over base tables.

2.5.4.1 Displaying Views

```
SELECT view_name FROM user_views;
```

2.5.4.2 Creating a View

```
CREATE VIEW emp_salaries AS  
SELECT emp_id, first_name, last_name, salary  
FROM employees  
WHERE salary > 5000;
```

2.5.4.3 Deleting a View

```
DROP VIEW emp_salaries;
```

2.5.5. Working with Other Schema Objects

2.5.5.1 Sequences

- Used to generate unique numeric values, often for primary keys.

```
CREATE SEQUENCE emp_seq  
START WITH 1  
INCREMENT BY 1  
NOCACHE NOCYCLE;
```

```
INSERT INTO employees (emp_id, first_name, last_name, hire_date, salary)  
VALUES (emp_seq.NEXTVAL, 'Alice', 'Green', SYSDATE, 8000);
```

2.5.5.2 Synonyms

- Provide alternative names for schema objects.
- Useful to simplify object access or hide ownership details.

```
-- Private synonym
CREATE SYNONYM emp FOR hr.employees;

-- Public synonym
CREATE PUBLIC SYNONYM all_emps FOR hr.employees;
```

Exercise 2.5

Part A – Conceptual Questions

1. Differentiate between:
 - **Tables vs Views**
 - **Indexes vs Primary Keys**
 - **Sequences vs Synonyms**
2. Why might a DBA use **views** instead of granting direct access to tables?
3. How do **indexes** improve performance, and in which cases might they **degrade** performance?
4. Explain a scenario where a **synonym** would make SQL development easier.

Part B – Hands-On Tasks

You are the DBA of a schema COLLEGE_DB. Perform the following tasks:

1. Managing Tables

- Create a table:

```
CREATE TABLE students (  
    student_id NUMBER PRIMARY KEY,  
    name VARCHAR2(50),  
    dob DATE,  
    department VARCHAR2(30)  
);
```

- Modify the table to add a column:

```
ALTER TABLE students ADD email VARCHAR2(100);
```

- Insert sample records:

```
INSERT INTO students VALUES (1, 'Alice', DATE '2002-05-10', 'Computer Science',  
'alice@example.com');
```

```
INSERT INTO students VALUES (2, 'Bob', DATE '2001-12-21', 'Mechanical',  
'bob@example.com');
```

- Drop the table (when no longer needed):

```
DROP TABLE students;
```

2. Managing Indexes

- Create an index on the department column:

```
CREATE INDEX idx_department ON students(department);
```

- View indexes in user schema:

```
SELECT index_name, table_name FROM user_indexes;
```

- Drop an index:

```
DROP INDEX idx_department;
```

3. Managing Views

- Create a view showing only names and departments:

```
CREATE VIEW student_summary AS  
SELECT name, department FROM students;
```

- Query the view:

```
SELECT * FROM student_summary;
```

- Drop the view:

```
DROP VIEW student_summary;
```

4. Managing Sequences

- Create a sequence to auto-generate student IDs:

```
CREATE SEQUENCE student_seq START WITH 100 INCREMENT BY 1;
```

- Use it in insert:

```
INSERT INTO students (student_id, name, dob, department, email)
VALUES (student_seq.NEXTVAL, 'Clara', DATE '2003-04-14', 'Electrical',
'clara@example.com');
```

- Drop the sequence:

```
DROP SEQUENCE student_seq;
```

5. Managing Synonyms

- Create a synonym for the students table:

```
CREATE SYNONYM stu FOR students;
```

- Query using synonym:

```
SELECT * FROM stu;
```

- Drop the synonym:

```
DROP SYNONYM stu;
```

Part C – Reflection

1. What advantages do **views** bring when managing **data security**?
2. If a sequence generates duplicate values accidentally, what database design issue could be the cause?
3. Why should synonyms be carefully managed in large systems?

LEARNING UNIT 3: SECURE ORACLE DATABASE

Learning Outcome 3.1: Administer user account and security

3.1.1. Administering Roles

Roles in Oracle are **named groups of privileges** that simplify privilege management by assigning multiple privileges to a role, then granting the role to users.

3.1.1.1 Describing Roles

- A role is a **collection of system and object privileges**.
- Roles can be **granted to users** or other roles to simplify security administration.
- Query roles available in the database:

```
SELECT role FROM dba_roles;
```

3.1.1.2 Creating a Role

```
CREATE ROLE manager_role;
```

3.1.1.3 Granting Privileges to a Role

```
-- Grant system privilege
```

```
GRANT CREATE TABLE, CREATE VIEW TO manager_role;
```

```
-- Grant object privilege
```

```
GRANT SELECT, INSERT ON hr.employees TO manager_role;
```

3.1.1.4 Revoking Privileges from a Role

```
REVOKE CREATE VIEW FROM manager_role;
```

3.1.1.5 Modifying a Role

- Roles cannot be altered directly, but privileges can be granted/revoked as needed.
- Assign default roles to users:

```
ALTER USER john DEFAULT ROLE manager_role;
```

3.1.2. Administering User Accounts

User accounts represent database users who can connect and perform operations depending on assigned privileges and roles.

3.1.2.1 Predefined User Accounts

- Oracle ships with predefined accounts such as:
 - SYS → DBA account with all privileges.
 - SYSTEM → Administrative account.
- These accounts are used for administration and should be secured.

3.2.2.2 User Privileges and Roles

- Privileges determine what a user can do.
- Roles allow grouping privileges for easier management.
- Example: grant the manager_role to a user.

3.2.2.3 Administrative Accounts and Privileges

- Administrative users may have system privileges like DBA, CREATE USER, ALTER USER.
- Always assign privileges according to the principle of **least privilege**.

3.2.2.4 Creating a User Account

```
CREATE USER john IDENTIFIED BY "SecurePass123"  
DEFAULT TABLESPACE users  
TEMPORARY TABLESPACE temp  
PROFILE default;
```

3.2.2.5 Viewing User Accounts

```
SELECT username, account_status, default_tablespace, profile  
FROM dba_users;
```


3.2.2.6 Modifying User Account

- **Change password:**

```
ALTER USER john IDENTIFIED BY "NewSecurePass123";
```

- **Lock or unlock account:**

```
ALTER USER john ACCOUNT LOCK;  
ALTER USER john ACCOUNT UNLOCK;
```

- **Change default tablespace:**

```
ALTER USER john DEFAULT TABLESPACE sales;
```

3.2.2.7 Granting Privileges and Roles to User Accounts

-- Grant system privilege

```
GRANT CREATE SESSION, CREATE TABLE TO john;
```

-- Grant role

```
GRANT manager_role TO john;
```

3.1.3. Managing Profiles

Profiles allow DBAs to **control resource usage** and enforce **security policies** such as password complexity, failed login attempts, and idle time limits.

3.1.3.1 Describing a Profile

- Profiles define limits for:
 - Password life, reuse, and complexity
 - Session idle time
 - CPU and connect time
- Query profiles:

```
SELECT * FROM dba_profiles;
```

3.1.3.2 Creating a Profile

```
CREATE PROFILE secure_profile LIMIT  
SESSIONS_PER_USER 5  
CPU_PER_SESSION 10000  
FAILED_LOGIN_ATTEMPTS 5  
PASSWORD_LIFE_TIME 90  
PASSWORD_REUSE_TIME 180  
PASSWORD_VERIFY_FUNCTION verify_function;
```

3.1.3.3 Dropping a Profile

```
DROP PROFILE secure_profile CASCADE;
```

- CASCADE option ensures users assigned to the profile are not left without a profile.

Exercise 3.1

Part A – Conceptual Questions

1. Differentiate between:
 - **System privileges** vs **Object privileges**
 - **Roles** vs **Profiles**
 - **Authentication** vs **Authorization**
2. Why is it recommended to grant privileges through **roles** instead of directly to users?
3. Explain how **password profiles** help enforce database security policies.
4. What are **locked accounts** and why might a DBA lock them?

Part B – Hands-On Tasks

You are the DBA of a database instance SECUREDB. Perform the following tasks:

1. Create User Accounts

- Create a new user:

```
CREATE USER student1 IDENTIFIED BY StrongPass#123  
DEFAULT TABLESPACE users  
TEMPORARY TABLESPACE temp  
QUOTA 50M ON users;
```

- Grant the user privilege to connect:

```
GRANT CREATE SESSION TO student1;
```

2. Grant System and Object Privileges

- Grant system privilege:

```
GRANT CREATE TABLE TO student1;
```

- Create a sample schema object as SYS:

```
CREATE TABLE hr.employees (id NUMBER, name VARCHAR2(50));
```

- Grant object privilege:

```
GRANT SELECT, INSERT ON hr.employees TO student1;
```

3. Create and Manage Roles

- Create a role:

```
CREATE ROLE app_user;
```

- Grant privileges to the role:

```
GRANT CREATE SESSION, CREATE TABLE TO app_user;
```

- Assign the role to a user:

```
GRANT app_user TO student1;
```

4. Apply Security Profiles (Password & Resource Limits)

- Create a profile with password policy:

```
CREATE PROFILE secure_profile LIMIT  
FAILED_LOGIN_ATTEMPTS 3  
PASSWORD_LIFE_TIME 30  
PASSWORD_REUSE_TIME 180  
PASSWORD_REUSE_MAX 5  
PASSWORD_LOCK_TIME 1/24;
```

- Assign profile to user:

```
ALTER USER student1 PROFILE secure_profile;
```

5. Account Locking and Unlocking

- Lock the account:

```
ALTER USER student1 ACCOUNT LOCK;
```

- Unlock the account:

```
ALTER USER student1 ACCOUNT UNLOCK;
```

6. Audit User Activities (Optional – if Unified Auditing is enabled)

- Enable auditing for a user:

```
AUDIT CREATE TABLE BY student1;
```

- Query audit trail:

```
SELECT * FROM UNIFIED_AUDIT_TRAIL WHERE USERNAME = 'STUDENT1';
```

Part C – Reflection

- Why is it a security risk to assign the **DBA role** directly to users?
- What is the difference between **revoking privileges** and **dropping a user**?
- How do **profiles** help enforce compliance with organizational security standards?

Learning Outcome 3.2: Perform Database backup and recovery

3.2.1. Overview of Backup and Recovery

3.2.1.1 Introduction to Backup and Recovery

- Backup and recovery ensure data **availability, integrity, and protection against failure**.
- Oracle provides two approaches:
 - **RMAN (Recovery Manager):** Oracle's recommended and automated backup and recovery tool.
 - **User-managed methods:** Manual backup and recovery using SQL*Plus and OS commands.
- Backup types:
 - **Physical backup:** Copying datafiles, control files, and redo logs.
 - **Logical backup:** Using Data Pump to export/import tables, schemas, or databases.

3.2.2. Starting and Configuring RMAN and Flashback Database

3.2.2.1 RMAN Architecture

- **RMAN Client:** User interface to issue backup and recovery commands.
- **Target Database:** The database being backed up or restored.

- **RMAN Repository:** Metadata stored in the target database control file or a recovery catalog schema.

3.2.2.2 Starting and Interacting with RMAN

`rman target /`

or

`rman target sys/password@NEW2DB`

3.2.2.3 Configuring the RMAN Environment

-- Configure default backup type as compressed backupset

`CONFIGURE DEVICE TYPE DISK BACKUP TYPE TO COMPRESSED BACKUPSET;`

-- Configure backup retention policy

`CONFIGURE RETENTION POLICY TO RECOVERY WINDOW OF 7 DAYS;`

-- Configure control file autobackup

`CONFIGURE CONTROLFILE AUTOBACKUP ON;`

3.2.2.4 Using Flashback Database and Restore Points

- **Flashback Database** allows fast point-in-time recovery without traditional restore.
- **Restore points** are markers for flashback operations.

-- Create restore point

`CREATE RESTORE POINT before_patch GUARANTEE FLASHBACK DATABASE;`

-- Flashback to restore point

`FLASHBACK DATABASE TO RESTORE POINT before_patch;`

3.2.3. Backing Up and Archiving Data

3.2.3.1 RMAN Backup Concepts

- **Full backup:** Copies all datafiles.
- **Incremental backup:** Backs up only changed blocks since last backup.
- **Archived redo logs:** Must be backed up for complete recovery.

3.2.3.2 Backing Up the Database

BACKUP DATABASE PLUS ARCHIVELOG;

3.2.3.3 Advanced Database Backup Features

- **Block change tracking** improves incremental backup speed.

ALTER DATABASE ENABLE BLOCK CHANGE TRACKING;

- **Compressed backups** reduce storage use.
- **Encrypted backups** secure sensitive data.

3.2.4. Managing RMAN Backups

3.2.4.1 Reporting on RMAN Operations

REPORT NEED BACKUP;
LIST BACKUP SUMMARY;

3.2.4.2 Maintaining RMAN Backups and Repository Records

DELETE NOPROMPT OBSOLETE;
CROSSCHECK BACKUP;
DELETE EXPIRED BACKUP;

3.2.4.3 Managing a Recovery Catalog

- A **recovery catalog** is a separate schema storing RMAN metadata.

-- Connect to catalog

```
rman catalog rman_user/rman_pass@catdb;
```

-- Register database

```
REGISTER DATABASE;
```

3.2.5. Transferring Data with RMAN

3.2.5.1 Duplicating Databases

- Create a duplicate (clone) database for testing or standby purposes.

```
DUPLICATE TARGET DATABASE TO testdb FROM ACTIVE DATABASE;
```

3.2.5.2 Creating Transportable Tablespace Sets

- Allows moving a self-contained set of tablespaces across databases.

```
EXECUTE DBMS_TTS.TRANSPORT_SET_CHECK('sales', TRUE);
```

3.2.5.3 Transporting Data Across Platforms

- RMAN can convert datafiles for use across different OS platforms.

```
CONVERT DATABASE NEW DATABASE 'newdb'  
  TRANSPORT SCRIPT '/u01/scripts/transport.sql'  
  FORMAT '/u02/backup/%U';
```

3.2.6. Performing User-Managed Backup and Recovery

3.2.6.1 Making User-Managed Database Backups

- Shut down the database cleanly, then copy files using OS commands.


```
SHUTDOWN IMMEDIATE;  
cp /u01/app/oracle/oradata/NEW2DB/*.dbf /backup/  
cp /u01/app/oracle/oradata/NEW2DB/*.ctl /backup/  
cp /u01/app/oracle/oradata/NEW2DB/*.log /backup/
```

3.2.6.2 Performing User-Managed Database Flashback and Recovery

- Restore datafiles manually and apply archived logs.

```
-- Mount the database
```

```
STARTUP MOUNT;
```

```
-- Restore datafiles from OS backup
```

```
-- Then recover
```

```
RECOVER DATABASE;
```

```
-- Open database
```

```
ALTER DATABASE OPEN RESETLOGS;
```

Exercise 3.2

Part A – Conceptual Questions

1. Differentiate between:
 - **User-managed backup** and **RMAN-managed backup**
 - **Full backup** and **Incremental backup**
 - **Consistent backup** and **Inconsistent backup**
2. What are the main advantages of using RMAN over user-managed backups?
3. Explain the difference between **restore** and **recovery** in Oracle.
4. Why is archiving redo logs critical for point-in-time recovery?

Part B – Hands-On Tasks

You are given a database instance BACKUPDB configured in **ARCHIVELOG mode**. Complete the following tasks.

1. User-Managed Backup

- Put the database in backup mode and copy datafiles at the OS level:

```
ALTER DATABASE BEGIN BACKUP;  
!cp /u01/app/oracle/oradata/BACKUPDB/system01.dbf /u02/backup/system01.bak  
ALTER DATABASE END BACKUP;
```

- Back up the control file:

```
ALTER DATABASE BACKUP CONTROLFILE TO '/u02/backup/control01.bak';
```

2. RMAN Backup

- Connect to RMAN:

```
rman target /
```

- Perform a full database backup:

```
BACKUP DATABASE PLUS ARCHIVELOG;
```

- Perform an incremental backup (level 1):

```
BACKUP INCREMENTAL LEVEL 1 DATABASE;
```

3. Simulating Data Loss

- Drop a non-critical tablespace:

```
DROP TABLESPACE test INCLUDING CONTENTS AND DATAFILES;
```

- Verify its loss:

```
SELECT tablespace_name FROM dba_tablespaces;
```

4. Performing Recovery with RMAN

- Restore the missing tablespace:

```
RESTORE TABLESPACE test;  
RECOVER TABLESPACE test;
```

- Open the database:

```
ALTER DATABASE OPEN;
```

5. Flashback Recovery

- Create a restore point:

```
CREATE RESTORE POINT before_changes GUARANTEE FLASHBACK DATABASE;
```

- Make changes (e.g., delete rows from a table).
- Perform flashback to restore point:

```
FLASHBACK DATABASE TO RESTORE POINT before_changes;
```

- Drop the restore point after verification:

```
DROP RESTORE POINT before_changes;
```

6. Reporting on RMAN Backups

- List backups recorded in the RMAN repository:

LIST BACKUP;

- Crosscheck and delete obsolete backups:

**CROSSCHECK BACKUP;
DELETE OBSOLETE;**

Part C – Reflection

- In which cases would you prefer **flashback recovery** over **traditional recovery**?
- How does RMAN ensure database consistency during incremental backups?
- Why should a DBA always test backups with a **restore and recovery simulation**?

Learning Outcome 3.3: Manage Data encryption

3.3.1. Data Encryption Introduction

3.3.1.1 Preventing Database Bypass with Encryption

Even if an attacker gains access to the underlying storage, **encrypted datafiles remain unreadable** without keys.

Encryption prevents:

- Reading data from copied disks.
- Unauthorized OS-level access to datafiles.
- Database bypass through direct file inspection.

3.3.1.2 Oracle Advanced Security Transparent Data Encryption (TDE)

- TDE provides **transparent encryption of sensitive data at rest**.
- Applications require **no changes** — encryption and decryption happen automatically at the storage level.

3.3.1.3 Protecting Sensitive Data Using TDE Column Encryption

- Encrypts only specific columns (e.g., credit card numbers, national IDs).
- Useful when **only certain data elements require encryption**.

-- Enable wallet for encryption

```
ADMINISTER KEY MANAGEMENT CREATE KEYSTORE  
'/u01/app/oracle/admin/ORCL/wallet' IDENTIFIED BY MyPassword;
```

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN IDENTIFIED BY  
MyPassword;
```

```
ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY IDENTIFIED BY  
MyPassword WITH BACKUP;
```

-- Encrypt column

```
ALTER TABLE customers MODIFY (credit_card ENCRYPT USING 'AES256');
```

3.3.1.4 Protecting Entire Applications Using TDE Tablespace Encryption

- Encrypts the **whole tablespace**, securing all objects (tables, indexes, LOBs) inside it.
- Preferred when an application handles **many sensitive attributes**.

```
CREATE TABLESPACE secure_tbs  
DATAFILE '/u02/oradata/secure01.dbf' SIZE 100M  
ENCRYPTION USING 'AES256'  
DEFAULT STORAGE(ENCRYPT);
```

3.3.1.5 Protecting the Database Using TDE Database Encryption

- Encrypts **all tablespaces** in the database, ensuring total coverage.
- This approach secures **every object** without developers needing to specify encryption explicitly.

3.3.1.6 Performance Characteristics

- Encryption introduces a **small CPU overhead**, depending on:
 - Algorithm (AES128 faster than AES256).

- Workload type (I/O intensive workloads show more impact).
- Oracle optimizes TDE with **hardware acceleration** (AES-NI).

3.3.1.7 Built-In Key Management

- Oracle uses a **software keystore** (wallet) or an **external key store (HSM/KMS)**.
- Keys are automatically rotated, backed up, and managed.

3.3.1.8 Encryption Impact for Common Operational Activities

- **Backups:** RMAN backs up encrypted data transparently.
- **Export/Import:** Data Pump supports encrypted exports.
- **Replication:** Streams and GoldenGate handle encrypted data securely.
- **Indexing:** Encrypted columns remain indexable (except for salt-added cases).

3.3.2. Limiting Sensitive Data Exposure with Data Redaction

3.3.2.1 Oracle Advanced Security Data Redaction

- **Data Redaction** masks sensitive data **at runtime** before it reaches end-users.
- Protects against **data exposure in applications** while keeping underlying data intact.

3.3.2.2 Policies and Transformations

Policies define **when and how** data should be redacted.

Transformations include:

- **Full redaction:** Replace all values (e.g., show XXXX-XXXX-XXXX-1234).
- **Partial redaction:** Hide part of the value.
- **Random redaction:** Replace with random characters/numbers.
- **Null redaction:** Display as NULL.

BEGIN

```
DBMS_REDACT.ADD_POLICY(
  object_schema => 'HR',
  object_name   => 'EMPLOYEES',
  column_name   => 'SALARY',
  policy_name   => 'salary_redact',
  function_type => DBMS_REDACT.FULL,
```

```

expression => 'SYS_CONTEXT("USERENV", "SESSION_USER") <> "HR_ADMIN"
);
END;
/

```

3.3.2.3 Performance Characteristics

- Minimal overhead since redaction occurs at **query result time**.
- Policies are enforced at the **database kernel level**.

3.3.2.4 Security Considerations

- Redaction prevents **unintended exposure** in reports, queries, or application screens.
- It does **not replace encryption**, which secures data at rest.
- Must be combined with **privilege management** and auditing.

3.3.2.5 Easy to Deploy Data Redaction

- Can be applied **without modifying applications**.
- Example: mask SSNs or credit cards in legacy apps.
- Policies can be dynamically enabled/disabled.

3.3.2.6 Comparison to Alternative Approaches

Approach	When to Use	Limitation
Encryption (TDE)	Data at rest security	Does not hide data in SELECT queries
Data Redaction	Data-in-use masking	Does not protect stolen storage
Application-level masking	Custom business rules	Requires code changes
Data anonymization	Data for testing	Irreversible; cannot use for production

Exercise 3.3

Part A – Conceptual Questions

1. Explain the difference between:
 - **Column-level encryption**

- **Tablespace encryption**
 - **Database-wide encryption**
2. What is the role of the **Oracle Wallet/Keystore** in TDE?
 3. Why might a database administrator choose **tablespace encryption** over column encryption?
 4. Identify two operational tasks that are impacted by Transparent Data Encryption (TDE).

Part B – Hands-On Tasks

You are given a database instance SECUREDB. Configure **Transparent Data Encryption (TDE)** as follows:

1. Configure the Keystore (Wallet)

- Create a keystore for encryption keys.
- Open the keystore and set a master key.

```
ADMINISTER KEY MANAGEMENT CREATE KEYSTORE
'/u01/app/oracle/admin/SECUREDB/wallet'
IDENTIFIED BY StrongPassw0rd;
```

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN
IDENTIFIED BY StrongPassw0rd;
```

```
ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY
IDENTIFIED BY StrongPassw0rd WITH BACKUP;
```

2. Column-Level Encryption

- Encrypt the CREDIT_CARD column in the CUSTOMERS table using AES256.

```
ALTER TABLE customers MODIFY (credit_card ENCRYPT USING 'AES256');
```

- Verify the encryption status:


```
SELECT table_name, column_name, encryption_alg  
FROM user_encrypted_columns;
```

3. Tablespace Encryption

- Create an encrypted tablespace for sensitive HR data.

```
CREATE TABLESPACE secure_hr_tbs  
DATAFILE '/u02/oradata/secure_hr01.dbf' SIZE 100M  
ENCRYPTION USING 'AES256'  
DEFAULT STORAGE(ENCRYPT);
```

- Move an existing table EMPLOYEES into this tablespace:

```
ALTER TABLE hr.employees MOVE TABLESPACE secure_hr_tbs;
```

4. Verifying Encrypted Datafiles

- Check encryption status of datafiles:

```
SELECT name, encryptionalg, encrypted  
FROM v$datafile;
```

5. Test Backup with Encryption

- Create an encrypted RMAN backup of the database:

```
rman target /
```

```
RMAN> BACKUP DATABASE ENCRYPTION USING 'AES256' FORMAT  
'/u03/backup/securedb_%U.bkp';
```

Part C – Reflection

- Discuss the **performance impact** of enabling TDE in a high-transaction OLTP system.
- Explain how encryption supports **compliance requirements** (e.g., GDPR, PCI-DSS).

Learning Outcome 3.4: Manage Database auditing

3.4.1. Introduction to Auditing

3.4.1.1 Definition of Auditing

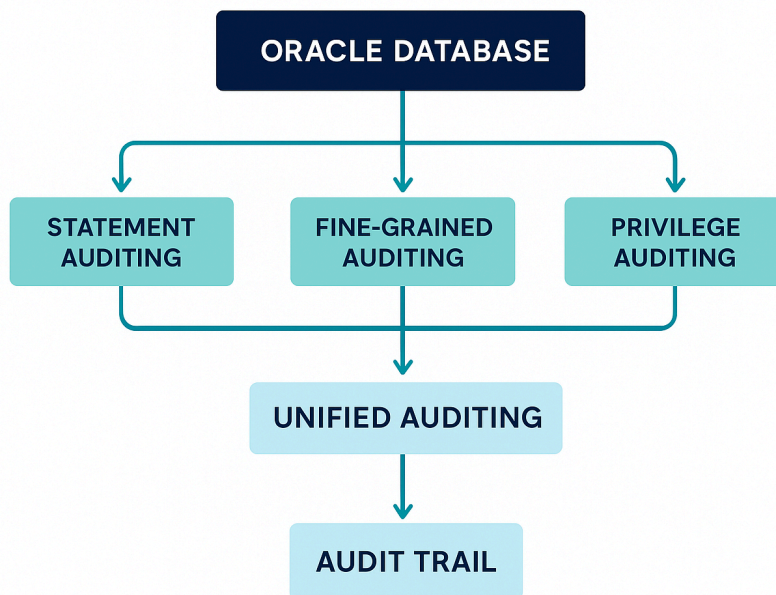
Auditing in Oracle is the process of **recording and analyzing database activity**.

It helps:

- Detect unauthorized access.
- Ensure compliance with regulations.
- Track changes made by users.
- Support forensic analysis after incidents.

3.4.1.2 Definition of Unified Auditing

Unified Auditing (introduced in Oracle 12c) integrates auditing into a **single consolidated trail**, replacing multiple audit trails.



Benefits:

- Centralized storage of audit records.
- Enhanced performance due to optimized architecture.
- Easier management using predefined policies.
- Compatibility with standard and fine-grained auditing.

3.4.2. Configuring Audit Policies

3.4.2.1 Selecting an Auditing Type

- **Standard Auditing** (pre-12c) → Deprecated but still supported.
- **Unified Auditing** → Default in Oracle 12c and later.

Types:

- **Privilege auditing** → Tracks use of system privileges.
- **Object auditing** → Tracks operations on specific objects.
- **Fine-grained auditing (FGA)** → Tracks row-level and column-level activity.

3.4.2.2 Auditing Activities with Unified Audit Policies and the AUDIT Statement

- Create audit policies with **CREATE AUDIT POLICY**.
- Enable auditing with **AUDIT POLICY**.

-- Create policy for user logins

```
CREATE AUDIT POLICY login_audit
ACTIONS LOGON;
```

-- Enable policy

```
AUDIT POLICY login_audit;
```

3.4.2.3 Auditing Activities with Predefined Unified Audit Policies

- Oracle provides **predefined policies**, such as:
 - **ORA_LOGON_FAILURES** → captures failed login attempts.

- ORA_DATABASE_PARAMETER → captures parameter changes.
- ORA_ACCOUNT_MGMT → captures account changes.

AUDIT POLICY ORA_LOGON_FAILURES;

3.4.2.4 Auditing Specific Activities with Fine-Grained Auditing (FGA)

- FGA allows auditing of **specific conditions** on tables and columns.

BEGIN

```
DBMS_FGA.ADD_POLICY(
  object_schema => 'HR',
  object_name   => 'EMPLOYEES',
  policy_name   => 'fga_salary',
  audit_condition => 'SALARY > 10000',
  audit_column  => 'SALARY'
);
```

END;

/

→ This policy records only salary queries above 10,000.

3.4.2.5 Audit Policy Data Dictionary Views

- Key views for monitoring audit policies:
 - AUDIT_UNIFIED_POLICIES → shows defined audit policies.
 - AUDIT_UNIFIED_ENABLED_POLICIES → shows active policies.
 - DBA_AUDIT_MGMT_CONFIG_PARAMS → shows audit trail configurations.

3.4.3. Administering the Audit Trail

3.4.3.1 Managing the Unified Audit Trail

Records are stored in the **UNIFIED_AUDIT_TRAIL** view.

Unified audit trail consolidates data from:

- Standard auditing.
- FGA.
- RMAN auditing.
- Data Pump auditing.

```
SELECT event_timestamp, username, action_name
FROM unified_audit_trail
ORDER BY event_timestamp DESC;
```

3.4.3.2 Archiving the Audit Trail

- Audit data can be **archived** using the **DBMS_AUDIT_MGMT** package.

```
EXEC DBMS_AUDIT_MGMT.INIT_CLEANUP(
    audit_trail_type => DBMS_AUDIT_MGMT.AUDIT_TRAIL_UNIFIED,
    default_cleanup_interval => 24);
```

```
EXEC DBMS_AUDIT_MGMT.CLEAN_AUDIT_TRAIL(
    audit_trail_type => DBMS_AUDIT_MGMT.AUDIT_TRAIL_UNIFIED,
    use_last_arch_timestamp => TRUE);
```

3.4.3.3 Purging Audit Trail Records

- Old audit records may be **purged** to free space.

```
EXEC DBMS_AUDIT_MGMT.SET_LAST_ARCHIVE_TIMESTAMP(
    audit_trail_type => DBMS_AUDIT_MGMT.AUDIT_TRAIL_UNIFIED,
```

```
last_archive_time => SYSTIMESTAMP - 30);
```

```
EXEC DBMS_AUDIT_MGMT.CLEAN_AUDIT_TRAIL(  
  audit_trail_type => DBMS_AUDIT_MGMT.AUDIT_TRAIL_UNIFIED,  
  use_last_arch_timestamp => TRUE);
```

3.4.3.4 Audit Trail Management Data Dictionary Views

- **Key views for audit trail administration:**

- DBA_COMMON_AUDIT_TRAIL → unified view of all audit records.
- UNIFIED_AUDIT_TRAIL → main unified audit table.
- DBA_OBJ_AUDIT_OPTS → auditing options for objects.
- DBA_STMT_AUDIT_OPTS → statement audit options.
- DBA_PRIV_AUDIT_OPTS → privilege audit options.

Exercise 3.4

Part A – Conceptual Questions

1. Define Unified Auditing and explain two advantages it has over standard auditing.
2. Differentiate between:
 - Predefined unified audit policies
 - Custom audit policies
 - Fine-grained auditing (FGA)
3. Which data dictionary view provides access to the consolidated audit trail?
4. Why is archiving and purging audit trail records important for database performance?

Part B – Hands-On Tasks

You are given an Oracle Database instance UNI_AUDDDB. Perform the following:

1. Check Predefined Policies
 - List all predefined unified audit policies.

```
SELECT policy_name FROM audit_unified_policies;
```

2. Enable a Predefined Policy

- Enable the ORA_LOGON_FAILURES policy.

```
AUDIT POLICY ORA_LOGON_FAILURES;
```

3. Create and Enable a Custom Policy

- Create a policy named LOGIN_AUDIT that audits all successful logins.
- Enable this policy.

```
CREATE AUDIT POLICY login_audit ACTIONS LOGON;  
AUDIT POLICY login_audit;
```

4. Apply Fine-Grained Auditing (FGA)

- Create an FGA policy on the HR.EMPLOYEES table to log queries where salary > 15000.

```
BEGIN  
DBMS_FGA.ADD_POLICY(  
  object_schema => 'HR',  
  object_name   => 'EMPLOYEES',  
  policy_name   => 'fga_salary_check',  
  audit_condition => 'SALARY > 15000',  
  audit_column  => 'SALARY'  
);  
END;  
/
```

5. View Audit Records

- Query the unified audit trail for the last 10 audit entries.

```
SELECT event_timestamp, username, action_name  
FROM unified_audit_trail  
ORDER BY event_timestamp DESC  
FETCH FIRST 10 ROWS ONLY;
```

6. Archive and Purge Audit Data

- Initialize cleanup for the unified audit trail.
- Set last archive timestamp to 7 days ago.
- Purge records older than this.

```
EXEC DBMS_AUDIT_MGMT.INIT_CLEANUP(  
  audit_trail_type => DBMS_AUDIT_MGMT.AUDIT_TRAIL_UNIFIED,  
  default_cleanup_interval => 24);
```

```
EXEC DBMS_AUDIT_MGMT.SET_LAST_ARCHIVE_TIMESTAMP(  
  audit_trail_type => DBMS_AUDIT_MGMT.AUDIT_TRAIL_UNIFIED,  
  last_archive_time => SYSTIMESTAMP - 7);
```

```
EXEC DBMS_AUDIT_MGMT.CLEAN_AUDIT_TRAIL(  
  audit_trail_type => DBMS_AUDIT_MGMT.AUDIT_TRAIL_UNIFIED,  
  use_last_arch_timestamp => TRUE);
```

Part C – Reflection

- Discuss one security risk that could arise if auditing is disabled.
- Explain how Unified Auditing supports compliance with data protection regulations (e.g., GDPR, HIPAA).

References

- Johnson, Luanne. "Oracle Database - Wikipedia." *Wikipedia, the free encyclopedia*, 2019, https://en.wikipedia.org/wiki/Oracle_Database. Accessed 26 September 2025.
- Oracle. *Database concepts*. Oracle, 2025, <https://docs.oracle.com/en/database/oracle/oracle-database/21/cncpt/database-concepts.pdf>. Accessed Friday September 2025.
- Oracle. "1 Introduction to Oracle Database." *Oracle Help Center*, <https://docs.oracle.com/en/database/oracle/oracle-database/23/cncpt/introduction-to-oracle-database.html>. Accessed 26 September 2025.
- Oracle. "1 Introduction to Oracle Database." *Oracle Help Center*, https://docs.oracle.com/cd/E11882_01/server.112/e40540/intro.htm. Accessed 26 September 2025.
- Oracle. "Oracle Database Instance." *Database concepts*, September 2025, <https://docs.oracle.com/en/database/oracle/oracle-database/23/cncpt/oracle-database-instance.html#GUID-67247052-CE3F-44D2-BA3E-7067DEF4B6D5>. Accessed Friday September 2025.
- Oracle. "Oracle memory structure." *Memory architecture*, 2002, https://docs.oracle.com/cd/B10500_01/server.920/a96524/c08memor.htm. Accessed 26 September 2025.
- Oracle. "3 Oracle Grid Infrastructure." *Oracle Help Center*, https://docs.oracle.com/cd/E18185_01/doc/install.112/e16763/oraclerestart.htm. Accessed 26 September 2025.

Video reference for installation

 YouTube Tutorial: [Oracle Database 19c Installation on VirtualBox](#)